

Лабораторна робота №5. Створення вебзастосунка використовуючи фреймворк Django: розгортання серверу Django, робота з формами, валідація даних, підключення до бази даних та робота с моделями.

Завдання. Використовуючи досвід виконання та систему подання даних з попередньої лабораторної роботи завершити розробку засобами фреймворку Django вебзастосунок інформаційної системи для заданої предметної галузі. Для кожної сутності даних повинні бути реалізовані CRUD операції в залежності від ролі користувача.

ЗМІСТ

1. РОБОТА З ФОРМАМИ	5
1.1 Надсилання форм	5
1.1.1 Отримання простих даних	5
1.1.2 Отримання масивів	8
1.2 Визначення форм Django.....	11
1.2.1 Клас forms.Form	11
1.2.2 Отримання POST-запиту.....	13
1.3 Типи полів форми.....	15
1.3.1 Класи полів.....	15
1.3.2 Віджети Django	21
1.4 Налаштування форми та її полів.....	21
1.4.1 label.....	22
1.4.2 widget	23
1.4.3 Значення за замовчуванням	23
1.4.4 Порядок полів	24
1.4.5 help_text	24
1.4.6 Налаштування вигляду форми	25
1.5 Валідація даних	27
1.5.1 Правила валідації	28
1.5.2 Довжина тексту.....	29
1.5.3 Мінімальне та максимальне число.....	29
1.5.4 is_valid.....	29
1.6 Детальне налаштування полів форми	31
1.7 Стилiзація полів форм	35
2 МОДЕЛІ	41
2.1 Підключення до бази даних	41
2.1.1 Налаштування підключення до бази даних.....	41
2.1.2 Підтримувані СУБД	42
2.1.3 Параметри підключення.....	42
2.1.4 Міграції бази даних	44
2.2 Створення моделей	47
2.3 QuerySet API	53
2.4 Створення та отримання об'єктів моделі	57

2.4.1 Додавання даних.....	57
2.4.2 Отримання з БД.....	59
2.5 Редагування та видалення об'єктів моделі.....	64
2.5.1 Оновлення.....	65
2.5.2 Видалення	68
2.6 Фільтрація	69
2.6.1 exact та iexact.....	71
2.6.2 contains та icontains	71
2.6.3 in	72
2.6.4 gt, gte, lte, lt	72
2.6.5 startswith та istartswith.....	72
2.6.6 endswith та iendswith.....	72
2.6.7 range	73
2.6.8 isnull	73
2.6.8 regex та iregex	73
2.6.9 Специфікатори дат.....	74
2.6.10 Логічні оператори	76
2.7 Сортування.....	77
2.7.1 Функція <i>order_by()</i>	77
2.7.2 Функції <i>values()</i> та <i>values_list()</i>	78
2.8 Операції з множинами	80
2.8.1 Функція <i>distinct()</i>	80
2.8.2 Функція <i>union()</i>	81
2.8.3 Функція <i>intersection()</i>	83
2.8.4 Функція <i>difference()</i>	84
2.9 Отримання окремих об'єктів і перевірка їх наявності.....	85
2.9.1 Функції <i>latest()</i> та <i>earliest()</i>	85
2.9.2 Функції <i>first()</i> і <i>last()</i>	87
2.9.3 Функція <i>exists()</i>	88
2.9.4 Функція <i>contains()</i>	89
2.10 Агрегатні функції	90
2.10.1 Кількість об'єктів	90
2.10.2 Метод <i>aggregate()</i>	91
2.11 Виконання SQL-виразів.....	92
2.11.1 Функція <i>row()</i>	93
2.11.2 Передача параметрів.....	94
2.11.3 Виконання команд SQL.....	95

2.12 CRUD. Усі базові операції з моделями у веб-додатку	97
2.12.1 Визначення подань	97
2.12.2 Визначення шаблонів	99
2.12.3 Встановлення маршрутів	101
2.13 Відношення один до багатьох (One to Many).....	104
2.13.1 Розробка моделі	104
2.13.2 Операції з моделями	106
2.14 Практичний приклад зв'язку один до багатьох	107
2.15 Відношення багато до багатьох (Many to Many)	114
2.15.1 Операції з моделями	115
2.15.2 Визначення проміжної сутності	116
2.15.3 Використання проміжної моделі	118
2.16 Практичний приклад зв'язку багато до багатьох	120
2.17 Відношення один до одного (One to one)	125
2.17.1 Створення моделі	125
2.17.2 Операції з моделями	127

1. РОБОТА З ФОРМАМИ

1.1 Надсилення форм

Однією з форм надсилення даних на сервер є надсилення за допомогою форм *html*, зазвичай у запиті типу *POST*. У Django загалом можна використовувати два підходи для роботи з формами. По-перше, можна працювати зі стандартними формами *html*. По-друге, Django також надає спеціальний функціонал для роботи з формами, який, можливо, в якихось моментах спрощує роботу з даними. У цьому розділі розглянемо обидва підходи. А в цій статті подивимося, як отримувати дані звичайних форм *html*.

1.1.1 Отримання простих даних

Зазвичай дані форм передаються на сервер у запиті типу *POST*. Для отримання подібних даних у класі *HttpRequest* визначено властивість *POST*. Наприклад, нехай у нас є такий шаблон *index.html*:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
</head>
<body>
  <h2>User form</h2>
  <form method="post" action="postuser/">
    {% csrf_token %}
    <p>Name:<br> <input name="name" /></p>
    <p>Age:<br> <input name="age" type="number" /></p>
    <input type="submit" value="Send" />
  </form>
</body>
</html>
```

Тут визначено форму умовно для введення даних користувача, яка в запиті типу *POST* (атрибут *method="post"*) відправляє дані за адресою *"postuser/"* (атрибут *action="postuser/"*).

На формі визначено два поля введення. Перше поле призначене для введення імені користувача. Друге поле - для введення віку користувача.

Також всередині форми використовується тег *{% csrf_token %}*. Він дає змогу захистити застосунок від *CSRF*-атак, додаючи у форму у вигляді прихованого поля *csrf*-токен. Крім того, Django за замовчуванням вимагає наявності цього токена в одержуваних даних у запиті *POST*.

Для надсилання форми та отримання її даних визначимо у файлі *views.py* такі функції:

```
from django.shortcuts import render
from django.http import HttpResponseRedirect

def index(request):
    return render(request, "index.html")

def postuser(request):
    # отримуємо з даних запиту POST відправлені через форму дані
    name = request.POST.get("name", "Undefined")
    age = request.POST.get("age", 1)
    return HttpResponseRedirect(f"<h2>Name: {name} Age: {age}</h2>")
```

У поданні *index()* повертається шаблон, який містить форму введення.

У поданні *postuser()* отримуємо через словник *request.POST* відправлені з форми дані. У цьому словнику за ключем можна отримати значення елемента. При цьому як ключі виступають назви полів форм (значення атрибутів *name* елементів форми):

```
<input name="age" type="number" />
```

Так, у цьому випадку назва поля (значення атрибута *name*) дорівнює *"age"*. Відповідно в *request.POST* за цим ім'ям ми можемо отримати його значення:

```
age = request.POST.get("age", 1)
```

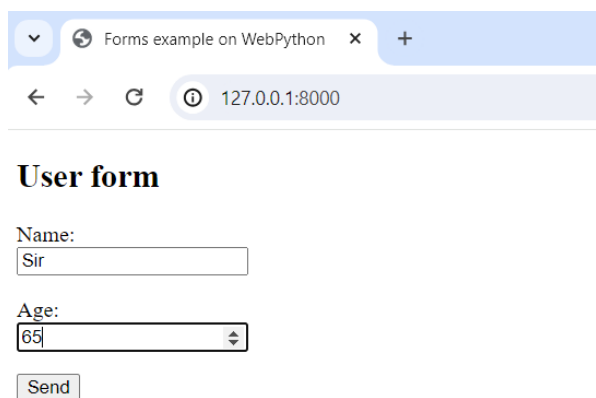
Якщо з якихось причин дані з ключем *"age"* у запиті відсутні, то повертається значення за замовчуванням - 1.

Далі у файлі *urls.py* зв'яжемо ці функції з маршрутами:

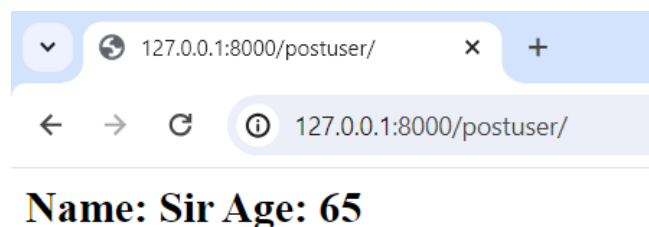
```
from django.urls import path
from lab_form import views

urlpatterns = [
    path("", views.index),
    path("postuser/", views.postuser),
]
```

І після отримання даних форми їх відправляють назад клієнту:



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000'. The page title is 'Forms example on WebPython'. The main content area is titled 'User form' and contains a form with two input fields: 'Name:' with the value 'Sir' and 'Age:' with the value '65'. A 'Send' button is located below the form.



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/postuser/'. The page content displays the result of the form submission: 'Name: Sir Age: 65'.

Рисунок 1.1 – Результат відправлення даних клієнту

1.1.2 Отримання масивів

Ускладнимо завдання і додамо у форму на сторінці *index.html* кілька полів, які представлятимуть масив:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
</head>
<body>
  <h2>User form</h2>
  <form method="post" action="postuser/">
    {% csrf_token %}
    <p>Name:<br /> <input name="name" /></p>
    <p>Age:<br /> <input name="age" type="number" /></p>
    <p>
      Languages:<br />
      <input name="languages" /><br />
      <input name="languages" /><br />
      <input name="languages" /><br />
    </p>
    <input type="submit" value="Send" />
  </form>
</body>
</html>
```

Тут практично та сама форма, тільки додано три поля для введення мови програмування. Причому кожне з цих полів має одне й те саме ім'я - *"languages"*. Завдяки цьому під час надсилання форми в даних запиту буде сформовано список *languages* з даних, введених у ці поля.

У файлі *views.py* змінимо функцію *postuser* для отримання масиву *languages*:

```
from django.shortcuts import render
from django.http import HttpResponse

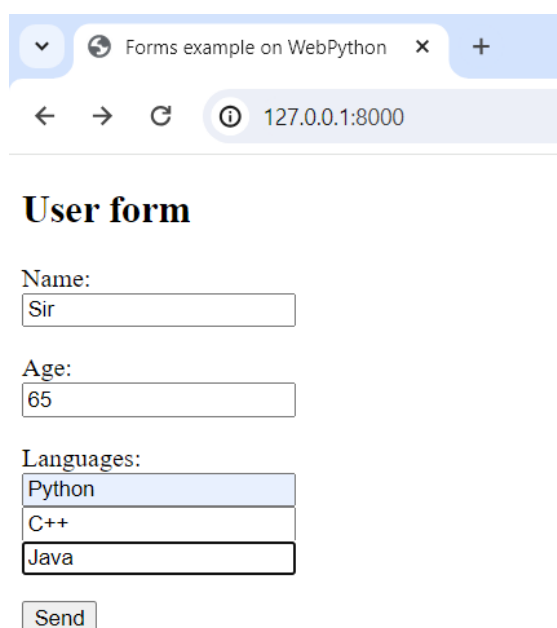
def index(request):
    return render(request, "index.html")

def postuser(request):
    # отримуємо з рядка запиту ім'я користувача
    name = request.POST.get("name", "Undefined")
    age = request.POST.get("age", 1)
    langs = request.POST.getlist("languages", ["python"])

    return HttpResponse(f"""
        <div>Name: {name} Age: {age}</div>
        <div>Languages: {langs}</div>
    """)
```

Ключовим компонентом при отриманні списку даних із запиту є метод *getlist()*, який працює так само, як і *get()*, тільки повертає список. Якщо в запиті не виявиться даних із ключем *languages*, то повертаємо список *["python"]*.

Отримавши список, ми можемо щось зробити з його елементами, перебрати, звернутися до елементів за індексом тощо. Але в цьому випадку просто передаємо весь список у сформовану відповідь.



Forms example on WebPython

127.0.0.1:8000

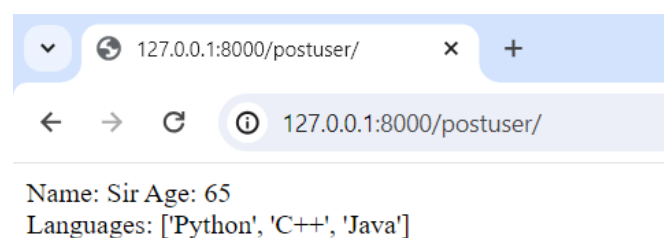
User form

Name:
Sir

Age:
65

Languages:
Python
C++
Java

Send



127.0.0.1:8000/postuser/

Name: Sir Age: 65
Languages: ['Python', 'C++', 'Java']

Рисунок 1.2 – Результат отримання масиву даних

Подібним чином можна передавати значення масиву полів інших типів, або полів, які представляють набір елементів, наприклад, елемента *select*, який підтримує множинний вибір:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
</head>
<body>
  <h2>User form</h2>
  <form method="post" action="postuser/">
    {% csrf_token %}
    <p>Name: <br />
      <input name="name" />
    </p>
    <p>Age: <br />
      <input name="age" type="number" />
    </p>
    <p>
      Languages:<br />
      <select multiple name="languages">
        <option>Python</option>
        <option>JavaScript</option>
        <option>C++</option>
        <option>Java</option>
      </select>
    </p>
    <input type="submit" value="Send" />
  </form>
</body>
</html>
```

The image shows two browser windows side-by-side. The left window, titled 'Forms example on WebPython', displays a web form titled 'User form'. The form contains three input fields: 'Name:' with the value 'Alex', 'Age:' with the value '45', and 'Languages:' which is a dropdown menu currently showing 'Python', 'JavaScript', 'C++', and 'Java'. Below these fields is a 'Send' button. The right window, titled '127.0.0.1:8000/postuser/', shows the result of the form submission. It displays the text 'Name: Alex Age: 45' and 'Languages: ['Python', 'Java']'.

Рисунок 1.3 – Результат застосування елементу select

1.2 Визначення форм Django

Django надає спеціальні можливості для роботи з формами, які дають змогу визначати функціональність форм в одному місці і використовувати багаторазово в різних місцях, спрощують валідацію даних, допомагають пов'язувати форми з моделями і багато іншого.

1.2.1 Клас `forms.Form`

Кожна форма визначається у вигляді окремого класу, який розширює клас `forms.Form`. Класи розміщуються всередині проєкту, де вони використовуються. Нерідко вони поміщаються в окремий файл, який називається, наприклад, `forms.py`. Однак також форми можуть розміщуватися всередині вже наявних у додатку файлів, наприклад, у `views.py` або `models.py`.

Наприклад, створимо в додатку новий файл `forms.py` і помістимо в нього такий код:

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField()
    age = forms.IntegerField()
```

Клас форми називається *UserForm*. Він визначає два поля. Поле *name* представляє тип *forms.CharField* і буде генерувати поле *input type="text"*. Поле *age* представляє тип *forms.IntegerField* і буде генерувати поле *input type="number"*. Тобто перше поле для введення тексту, а друге для введення чисел.

Далі у файлі *views.py* визначимо таке подання:

```
from django.shortcuts import render
from .forms import UserForm

def index(request):
    userform = UserForm()
    return render(request, "index.html", {"form": userform})
```

Тут об'єкт форми передається в шаблон *index.html* у вигляді змінної *form*.

У файлі *urls.py* пропишемо маршрут для цього подання:

```
from django.urls import path
from lab_form import views

urlpatterns = [
    path("", views.index),
]
```

І визначимо наступний шаблон *index.html*:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Forms example on WebPython</title>
</head>
<body>
    <table>
        {{ form }}
    </table>
</body>
</html>
```

У підсумку зі змінної *form* генеруватиметься такий код *html*:

```
<tr><th><label for="id_name">Name:</label></th><td><input type="text"
name="name" required id="id_name" /></td></tr>
<tr><th><label for="id_age">Age:</label></th><td><input type="number"
name="age" required id="id_age" /></td></tr>
```

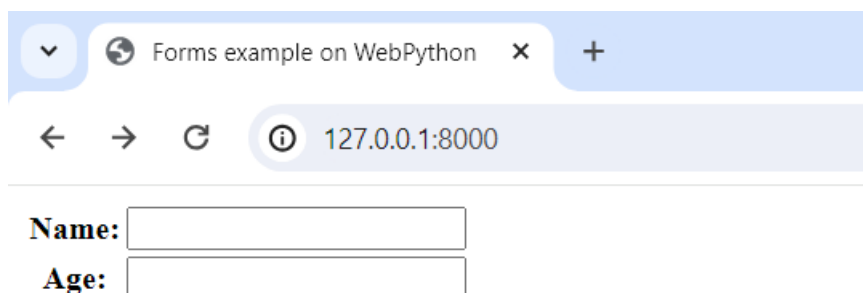


Рисунок 1.4 – Результат визначення форми

1.2.2 Отримання POST-запиту

Тепер за допомогою форм Django визначимо повнофункціональну форму, з якої можна відправляти дані на сервер.

Спочатку змінимо шаблон *index.html*:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
</head>
<body>
  <form method="POST">
    {% csrf_token %}
    <table>
      {{ form }}
    </table>
    <input type="submit" value="Send" >
  </form>
</body>
</html>
```

Для створення форми тут використано стандартний елемент `html <form>`. На початку форми розміщено вбудований тег Django `{% csrf_token %}`, який дає змогу захистити застосунок від *CSRF*-атак, додаючи у форму у вигляді прихованого поля *csrf*-токен.

Внизу форми визначено кнопку для надсилання цієї форми на сервер.

Тепер змінимо подання у файлі `views.py`:

```
from django.shortcuts import render
from django.http import HttpResponse
from .forms import UserForm

def index(request):
    if request.method == "POST":
        name = request.POST.get("name")
        age = request.POST.get("age")
        return HttpResponse(f"<h2>Hi, {name}, your age is {age}</h2>")
    else:
        userform = UserForm()
        return render(request, "index.html", {"form": userform})
```

Оскільки в шаблоні форма за замовчуванням надсилатиметься на ту саму адресу, то подання обробляє одразу два типи запитів *GET* і *POST*. Для визначення типу запиту перевіряємо значення `request.method`.

Якщо запит типу *POST*, то спочатку створюємо об'єкт `UserForm`, наповнюючи його даними, які прийшли в запиті через `request.POST`. Тобто це і будуть відправлені дані форми. Потім ми можемо отримати ці дані окремо для кожного поля форми. Після цього відправляємо користувачеві повідомлення через об'єкт `HttpResponse`. В принципі тут можна було б зробити переадресацію або використовувати інший шаблон для генерації відповіді.

Якщо запит має тип *GET*, то просто відправляємо форму для введення даних.

Таким чином, при зверненні до додатка ми спочатку побачимо форму введення. Введемо в неї деякі дані:

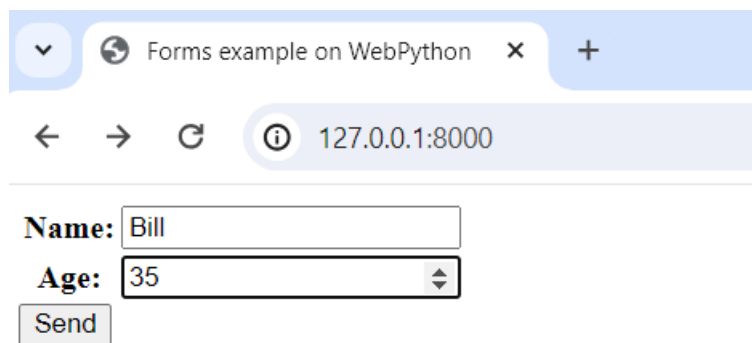


Рисунок 1.5 – Введення даних

Після натискання на кнопку введені дані в запиті *POST* знову ж таки підуть до представлення *index()*, яке опрацює їх і у відповідь надішле користувачеві повідомлення із введеним ім'ям:

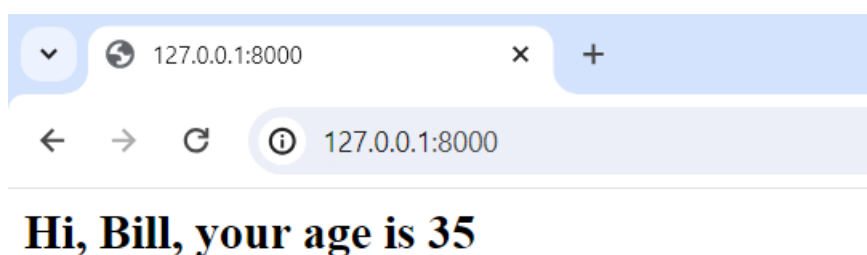


Рисунок 1.6 – Результат отримання даних

1.3 Типи полів форми

1.3.1 Класи полів

У формах Django ми можемо використовувати такі класи для створення полів форм:

- *BooleanField*: створює поле `<input type="checkbox">`. Повертає значення *Boolean: True* - якщо прапорець позначений і *False* - якщо прапорець не позначений.
- *NullBooleanField*: створює наступну розмітку:

```
<select>
<option value="1" selected="selected">Unknown</option>
<option value="2">Yes</option>
<option value="3">No</option>
</select>
```

- *CharField*: призначений для введення тексту і створює таку розмітку:

```
<input type="text">
```

Приймає такі параметри:

- *max_length*: максимальна довжина тексту, що вводиться
- *min_length*: мінімальна довжина тексту, що вводиться
- *strip*: при значенні *True* (за замовчуванням) початкові та кінцеві пробіли видаляються
- *empty_value*: значення, що застосовується для представлення порожнього поля

- *EmailField*: призначений для введення адреси електронної пошти і створює таку розмітку:

```
<input type="email">
```

Застосовує ті самі параметри, що й *CharField*.

- *GenericIPAddressField*: призначений для введення IP-адреси у форматі IPv4 або IPv6 і створює таку розмітку:

```
<input type="text">
```

- *RegexField* (*regex="регулярний_вираз"*): призначене для введення тексту, який має відповідати певному регулярному виразу. Створює текстове поле:

```
<input type="text">
```


– *SlugField()*: призначений для введення тексту, який умовно називається "slug", тобто послідовність символів у нижньому регістрі, чисел, дефісів і знаків підкреслення. Створює текстове поле:

```
<input type="text">
```

– *URLField()*: призначений для введення посилань. Створює таке поле:

```
<input type="url">
```

– *UUIDField()*: призначене для введення *UUID* (універсального унікального ідентифікатора). Створює таке поле:

```
<input type="text">
```

– *ComboField(fields=[field1, field2,...])*: аналогічний до звичайного текстового поля, за тим винятком, що вимагає, щоб текст, який вводиться, відповідав вимогам тих полів, які передаються через параметр *fields*. Створює таке поле:

```
<input type="text">
```

– *MultiValueField(fields=[field1, field2,...])*: призначений для створення складних компонентів, що складаються з декількох полів.

– *FilePathField(path="каталог файлів")*: створює список *select*, який містить усі папки та файли в певному каталозі:

```
<select>
  <option value="file1">folder/file1</option>
  <option value="file2">folder/file2</option>
  <option value="file3">folder/file3</option>
  //.....
</select>
```

Параметри поля:

- *path*: абсолютний шлях до каталогу

- *recursive*: якщо дорівнює *False* (значення за замовчуванням), то елемент відображає файли тільки безпосередньо з цієї папки. Якщо дорівнює *True*, то також відображається вміст вкладених папок
- *match*: регулярний вираз, якому повинні відповідати файли для відображення
- *allow_files*: представляє значення *True* (значення за замовчуванням) або *False*. Вказує, чи повинні бути включені файли за певним шляхом
- *allow_folders*: представляє значення *True* (значення за замовчуванням) або *False*. Вказує, чи повинні бути включені папки за певним шляхом

– *FileField()*: призначений для вибору файлу. Створює таке поле:

```
<input type="file">
```

– *ImageField()*: призначений також для вибору файлу, але при цьому додає низку додаткових можливостей. Створює таке поле:

```
<input type="file">
```

– *DateField()*: призначений для встановлення дати. У створюване поле вводиться текст, який може бути сконвертований у дату, наприклад, *2017-12-25* або *11/25/17*. Створює таке поле:

```
<input type="text">
```

– *TimeField()*: призначений для введення часу, наприклад, *14:30:59* або *14:30*. Створює таке поле:

```
<input type="text">
```

– *DateTimeField()*: призначене для введення дати і часу, наприклад, *2017-12-25 14:30:59* або *11/25/17 14:30*. Створює таке поле:

```
<input type="text">
```

– *DurationField()*: призначене для тимчасового проміжку. Текст, що вводиться, має відповідати формату "DD HH:MM:SS", наприклад, 2 1:10:20 (2 дні 1 година 10 хвилин 20 секунд). Створює таке поле:

```
<input type="text">
```

– *SplitDateTimeField()*: створює два текстових поля для введення відповідно дати і часу:

```
<input type="text" name="_0" >  
<input type="text" name="_1" >
```

– *IntegerField()*: призначений для введення чисел. Створює таке поле:

```
<input type="number">
```

Параметри поля:

- *max_value*: максимально можливе значення
 - *min_value*: мінімально можливе значення
 - *step_size*: крок приросту числа
- *DecimalField()*: призначений для введення чисел. Створює таке поле:

```
<input type="number">
```

Параметри поля:

- *max_value*: максимально можливе значення
 - *min_value*: мінімально можливе значення
 - *max_digits*: максимальна кількість цифр у числі
 - *decimal_places*: максимальна кількість знаків після коми
 - *step_size*: крок приросту числа
- *FloatField()*: призначений для введення чисел. Створює таке поле:

```
<input type="number">
```

Параметри поля:

- *max_value*: максимально можливе значення
- *min_value*: мінімально можливе значення
- *step_size*: крок приросту числа

– *ChoiceField(choices=кортеж_кортежів)*: генерує список *select*, кожен з його елементів формується на основі окремого кортежу. Наприклад, таке поле:

```
languages = forms.ChoiceField(choices=((1, "English"), (2, "German"), (3, "French")))
```

генеруватиме наступну розмітку:

```
<select name="languages" id="id_languages">
  <option value="1">English</option>
  <option value="2">German</option>
  <option value="3">French</option>
</select>
```

– *TypedChoiceField(choices=кортеж_кортежів, coerce=функція_перетворення, empty_value=None)*: також генерує список *select* на основі кортежу. Однак додатково приймає функцію перетворення, яка перетворює кожен елемент. І також приймає параметр *empty_value*, який вказує на значення за замовчуванням.

– *MultipleChoiceField(choices=кортеж_кортежів)*: також генерує список *select* на основі кортежу, як і *forms.ChoiceField*, додаючи до створюваного поля атрибут *multiple="multiple"*. Тобто список підтримує множинний вибір.

– *TypedMultipleChoiceField(choices=кортеж_кортежів, coerce=функція_перетворення, empty_value=None)*: аналог *TypedChoiceField* для списку з множинним вибором.

– *JSONField()*: поле, яке приймає дані у форматі *json*.

Приймає два додаткові параметри:

- *encoder*: клас похідний від *json.JSONEncoder* для серіалізації даних у формат *JSON*. Значення за замовчуванням - *json.JSONEncoder*.
- *decoder*: клас похідний від *json.JSONDecoder* для десеріалізації даних із формату *JSON*. Значення за замовчуванням - *json.JSONDecoder*.

1.3.2 Віджети Django

Вище розглянуті поля під час генерації розмітки використовують певні віджети з пакета *forms.widgets*. Наприклад, клас *CharField* використовує віджет *forms.widgets.TextInput*, а *ChoiceField* використовує *forms.widgets.Select*. Але є низка віджетів, які за замовчуванням не використовуються полями форм, але тим не менш ми їх можемо задіяти:

- *PasswordInput*: генерує поле для введення пароля `<input type="password">`
- *HiddenInput*: генерує приховане поле `<input type="hidden">`
- *MultipleHiddenInput*: генерує набір прихованих полів
- *TextArea*: генерує багаторядкове текстове поле `<textarea></textarea>`
- *RadioSelect*: генерує список перемикачів (радіокнопок) `<input type="radio">`
- *CheckboxSelectMultiple*: генерує список прапорців `<input type="checkbox" >`
- *TimeInput*: генерує поле для введення часу (наприклад, 12:41 або 12:41:32)
- *SelectDateWidget*: генерує три поля *select* для вибору дня, місяця та року
- *SplitHiddenDateTimeWidget*: використовує приховане поле для зберігання

дати та часу

- *FileInput*: генерує поле для вибору файлу.

Наприклад, *CharField* за промовчанням використовує віджет *TextInput* для створення однорядкового текстового поля. Але що якщо ми хочемо створити багаторядкове текстове поле? Для цієї мети застосуємо віджет *TextArea*:

```
comment = forms.CharField(widget=forms.Textarea)
```

1.4 Налаштування форми та її полів

1.4.1 label

Властивість *label* дає змогу встановити текстову мітку, яка відображається поруч із полем. За замовчуванням вона відображає назву самого поля з великої літери. Наприклад:

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField(label="Ім'я")
    age = forms.IntegerField(label="Вік")
```

Ця форма сформує такі поля введення:

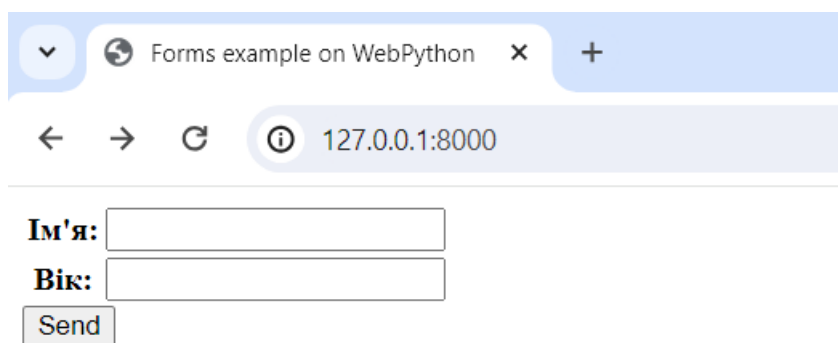
A screenshot of a web browser window. The title bar shows a single tab titled "Forms example on WebPython". The address bar displays the URL "127.0.0.1:8000". Below the address bar, the form is rendered. It consists of two text input fields. The first field is preceded by the label "Ім'я:" and the second by "Вік:". Below these fields is a button labeled "Send".

Рисунок 1.7 – Результат дії елемента label

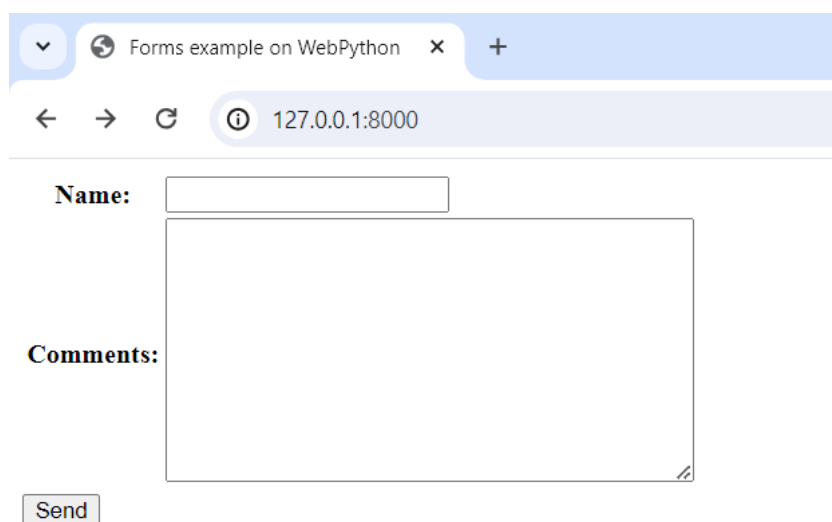
1.4.2 widget

Параметр *widget* дає змогу задати віджет, який буде використовуватися для генерації розмітки *html*:

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField(label="Name")
    comment = forms.CharField(label="Comments", widget=forms.Textarea)
```

За замовчуванням поле *CharField* використовує віджет *forms.widgets.TextInput*, який створює однорядкове текстове поле. Однак якщо нам треба створити багаторядкове текстове поле, то необхідно скористатися віджетом *forms.Textarea*:



The screenshot shows a web browser window with the title 'Forms example on WebPython'. The address bar displays '127.0.0.1:8000'. The form contains two fields: 'Name' with a single-line text input, and 'Comments' with a multi-line text area. A 'Send' button is located at the bottom left of the form.

Рисунок 1.8 – Результат дії *widget*

1.4.3 Значення за замовчуванням

За допомогою параметра *initial* можна встановити значення за замовчуванням.

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField(initial="undefined")
    age = forms.IntegerField(initial=18)
```

1.4.4 Порядок полів

Поля введення відображаються на веб-сторінці в тому порядку, в якому вони визначені в класі форми. За допомогою властивості *field_order* можна перевизначити порядок, як у класі форми:

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField()
    age = forms.IntegerField()
    field_order = ["age", "name"]
```

Так і під час визначення об'єкта форми в поданні:

```
from django.shortcuts import render
from django.http import HttpResponse
from .forms import UserForm

def index(request):
    userform = UserForm(field_order = ["age", "name"])
    return render(request, "index.html", {"form": userform})
```

1.4.5 help_text

Параметр *help_text* встановлює підказку поруч із полем введення:

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField(help_text="Enter your name")
    age = forms.IntegerField(help_text="Enter your age")
```

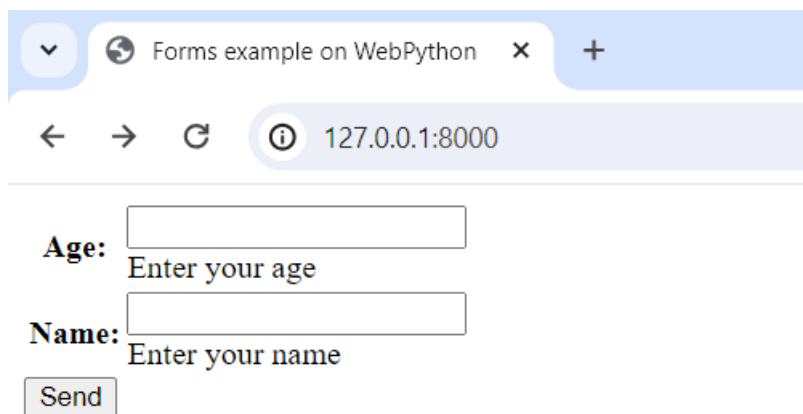



Рисунок 1.9 – Результат дії елемента *help_text*

1.4.6 Налаштування вигляду форми

За допомогою спеціальних методів можна налаштувати загальне відображення форми:

- *as_table()*: відображення у вигляді таблиці
- *as_ul()*: відображення у вигляді списку
- *as_p()*: кожне поле форми відображається в окремому параграфі
- *as_div()*: кожне поле форми відображається в окремому блоці *div*

Застосування методів у файлі *index.html*:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
</head>
<body>
  <h2>as_table</h2>
  <form method="POST">
    {% csrf_token %}
    <table>
      {{ form.as_table }}
    </table>
    <input type="submit" value="Send" >
  </form>
  <h2>as_ul</h2>
  <form method="POST">
    {% csrf_token %}
    <ul>
      {{ form.as_ul }}
    </ul>
    <input type="submit" value="Send" >
  </form>
  <h2>as_p</h2>
  <form method="POST">
    {% csrf_token %}
    <div>
      {{ form.as_p }}
    </div>
    <input type="submit" value="Send" >
  </form></body>
</html>
```

The screenshot shows a web browser window with the title 'Forms example on WebPython'. The address bar shows '127.0.0.1:8000'. Below the browser window, three form examples are displayed:

as_table

Age: Enter your age

Name: Enter your name

as_ul

- Age: Enter your age
- Name: Enter your name

as_p

Age: Enter your age

Name: Enter your name

Рисунок 1.10 – Результат змін налаштування форми

1.5 Валідація даних

Теоретично користувач може ввести у форму які завгодно дані та надіслати їх. Однак не всі дані бувають доречними або коректними. Наприклад, у полі для віку користувач може ввести від'ємне число, що навряд чи може вважатися коректним віком. У зв'язку з цим для перевірки коректності введених даних використовується механізм валідації.

1.5.1 Правила валідації

Основним елементом валідації є правила, які задають параметри коректності даних, що вводяться. Наприклад, для всіх полів за замовчуванням встановлюється обов'язковість введення значення. І під час генерації *html*-коду для поля введення встановлюється атрибут *required*. І якщо ми спробуємо надіслати форму, якщо в якесь із її полів не введено жодного значення, то ми отримаємо помилку:

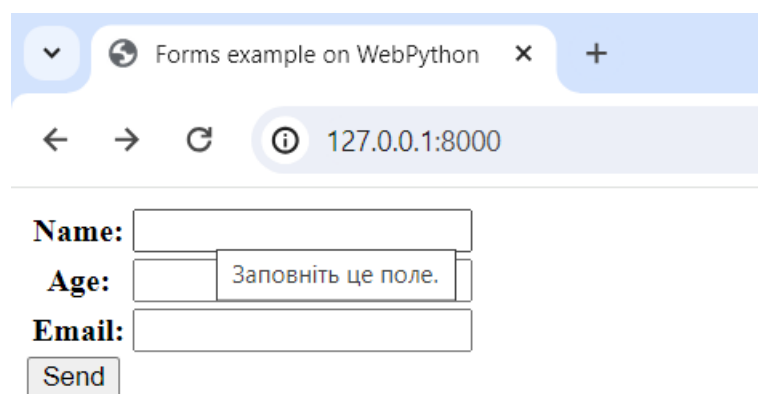


Рисунок 1.11 – Попередження про необхідність введення даних

І нам треба буде обов'язково ввести якесь значення в незаповнене поле. Однак це не завжди потрібно. Припустимо, одне поле може мати, а може не мати значення. У цьому разі ми можемо відключити атрибут *required*:

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField()
    age = forms.IntegerField(required=False)
    email = forms.EmailField(required=False)
```

1.5.2 Довжина тексту

Для полів, які вимагають введення тексту, наприклад, *CharField*, *EmailField* та ін., за допомогою параметрів *max_length* і *min_length* можна задати відповідно максимальну і мінімальну довжину тексту, що вводиться, в символах.

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField(min_length=2, max_length=20)
    age = forms.IntegerField(required=False)
    email = forms.EmailField(required=False, min_length=7)
```

Під час генерації розмітки для полів введення встановлюватимуться атрибути *maxlength* і *minlength*.

1.5.3 Мінімальне та максимальне число

Для об'єктів *IntegerField*, *DecimalField* і *FloatField* можна встановлювати параметри *max_value* і *min_value*, які задають відповідно максимально допустиме і мінімально допустиме значення.

DecimalField додатково може приймати ще параметр *decimal_places*, який вказує на максимальну кількість знаків після коми.

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField()
    age = forms.IntegerField(min_value=1, max_value=100)
    weight = forms.DecimalField(min_value=3, max_value=200, decimal_places=2)
```

1.5.4 is_valid

Вище розглянуті атрибути дають змогу валідувати значення під час введення на стороні клієнта. Однак практично користувачі, маючи певні навички, можуть все одно надіслати форму зі свідомо некоректними даними. Наприклад, через Барбарук Віктор Миколайович. Сучасні технології розробки WEB-застосунків з використанням мови Python

інструменти для розробників у веб-браузері можна підправити вихідний код форми, додавши їй атрибут *novalidate*, який відключає клієнтську валідацію. Тому перевірку на валідність даних також треба визначати на стороні сервера. Для цього у форми викликається метод *is_valid()*, який повертає *True*, якщо дані коректні, і *False* - якщо дані некоректні. Щоб використовувати цей метод, треба створити об'єкт форми і передати їй дані, що прийшли із запиту.

Отже, визначимо таке подання у файлі *views.py*:

```
from django.shortcuts import render
from django.http import HttpResponse
from .forms import UserForm

def index(request):
    if request.method == "POST":
        userform = UserForm(request.POST)
        if userform.is_valid():
            name = userform.cleaned_data["name"]
            return HttpResponse(f"<h2>Hello, {name}</h2>")
        else:
            return HttpResponse("Invalid data")
    else:
        userform = UserForm()
        return render(request, "index.html", {"form": userform})
```

Якщо приходить *POST*-запит, то на початку заповнюємо форму даними, що прийшли:

```
userform = UserForm(request.POST)
```

Потім перевіряємо їхню коректність:

```
if userform.is_valid():
```

Після перевірки на валідність ми можемо отримати дані через об'єкт *cleaned_data* (якщо дані коректні):

```
name = userform.cleaned_data["name"]
```

Якщо дані некоректні, можна передбачити альтернативний висновок:

```
return HttpResponse("Invalid data")
```

Для тестування форми можна встановити в неї атрибут *novalidate*:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
</head>
<body>
  <form method="POST" novalidate>
    {% csrf_token %}
    <table>
      {{ form }}
    </table>
    <input type="submit" value="Send" >
  </form>
</body>
</html>
```



Рисунок 1.12 – Результат валідації даних

1.6 Детальне налаштування полів форми

Форма і поля допускають встановлення низки параметрів, які дають змогу частково кастомізувати відображення полів і форми. Проте цього нерідко буває недостатньо. Наприклад, необхідно застосувати стилізацію або додати поруч із полем введення який-небудь спеціальний текст. І Django дає нам змогу докорінно змінити всю композицію створюваних полів.

Зокрема, у шаблоні компонента ми можемо звернутися до кожного окремого поля форми через назву форми: *form.назва_поля*. За назвою поля ми можемо отримати безпосередньо генерований ним елемент-*html* без зовнішніх написів і якогось додаткового коду. Крім того, кожне поле має низку асоційованих із ним значень:

- *form.назва_поля.name*: повертає назву поля
- *form.назва_поля.value*: повертає значення поля, яке йому було передано за замовчуванням
- *form.назва_поля.label*: повертає текст мітки, що генерується поруч із полем
- *form.name_поля.id_for_label*: повертає *id* для поля, яке за замовчуванням створюється за схемою *id_ім'яполя*.
- *form.назва_поля.auto_id*: повертає *id* для поля, яке за замовчуванням створюється за схемою *id_ім'яполя*.
- *form.назва_поля.label_tag*: повертає елемент *label*, який представляє мітку поруч із полем
- *form.назва_поля.help_text*: повертає текст підказки, асоційований із полем
- *form.назва_поля.errors*: повертає помилки валідації, пов'язані з полем
- *form.назва_поля.css_classes*: повертає *CSS*-класи поля
- *form.назва_поля.as_hidden*: генерує для поля розмітку у вигляді прихованого поля `<input type="hidden">`
- *form.название_поля.is_hidden*: повертає *True* или *False* залежно від того, чи є поле прихованим, чи ні
- *form.название_поля.as_text*: генерує для поля розмітку у вигляді текстового поля `<input type="text">`
- *form.название_поля.as_textarea*: генерує для поля розмітку у вигляді `<textarea></textarea>`

– `form.название_поля.as_widget`: повертає віджет Django, асоційований з полем

Так, щоб отримати текст на мітці поля, яке називається *age*, нам треба використовувати вираз `form.age.label`.

Наприклад, візьмемо найпростішу форму:

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField()
    age = forms.IntegerField()
```

У поданні передамо цю форму в шаблон:

```
from django.shortcuts import render
from django.http import HttpResponse
from .forms import UserForm

def index(request):
    userform = UserForm()
    if request.method == "POST":
        userform = UserForm(request.POST)
        if userform.is_valid():
            name = userform.cleaned_data["name"]
            return HttpResponse(f"<h2>Hello, {name}</h2>")
    return render(request, "index.html", {"form": userform})
```

І в шаблоні *index.html* пропишемо використання полів форми:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Forms example on WebPython</title>
</head>
<body>
    <form method="POST" novalidate>
        {% csrf_token %}
        <div>
            {% for field in form %}
            <div class="form-group">
                {{field.label_tag}}
                <div>{{field}}</div>
                <div class="error">{{field.errors}}</div>
            </div>
            {% endfor %}
        </div>
        <p><input type="submit" value="Send" ></p>
    </form>
</body>
</html>
```

Фактично форма являє собою набір полів, і за допомогою виразу *{% for field in form %}* ми пробігаємося кожним полем на формі та можемо керувати його відображенням - відображенням власне поля і пов'язаних із ним атрибутів - помилок, тексту підказки, мітки тощо.

Наприклад, після надсилання некоректних даних ми отримаємо таку веб-сторінку:

The screenshot shows a web browser window with the title 'Forms example on WebPython'. The address bar displays '127.0.0.1:8000'. The form contains two input fields. The first field is labeled 'Name:' and has a red error message below it: '• This field is required.' The second field is labeled 'Age:' and also has a red error message below it: '• This field is required.' At the bottom of the form is a button labeled 'Send'.

Рисунок 1.13 – Результат налаштування форми

Одне поле може містити кілька помилок. У цьому разі можна використовувати тег `for` для їх послідовного виведення:

```
{% for error in field.errors %}
    <div class="alert alert-danger">{{error}}</div>
{% endfor %}
```

1.7 Стилiзація полів форм

Поля форми застосовують деякі стилі за замовчуванням. Якщо ж ми хочемо застосувати до них якісь власні стилі та класи, то нам треба використовувати низку механізмів.

Насамперед ми можемо вручну виводити кожне поле і визначати правила стилізації для цього поля або оточуючих його блоків. Візьмемо найпростішу форму:

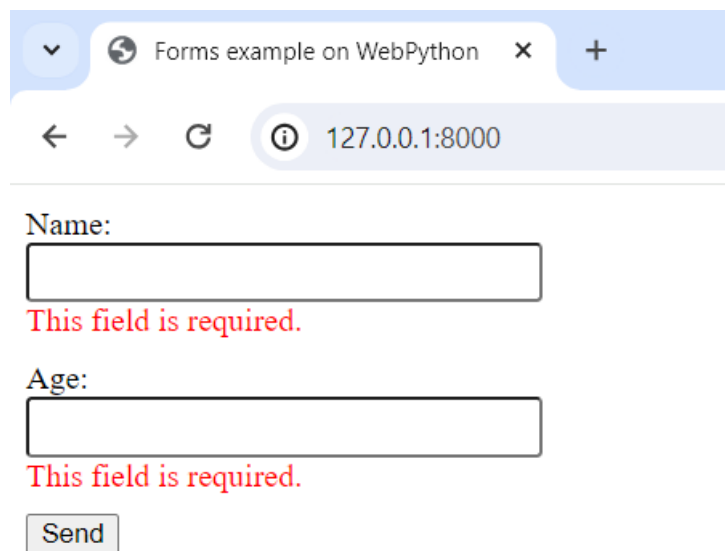
```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField(min_length=3)
    age = forms.IntegerField(min_value=1, max_value=100)
```

У шаблоні пропишемо її використання:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
  <style>
    .alert{color:red}
    .form-group{margin: 10px 0;}
    .form-group input{width:250px;height: 25px;border-radius:3px;}
  </style>
</head>
<body class="container">
  <form method="POST" novalidate>
    {% csrf_token %}
    <div>
      {% for field in form %}
      <div class="form-group">
        {{field.label_tag}}
        <div>{{field}}</div>
        {% if field.errors%}
        {% for error in field.errors %}
        <div class="alert alert-danger">
          {{error}}
        </div>
        {% endfor %}
        {% endif %}
      </div>
      {% endfor %}
    </div>
    <input type="submit" value="Send" >
  </form>
</body>
</html>
```

Результатом під час надсилання форми з помилками:



The screenshot shows a web browser window with the title 'Forms example on WebPython'. The address bar displays '127.0.0.1:8000'. The page contains a form with two input fields. The first field is labeled 'Name:' and the second is labeled 'Age:'. Both fields are empty and have a red error message 'This field is required.' displayed below them. A 'Send' button is located at the bottom of the form.

Рисунок 1.14 – Результат

Другий механізм представляють властивості форми *required_css_class* і *error_css_class*, які відповідно застосовують клас CSS до мітки, створюваної для поля форми, і до блоку асоційованих із ним помилок.

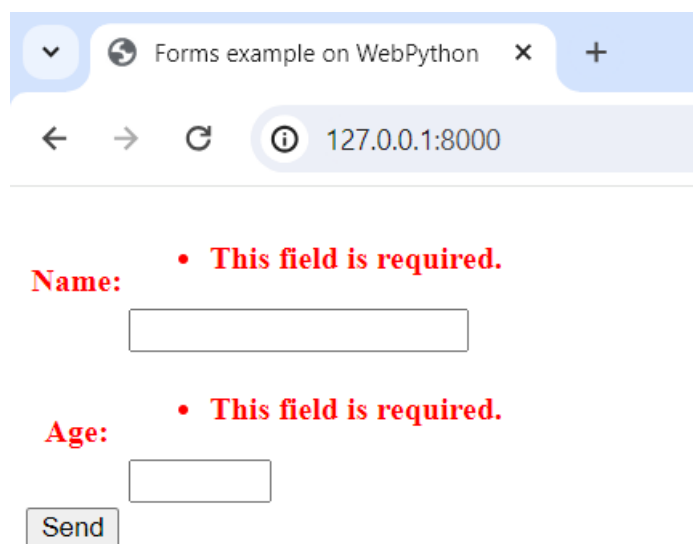
Наприклад, визначимо таку форму:

```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField(min_length=3)
    age = forms.IntegerField(min_value=1, max_value=100)
    required_css_class = "field"
    error_css_class = "error"
```

У цьому разі в шаблоні в нас мають бути визначені або підключені класи *"field"* і *"error"*:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
  <style>
    .field{font-weight:bold;}
    .error{color:red;}
  </style>
</head>
<body class="container">
  <form method="POST" novalidate>
    {% csrf_token %}
    <table>
      {{form}}
    </table>
    <input type="submit" value="Send" >
  </form>
</body>
</html>
```



The screenshot shows a web browser window with the title "Forms example on WebPython". The address bar shows the URL "127.0.0.1:8000". The page content displays a form with two input fields. The first field is labeled "Name:" and has a red error message "• This field is required." next to it. The second field is labeled "Age:" and also has a red error message "• This field is required." next to it. Below the "Age:" field is a "Send" button.

Рисунок 1.15 – Результат роботи

Але також можна було б комбінувати обидва способи:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
  <style>
    .field{font-weight:bold;}
    .error{color:red;}
  </style>
</head>
<body>
  <form method="POST" novalidate>
    {% csrf_token %}
    <div>
      {% for field in form %}
      <div class="row">
        {{field.label_tag}}
        <div>{{field}}</div>
        {% if field.errors%}
        <div class="error">{{field.errors}}</div>
        {% endif %}
      </div>
      {% endfor %}
    </div>
    <input type="submit" value="Send" >
  </form>
</body>
</html>
```

Третій механізм стилізації представляє встановлення класів і стилів через віджети:

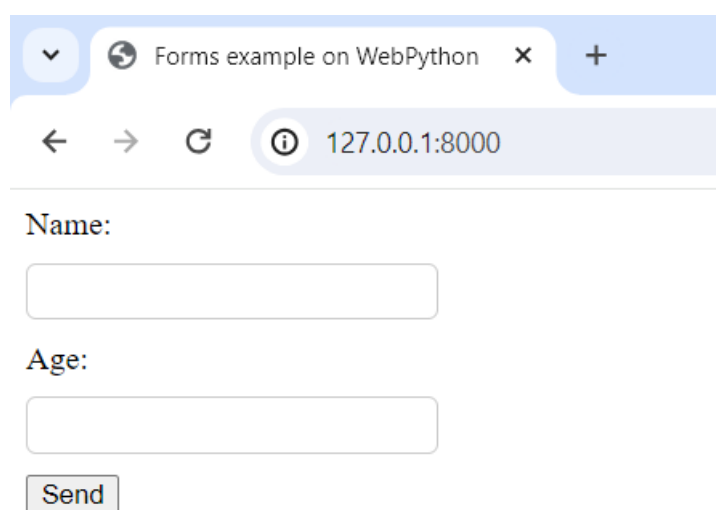
```
from django import forms

class UserForm(forms.Form):
    name = forms.CharField(widget=forms.TextInput(attrs={"class":"myfield"}))
    age = forms.IntegerField(widget=forms.NumberInput(attrs={"class":"myfield"}))
```

У цьому випадку через параметр віджетів *attrs* встановлюються атрибути того елемента *html*, який буде генеруватися. Зокрема, тут для обох полів встановлюється атрибут *class*, який представляє клас *myfield*.

І, припустимо, у шаблоні буде визначено клас *myfield*:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Forms example on WebPython</title>
  <style>
    .myfield{
      border:1px solid #ccc;
      border-radius:5px;
      height:25px;
      width:200px;
      margin: 10px 10px 10px 0;
    }
  </style>
</head>
<body>
  <form method="POST">
    {% csrf_token %}
    <div>
      {% for field in form %}
      <div>
        {{field.label_tag}}
        <div>{{field}}</div>
      </div>
      {% endfor %}
    </div>
    <input type="submit" value="Send" >
  </form>
</body>
</html>
```



Forms example on WebPython

127.0.0.1:8000

Name:

Age:

Send

Рисунок 1.16 – Результат роботи

2 МОДЕЛІ

2.1 Підключення до бази даних

Для зберігання даних у веб-додатку, як правило, застосовуються бази даних. І фреймворк Django вже за замовчуванням надає зручний функціонал для роботи з різними системами баз даних.

2.1.1 Налаштування підключення до бази даних

За замовчуванням Django як базу даних використовує *SQLite*. Вона дуже проста у використанні і не вимагає запущеного сервера. Всі файли бази даних можуть легко переноситися з одного комп'ютера на інший. Однак за необхідності ми можемо використовувати в Django більшість поширених СУБД.

Для роботи з базами даних у проєкті Django у файлі *settings.py* визначено параметр *DATABASES*, який за замовчуванням має такий вигляд:

```
# Database
# https://docs.djangoproject.com/en/4.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

Змінна *DATABASES* містить набір конфігурацій підключень до баз даних у вигляді словника. Ключі в цьому словнику - назви підключень. Тобто ми можемо визначити купу підключень. Але щонайменше одне підключення має бути визначене у змінній *DATABASES* - підключення з ім'ям *default*, яке представляє підключення за замовчуванням.

Конфігурація кожного підключення може складатися з низки параметрів. За замовчуванням вказуються тільки два параметри. Параметр *ENGINE* вказує на

використовуваний движок для доступу до БД. У цьому випадку це вбудований пакет `django.db.backends.sqlite3`.

Другий параметр - *NAME* вказує на шлях до бази даних. За замовчуванням база даних називається `db.sqlite3`. Для встановлення шляху використовується каталог зі змінної *BASE_DIR*, яка задана на початку файлу:

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

За замовчуванням *BASE_DIR* вказує на каталог, в якому знаходиться папка проекту. І після першого запуску проекту в зазначеному каталозі за замовчуванням буде створено файл `db.sqlite3`, який, власне, і буде використовуватися як база даних.

2.1.2 Підтримувані СУБД

Щоб використовувати інші системи керування базами даних, необхідно буде встановити відповідний пакет.

СУБД	Пакет	Команда встановлення
PostgreSQL	psycopg2	pip install psycopg2
MySQL	mysql-python	pip install mysql-python
Oracle	cx_Oracle	pip install cx_Oracle

Це вбудовані рушії, але також спільнота може надавати свої рушії до інших СУБД. Наприклад, компанія Microsoft надає пакет `mssql-django` для роботи з MS SQL Server (встановлюється командою `pip install mssql-django`).

2.1.3 Параметри підключення

Вище в налаштуваннях за замовчуванням застосовувалися тільки два параметри: *ENGINE* і *NAME*. Однак за потреби можна вказати й додаткові параметри:

- *ATOMIC_REQUESTS*: при значенні *True* обгортає кожен запит до БД в транзакцію. За замовчуванням дорівнює *False*
- *AUTOCOMMIT*: при значенні *False* відключає систему управління транзакціями Django. За замовчуванням дорівнює *True*
- *ENGINE*: рушій БД. Підтримуються такі вбудовані движки:
 - *django.db.backends.postgresql*
 - *django.db.backends.mysql*
 - *django.db.backends.sqlite3*
 - *django.db.backends.oracle*
- *HOST*: хост бази даних
- *NAME*: ім'я бази даних, для *sqlite* - шлях до БД
- *CONN_MAX_AGE*: час життя підключення в секундах. Значення *0* закриває підключення відразу після завершення кожного запиту до БД. А значення *None* вказує на постійне підключення. За замовчуванням дорівнює *0*
- *CONN_HEALTH_CHECKS*: за значення *True* перевіряє активність під'єднання, і якщо воно закрите, заново встановлює з'єднання. За замовчуванням дорівнює *False*
- *OPTIONS*: додатковий набір параметрів, який може знадобитися при підключенні до певних баз даних.
- *PASSWORD*: пароль. За замовчуванням дорівнює "" (порожній рядок)
- *PORT*: порт БД. За замовчуванням дорівнює ""
- *TIME_ZONE*: часовий пояс. За замовчуванням дорівнює *None*
- *USER*: ім'я користувача. За замовчуванням дорівнює ""
- *TEST*: набір налаштувань у вигляді словника для тесту підключення

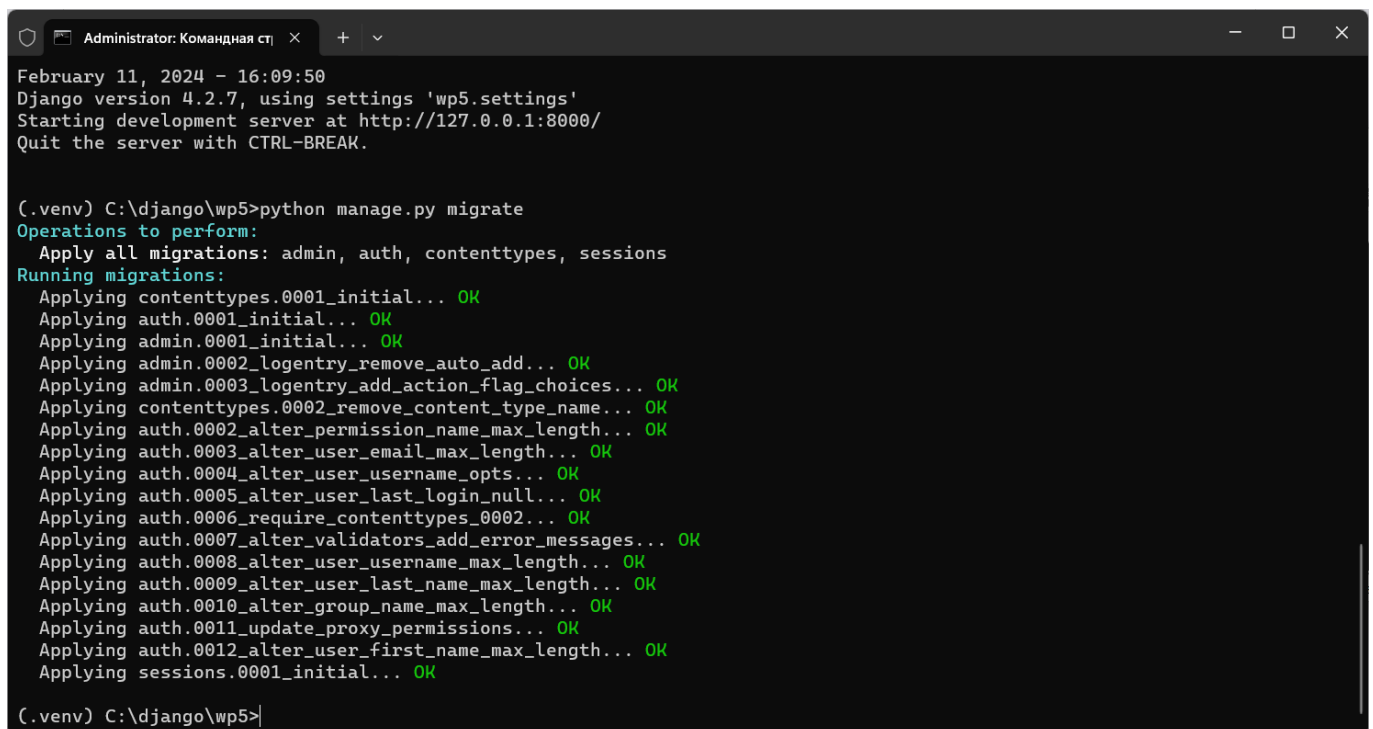
Річ у тім, що для низки застосунків, під'єднаних до проекту за замовчуванням, потрібна база даних для збереження деяких своїх даних. Так, у файлі *settings.py* у змінній *INSTALLED_APPS* за замовчуванням вказано низку додатків.

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'lab_db',
]
```

Перші чотири додатки так чи інакше використовують базу даних. Однак, щоб база даних створила потрібні для цих додатків таблиці, необхідно виконати міграції - спеціальні скрипти, які змінюють структуру бази даних. Для цього виконаємо в консолі команду

```
python manage.py migrate
```



```
Administrator: Командная строка
February 11, 2024 - 16:09:50
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

(.venv) C:\django\wp5>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK

(.venv) C:\django\wp5>
```

Рисунко 2.2 – Результат міграції даних

Якщо після цього ми відкриємо базу даних *db.sqlite3*, яка є в проєкті, в якій-небудь спеціальній програмі для перегляду БД SQLite, то ми побачимо, що вона містить низку таблиць:

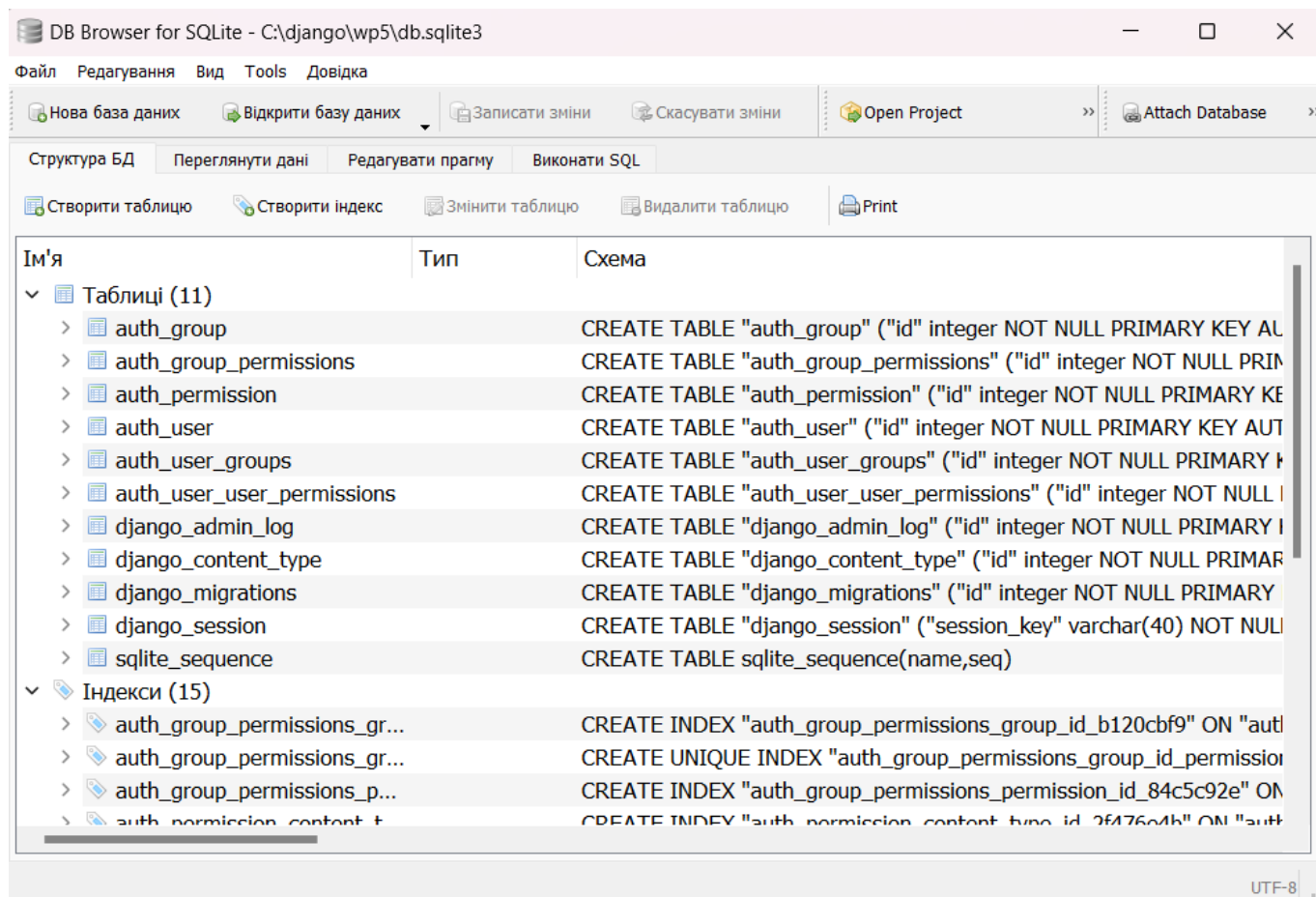


Рисунок 2.3 – Наповнення БД

Для перегляду та роботи з базами даних *SQLite* можна використовувати безкоштовну програму *DB Browser for SQLite*, яка доступна для всіх основних операційних систем.

Зокрема, можна побачити, що база даних містить понад десяток таблиць та індексів. Усі вони пов'язані з системою аутентифікації та авторизації, адміністрування та сесіями. І навіть якщо ми безпосередньо не використовуватимемо базу даних, але використовуватимемо системи автентифікації та авторизації, адміністрування або сесії, то для нормальної роботи нам все одно знадобиться база даних і необхідно буде виконати міграції.

2.2 Створення моделей

Моделі в Django описують структуру використовуваних даних. Використовувані в програмі дані зберігаються в базах даних, і за допомогою моделей якраз здійснюється взаємодія з базою даних.

При створенні програми за замовчуванням в його каталог додається файл *models.py*, який застосовується для визначення моделей. Модель являє собою клас, успадкований від *django.db.models.Model*.

Так, змінимо файл *models.py* таким чином:

```
from django.db import models

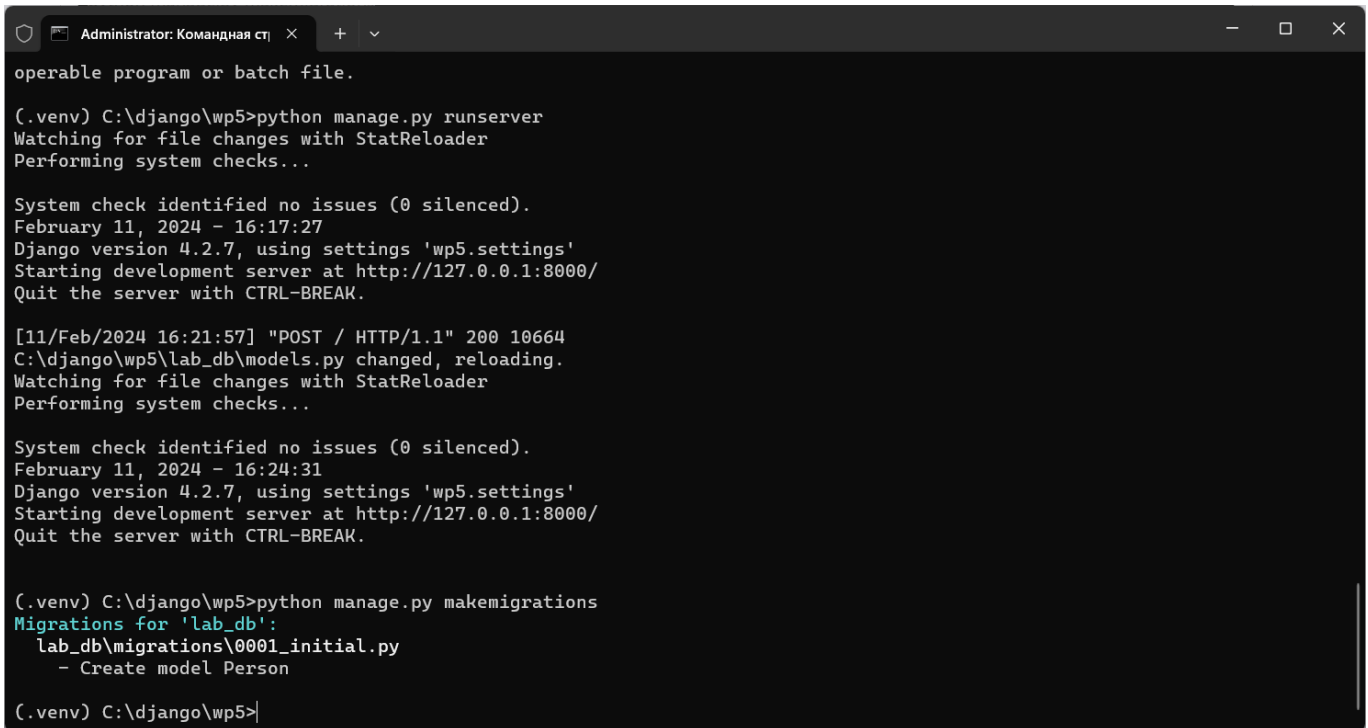
class Person(models.Model):
    name = models.CharField(max_length=20)
    age = models.IntegerField()
```

Тут визначено найпростішу модель, яка називається *Person* і яка представляє людину. У моделі визначено два поля. Поле *name* представляє тип *CharField* - текстове поле, яке зберігає послідовність символів. Воно буде зберігати ім'я людини. Для *CharField* обов'язково треба вказати параметр *max_length*, який задає максимальну довжину рядка, що зберігається. І поле *age* представляє тип *IntegerField* - числове поле, яке зберігає цілі числа. Воно призначене для зберігання віку людини.

Кожна модель зіставляється з певною таблицею в базі даних. Однак поки що в нас немає в БД таблиці, яка зберігає об'єкти моделі *Person*. І в цьому випадку нам треба створити і виконати міграцію. Міграція перетворює базу даних відповідно до визначення моделей.

Спочатку необхідно створити міграцію за допомогою команди

```
python manage.py makemigrations
```



```

Administrator: Командная строка
operable program or batch file.

(.venv) C:\django\wp5>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
February 11, 2024 - 16:17:27
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

[11/Feb/2024 16:21:57] "POST / HTTP/1.1" 200 10664
C:\django\wp5\lab_db\models.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
February 11, 2024 - 16:24:31
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

(.venv) C:\django\wp5>python manage.py makemigrations
Migrations for 'lab_db':
  lab_db\migrations\0001_initial.py
    - Create model Person

(.venv) C:\django\wp5>

```

Рисунок 2.6 – Результат створення міграції

Після цього в додатку в папці *migrations* ми виявимо новий файл *0001_initial.py*, який матиме приблизно такий вміст:

```

# Generated by Django 4.2.7 on 2024-02-11 14:29

from django.db import migrations, models

class Migration(migrations.Migration):

    initial = True

    dependencies = [
    ]

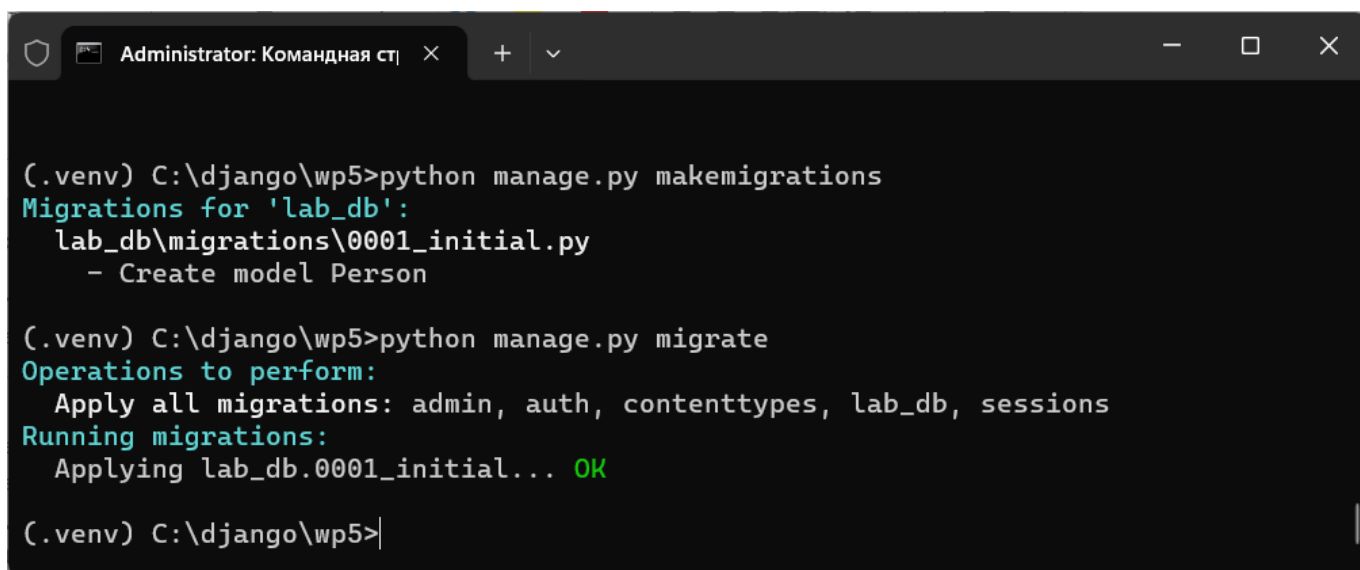
    operations = [
        migrations.CreateModel(
            name='Person',
            fields=[
                ('id', models.BigAutoField(auto_created=True, primary_key=True,
serialize=False, verbose_name='ID')),
                ('name', models.CharField(max_length=20)),
                ('age', models.IntegerField()),
            ],
        ),
    ]

```


Це і є міграція. Тут можна помітити, що створюється не два, а три поля - поле *id*, яке представлятиме первинний ключ, додається за замовчуванням. Тому в принципі в самій моделі нам не потрібно явно визначати будь-який ідентифікатор.

Тепер треба виконати цю міграцію. Для цього виконується команда

```
python manage.py migrate
```



```
Administrator: Командная строка
(.venv) C:\django\wp5>python manage.py makemigrations
Migrations for 'lab_db':
  lab_db\migrations\0001_initial.py
    - Create model Person

(.venv) C:\django\wp5>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, lab_db, sessions
Running migrations:
  Applying lab_db.0001_initial... OK

(.venv) C:\django\wp5>
```

Рисунок 2.5 – Результат безпосередньої міграції

Після цього, якщо ми відкриємо базу даних *db.sqlite3*, яка є в проєкті, то ми побачимо, що в неї додано таблицю для зберігання даних моделі *Person*:

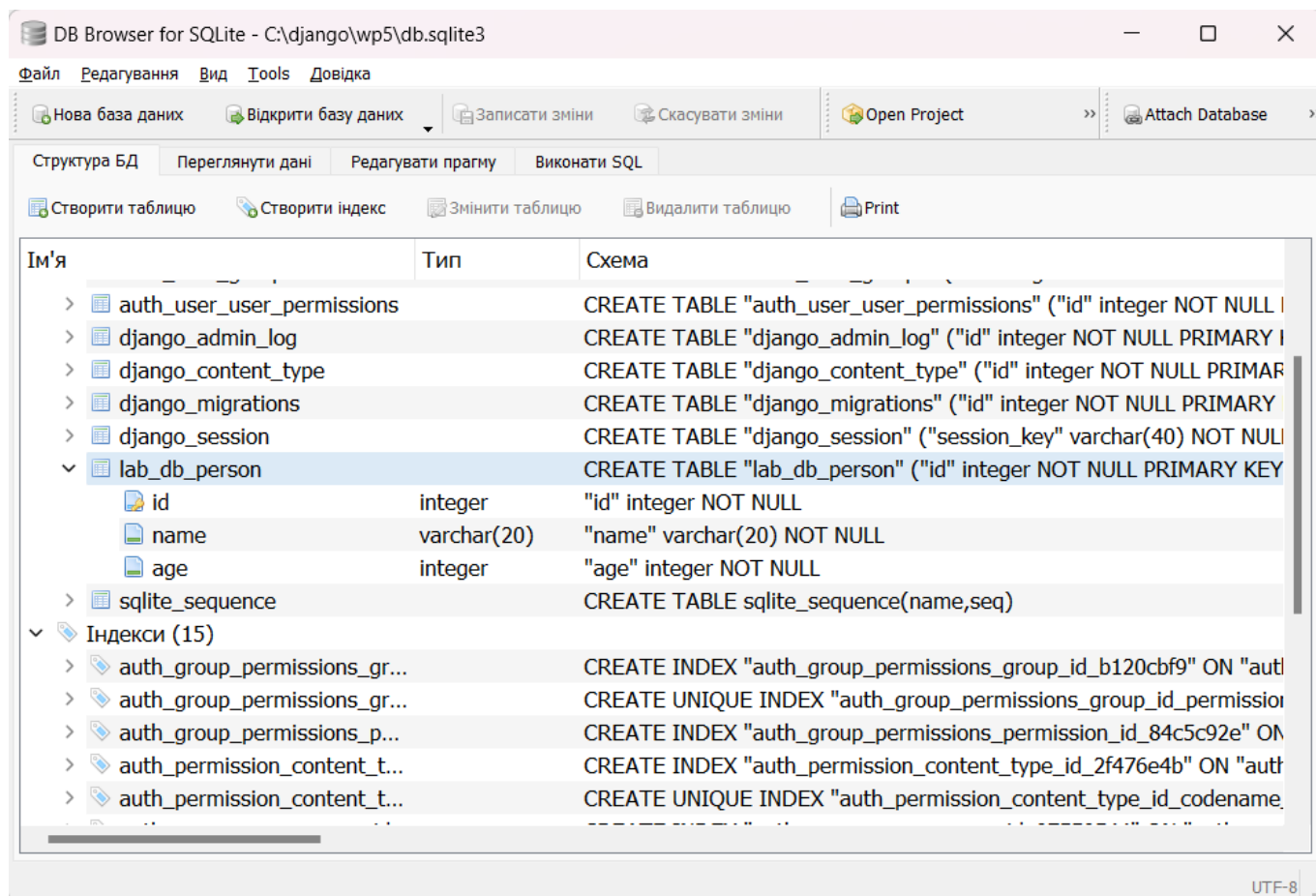


Рисунок 2.6 – Результат міграції моделі

Варто враховувати, що в процесі роботи над проєктом визначення моделі може змінюватися - до неї можуть додаватися нові поля, або можуть видалятися вже наявні, або можуть додаватися нові моделі, дані яких ми також захочемо зберігати в базі даних. І в цьому випадку знову треба створювати міграцію і застосовувати її до бази даних, щоб база даних була синхронізована з визначенням моделей.

Для визначення моделей ми можемо використовувати такі типи полів:

- *BinaryField()*: зберігає бінарні дані
- *BooleanField()*: зберігає значення *True* або *False* (0 або 1)
- *NullBooleanField()*: зберігає значення *True* або *False* або *Null*
- *DateField()*: зберігає дату
- *TimeField()*: зберігає час
- *DateTimeField()*: зберігає дату і час
- *DurationField()*: зберігає період часу

- *AutoField()*: зберігає цілочисельне значення, яке автоматично інкрементується, зазвичай застосовується для первинних ключів
- *BigAutoField()*: зберігає 64-бітове цілочисельне значення, але на відміну від *AutoField* гарантує, що число входить у діапазон від 1 до 9223372036854775807.
- *SmallAutoField()*: зберігає 16-бітове цілочисельне значення в діапазоні від 1 до 32767
- *BigIntegerField()*: представляє число - значення типу *Number*, яке вкладається в діапазон від -9223372036854775808 до 9223372036854775807. Залежно від обраної СУБД діапазон може трохи відрізнятись
- *DecimalField(decimal_places=X, max_digits=Y)*: представляє значення типу *Number*, яке має максимум *X* розрядів і *Y* знаків після коми
- *FloatField()*: зберігає, значення типу *Number*, яке представляє число з плаваючою крапкою
- *IntegerField()*: зберігає значення типу *Number*, яке представляє цілочисельне значення
- *PositiveIntegerField()*: зберігає значення типу *Number*, яке представляє позитивне цілочисельне значення (від 0 до 2147483647)
- *PositiveBigIntegerField()*: зберігає значення типу *Number*, яке представляє позитивне 64-бітове цілочисельне значення (від 0 до 9223372036854775807)
- *PositiveSmallIntegerField()*: зберігає значення типу *Number*, яке представляє невелике позитивне цілочисельне значення (від 0 до 32767)
- *SmallIntegerField()*: зберігає значення типу *Number*, яке представляє невелике цілочисельне значення (від -32768 до 32767)
- *CharField(max_length=N)*: зберігає рядок завдовжки не більше *N* символів
- *TextField()*: зберігає рядок невизначеної довжини
- *EmailField()*: зберігає рядок, який представляє email-адресу. Значення автоматично валідується вбудованим валідатором *EmailValidator*
- *FileField()*: зберігає рядок, який представляє ім'я файлу
- *FilePathField()*: зберігає рядок, який представляє шлях до файлу завдовжки в 100 символів

- *ImageField()*: зберігає рядок, який представляє дані про зображення
- *GenericIPAddressField()*: зберігає рядок, який представляє IP-адресу у форматі IPv4 або IPv6
- *SlugField()*: зберігає рядок, який може містити тільки букви в нижньому регістрі, цифри, дефіс і знак підкреслення
- *URLField()*: зберігає рядок, який представляє валідну URL-адресу
- *UUIDField()*: зберігає рядок, який представляє UUID-ідентифікатор
- *JSONField()*: зберігає дані у форматі JSON, яка представляє UUID-ідентифікатор

Таблиця зіставлення полів із типами в різних СУБД:

Тип	SQLite	MySQL	PostgreSQL	Oracle
<i>BinaryField()</i>	BLOB NOT NULL	longblob NOT NULL	bytea NOT NULL	BLOB NULL
<i>BooleanField()</i>	bool NOT NULL	bool NOT NULL	boolean NOT NULL	NUMBER(1) NOT NULL CHECK("Значення" IN(0,1))
<i>NullBooleanField()</i>	bool NULL	bool NULL	boolean NULL	NUMBER(1) NOT NULL CHECK(("Значення" IN(0,1)) OR ("Значення" IS NULL))
<i>DateField()</i>	date NULL	date NULL	date NULL	DATE NOT NULL
<i>TimeField()</i>	time NULL	time NULL	time NULL	TIMESTAMP NOT NULL
<i>DateTimeField()</i>	datetime NULL	datetime NULL	timestamp NULL	TIMESTAMP NOT NULL
<i>DurationField()</i>	bigint NOT NULL	bigint NOT NULL	interval NOT NULL	INTERVAL DAY(9) TO SECOND(6) NOT NULL
<i>AutoField()</i>	integer NOT NULL AUTOINCREMENT	integer AUTO_INCREMENT NOT NULL	serial NOT NULL	NUMBER(11) NOT NULL
<i>BigIntegerField</i>	bigint NOT NULL	bigint NOT NULL	bigint NOT NULL	NUMBER(19) NOT NULL
<i>DecimalField(decimal_places=X, max_digits=Y)</i>	decimal NOT NULL	numeric(X, Y) NOT NULL	numeric(X, Y) NOT NULL	NUMBER(10, 3) NOT NULL

FloatField	real NOT NULL	double precision NOT NULL	double precision NOT NULL	DOUBLE PRECISION NOT NULL
IntegerField	integer NOT NULL	integer NOT NULL	integer NOT NULL	NUMBER(11) NOT NULL
PositiveIntegerField	integer unsigned NOT NULL	integer UNSIGNED NOT NULL	integer NOT NULL CHECK ("Значення" > 0)	NUMBER NOT NULL CHECK ("Значення" > 0)
PositiveSmallIntegerField	smallint unsigned NOT NULL	smallint UNSIGNED NOT NULL	smallint NOT NULL CHECK ("Значення" > 0)	NUMBER(11) NOT NULL CHECK ("Значення" > 0)
SmallIntegerField	smallint NOT NULL	smallint NOT NULL	smallint NOT NULL	NUMBER(11) NOT NULL
CharField(max_length=N)	varchar(N) NOT NULL	varchar(N) NOT NULL	varchar(N) NOT NULL	NVARCHAR2(N) NULL
TextField()	text NOT NULL	longtext NOT NULL	text NOT NULL	NCLOB NULL
EmailField()	varchar(254) NOT NULL	varchar(254) NOT NULL	varchar(254) NOT NULL	NVARCHAR2(254) NULL
FileField()	varchar(100) NOT NULL	varchar(100) NOT NULL	varchar(100) NOT NULL	NVARCHAR2(100) NULL
FilePathField()	varchar(100) NOT NULL	varchar(100) NOT NULL	varchar(100) NOT NULL	NVARCHAR2(100) NULL
ImageField()	varchar(100) NOT NULL	varchar(100) NOT NULL	varchar(100) NOT NULL	NVARCHAR2(100) NULL
GenericIPAddressField()	char(39) NOT NULL	char(39) NOT NULL	inet NOT NULL	VARCHAR2(39) NULL
SlugField()	varchar(50) NOT NULL	varchar(50) NOT NULL	varchar(50) NOT NULL	NVARCHAR2(50) NULL
URLField()	varchar(200) NOT NULL	varchar(200) NOT NULL	varchar(200) NOT NULL	NVARCHAR2(200) NULL
UUIDField()	char(32) NOT NULL	char(32) NOT NULL	uuid NOT NULL	VARCHAR2(32) NULL
JSONField()			jsonb	

2.3 QuerySet API

У центрі роботи з моделями та базами даних знаходиться об'єкт *QuerySet* і його методи. *QuerySet* представляє набір запитів до бази даних, а його методи створюють подібні запити. А Django дає змогу транслювати виклики методів *QuerySet* у відповідні SQL-запити. Відзначу основні методи, які можна застосовувати для роботи з даними моделей:

- *filter()*: фільтрує набір даних і повертає вибірку даних, яка відповідає фільтру, також у вигляді об'єкта *QuerySet*

- *exclude()*: фільтрує набір даних і повертає вибірку, яка НЕ відповідає фільтру, у вигляді об'єкта *QuerySet*
- *order_by()*: сортує набір даних і повертає відсортовану вибірку у вигляді об'єкта *QuerySet*
- *distinct()*: повертає об'єкт *QuerySet*, який містить тільки унікальні об'єкти
- *values()*: повертає дані об'єктів у вигляді словника
- *values_list()*: повертає дані об'єктів у вигляді списку
- *none()*: повертає порожній набір даних у вигляді об'єкта *EmptyQuerySet* (клас похідний від *QuerySet*)
- *all()*: повертає об'єкт *QuerySet*, який містить усі об'єкти з таблиці
- *union()*: об'єднує два об'єкти *QuerySet* і повертає об'єднаний об'єкт *QuerySet*
- *intersection()*: повертає перетин двох об'єктів *QuerySet* у вигляді іншого *QuerySet*
- *difference()*: повертає різницю двох об'єктів *QuerySet* у вигляді іншого *QuerySet*
- *raw()*: виконує переданий як параметр SQL-запит
- *get()*: повертає один об'єкт моделі
- *create()*: створює один об'єкт моделі
- *get_or_create()*: повертає один об'єкт моделі за вказаними параметрами, а якщо він відсутній, то створює його
- *update_or_create()*: оновлює один об'єкт моделі за вказаними параметрами, а якщо він відсутній, то створює його
- *bulk_create()*: створює набір об'єктів моделі
- *save()*: зберігає один об'єкт моделі (якщо він раніше був відсутній у БД, то додає його, якщо був присутній, то оновлює його)
- *bulk_update()*: оновлює набір об'єктів
- *count()*: повертає кількість об'єктів у наборі
- *in_bulk()*: повертає словник об'єктів, які мають певні ідентифікатори
- *latest()*: повертає об'єкт із найостаннішими змінами
- *earliest()*: повертає об'єкт із найбільш ранніми змінами

- *first()*: повертає перший об'єкт набору
- *last()*: повертає останній об'єкт набору
- *aggregate()*: виконує агрегатну функцію (знаходження суми, середнього, мінімального, максимального значення тощо) і повертає результат у вигляді числа
- *exists()*: перевіряє, чи є в наборі як мінімум один об'єкт. Якщо є, то повертається *True*, інакше повертається *False*
- *contains()*: перевіряє, чи є в наборі певний об'єкт. Якщо є, то повертається *True*, інакше повертається *False*
- *update()*: оновлює один об'єкт моделі
- *delete()*: видаляє один об'єкт моделі

Повний список методів можна знайти в документації.

Причому виконання методів *QuerySet* у базі даних відбувається не відразу - під час виклику методу, а під час отримання конкретного результату їхнього *QuerySet*, наприклад, під час перебору набору. Подивимося виконання методів на прикладі. Припустимо, у файлі *models.py* визначено таку модель *Person*:

```
from django.db import models

class Person(models.Model):
    name = models.CharField(max_length=20)
    age = models.IntegerField()
```

Оскільки маршрутизація шляхів на даному етапі не важлива, то наповнимо файл *urls.py* таким змістом:

```
from django.urls import path
from lab_db import views

urlpatterns = [
    # path('', views.index),
]
```

Для спрощення протестуємо код у файлі *views.py*, код якого викликається під час запуску програми. Зокрема, визначимо в ньому такий код:

```
from .models import Person
```

```
# отримуємо всі об'єкти
people = Person.objects.all()
print(people.query)

# отримуємо об'єкти з ім'ям Bill
people = people.filter(name = "Bill")
print(people.query)

# отримуємо об'єкти з віком, що дорівнює 35
people = people.filter(age = 35)
print(people.query)

# тут відбувається виконання запиту в БД
for person in people:
    print(f"{person.id}. {person.name} - {person.age}")
```

Спочатку йде отримання всіх даних за допомогою методу `all()`:

```
people = Person.objects.all()
```

Змінна *people* тут представлятиме об'єкт *QuerySet*, який представляє набір з усіх об'єктів таблиці.

За допомогою властивості *query* у результаті запиту ми можемо отримати SQL-запит і, наприклад, вивести його на консоль:

```
print(people.query)
```

Потім фільтруємо набір за допомогою методу *filter()* і вибираємо тільки ті об'єкти, де поле *name* дорівнює "Bill":

```
people = people.filter(name = "Bill")
```

Метод *filter()* також повертає *QuerySet*, який присвоюється тій самій змінній *people*.

Потім вибираємо тільки ті об'єкти, де поле *name* дорівнює "Bill" та поле *age* дорівнює 35:

```
people = people.filter(age = 35)
```


Таким чином, до набору *QuerySet* було застосовано три методи, проте в жодному випадку не відбувається звернення до бази даних. Це відбувається, тільки коли ми безпосередньо звертаємося до об'єктів набору, перебираючи їх у циклі.

```
for person in people:
    print(f"{person.id}. {person.name} - {person.age}")
```

А за консольним виведенням ми можемо побачити, як у нас створюється фінальний SQL-запит, який і буде виконуватися в БД під час перебору об'єктів у циклі:

```
(.venv) C:\django\wp5>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

SELECT "lab_db_person"."id", "lab_db_person"."name", "lab_db_person"."age" FROM
"lab_db_person"
SELECT "lab_db_person"."id", "lab_db_person"."name", "lab_db_person"."age" FROM
"lab_db_person" WHERE "lab_db_person"."name" = Bill
SELECT "lab_db_person"."id", "lab_db_person"."name", "lab_db_person"."age" FROM
"lab_db_person" WHERE ("lab_db_person"."name" = Bill AND "lab_db_person"."age" =
35)
System check identified no issues (0 silenced).
February 11, 2024 - 17:06:18
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

2.4 Створення та отримання об'єктів моделі

Розглянемо додавання в базу даних і отримання з неї на прикладі моделі *Person*:

```
from django.db import models

class Person(models.Model):
    name = models.CharField(max_length=20)
    age = models.IntegerField()
```

2.4.1 Додавання даних

create()

Для додавання даних застосовується метод *create()*:

```
bill = Person.objects.create(name="Bill", age=40)
```

Якщо додавання пройде успішно, то об'єкт матиме *id*, який можна отримати через *bill.id*.

Асинхронна версія методу – *acreate()*

```
from .models import Person
import asyncio

async def acreate_person():
    person = await Person.objects.acreate(name="Jerry", age=25)
    print(person.name)

# запускаємо асинхронну функцію acreate_person
asyncio.run(acreate_person())
```

save()

Однак у своїй суті метод *create()* використовує інший метод - *save()*, який ми також можемо використовувати окремо для додавання об'єкта:

```
bill = Person(name="Bill", age=40)
bill.save()
```

Після успішного додавання також можна отримати ідентифікатор доданого запису за допомогою *bill.id*.

bulk_create()

Метод *bulk_create()* (і його асинхронна версія *abulk_create()*) дає змогу додати набір об'єктів, який передають як параметр у метод:

```
from .models import Person

people = Person.objects.bulk_create([
```

```

    Person(name="Donald", age=20),
    Person(name="Duck", age=25),
])

for person in people:
    print(f"{person.id}. {person.name}")

```

2.4.2 Отримання з БД

Отримання одного об'єкта

Метод `get()` повертає один об'єкт за певною умовою, яка передається як параметр:

```

tom = Person.objects.get(name="Bill")    # отримуємо запис, де name="Bill"
bob = Person.objects.get(age=20)         # отримуємо запис, де age=20

```

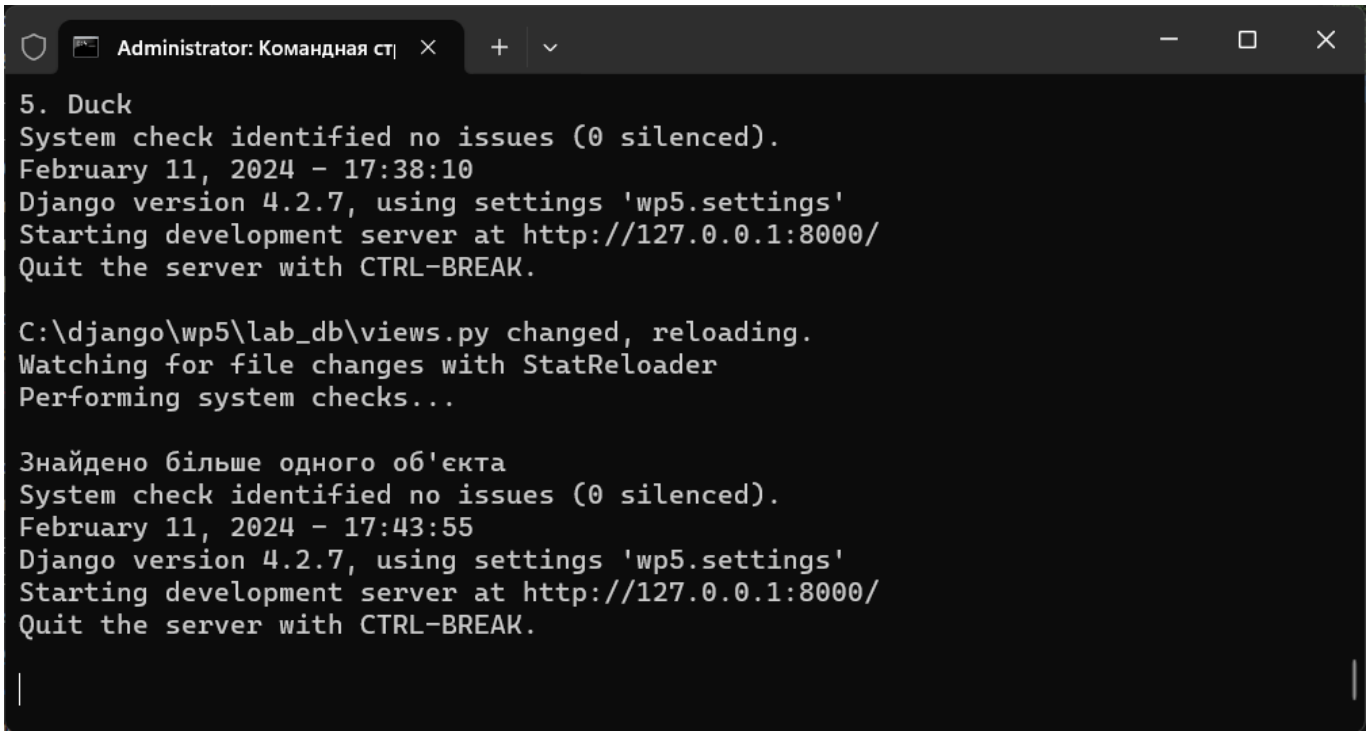
Під час використання цього методу треба враховувати, що він призначений для вибірки таких об'єктів, які є в одиничному числі в базі даних. Якщо в таблиці не виявиться подібного об'єкта, то ми отримаємо помилку *ім'я_моделі.DoesNotExist*. Якщо ж у таблиці буде кілька об'єктів, які відповідають умові, то буде згенеровано виняток *MultipleObjectsReturned*. Тому слід застосовувати цей метод з обережністю, або застосовувати обробку відповідних винятків:

```

from .models import Person
from django.core.exceptions import ObjectDoesNotExist, MultipleObjectsReturned

try:
    bill = Person.objects.get(name="Bill")    # MultipleObjectsReturned
    alex = Person.objects.get(name="Alex")    # ObjectDoesNotExist
except ObjectDoesNotExist:
    print("Об'єкт не існує")
except MultipleObjectsReturned:
    print("Знайдено більше одного об'єкта")

```



```

Administrator: Командная строка
5. Duck
System check identified no issues (0 silenced).
February 11, 2024 - 17:38:10
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp5\lab_db\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

Знайдено більше одного об'єкта
System check identified no issues (0 silenced).
February 11, 2024 - 17:43:55
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

Рисунок 2.7 – Результат обробки винятків

Асинхронна версія методу називається *aget()*:

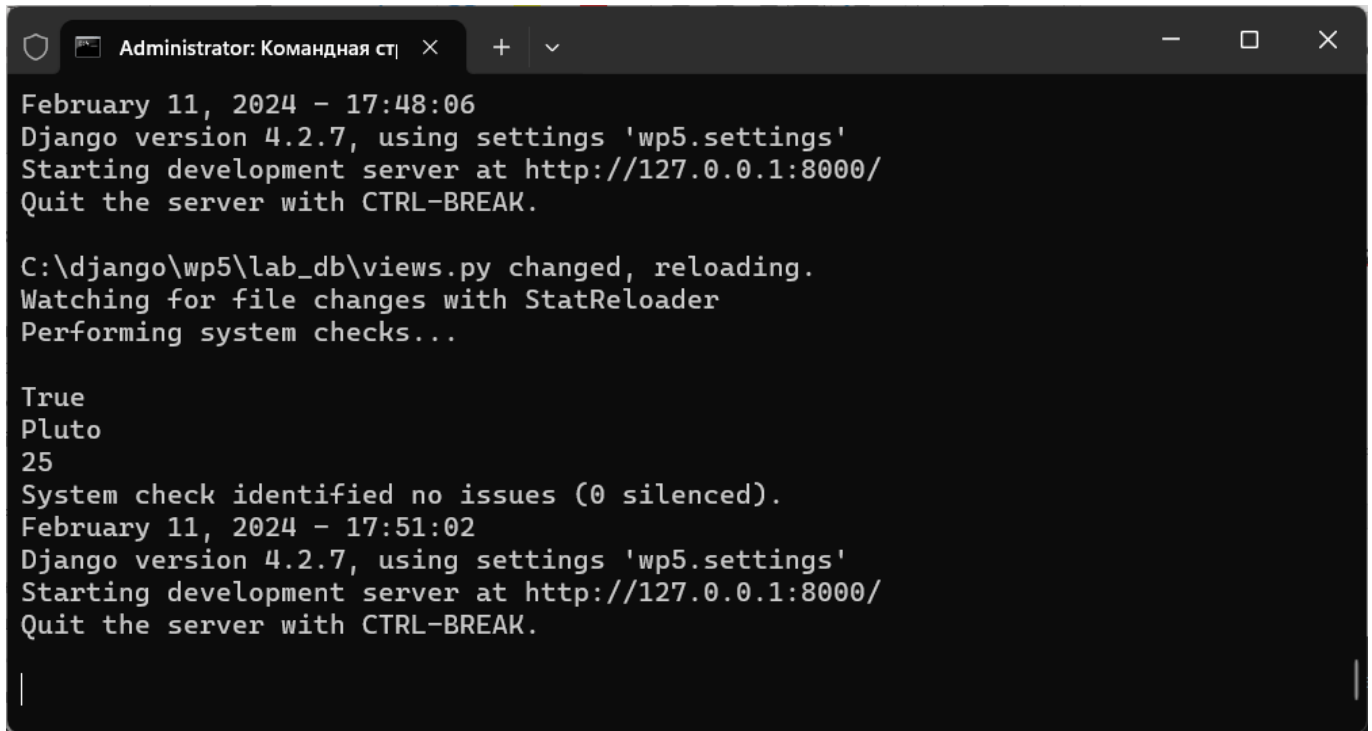
get_or_create()

Метод *get_or_create()* (і його асинхронна версія *aget_or_create()*) повертає об'єкт, а якщо його немає в БД, то додає в БД новий об'єкт.

```

from .models import Person
pluto, created = Person.objects.get_or_create(name="Pluto", age=25)
print(created)
print(pluto.name)
print(pluto.age)

```



```
Administrator: Командная строка
February 11, 2024 - 17:48:06
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp5\lab_db\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

True
Pluto
25
System check identified no issues (0 silenced).
February 11, 2024 - 17:51:02
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.8 – Результат роботи методу `get_or_create()`

У цьому випадку, якщо в таблиці немає об'єкта зі значеннями `name="Pluto"` і `age=25`, то він додається. Якщо є, то він повертається.

Метод повертає доданий об'єкт (у цьому випадку змінна `pluto`) і булеве значення (`created`), яке зберігає `True`, якщо додавання пройшло успішно.

Варто враховувати, що якщо в таблиці вже є кілька об'єктів (два і більше) із зазначеними значеннями, то згенерується виняток `MultipleObjectsReturned`.

`all()`

Якщо необхідно отримати всі наявні об'єкти, то застосовується метод `all()`:

```
people = Person.objects.all()
```

Цей метод повертає об'єкт типу `QuerySet`.

filter()

Якщо треба отримати всі об'єкти, які відповідають певному критерію, то застосовується метод *filter()*, який як параметр приймає критерій вибірки:

```
people = Person.objects.filter(age=23)
# використання декількох критеріїв
people2 = Person.objects.filter(name="Bill", age=40)
```

Метод *filter()* дає змогу визначати складніші умови, але оскільки це окрема велика тема, то докладніше буде розглянута в окремій статті.

exclude()

Метод *exclude()* дає змогу виключити з вибірки записи, які відповідають переданому як параметр критерію:

```
# виключаємо користувачів, у яких age=25
people = Person.objects.exclude(age=23)
```

Можна комбінувати два вище розглянуті методи:

```
# вибираємо всіх користувачів, у яких name="Bill", крім тих, у яких age=25
people = Person.objects.filter(name="Tom").exclude(age=23)
```

in_bulk()

Метод *in_bulk()* (і його асинхронна версія *ain_bulk()*) є більш ефективним способом для читання великої кількості записів. Як параметр у нього можна передати список ідентифікаторів об'єктів, які треба отримати. Як результат він повертає словник, тобто об'єкт *dict*:

```
people = Person.objects.in_bulk()
for id in people:
```

```

print(people[id].name)
print(people[id].age)

# получаем объекты с id=1 и id=3
people2 = Person.objects.in_bulk([1,3])
for id in people2:
    print(people2[id].name)
    print(people2[id].age)

```

```

Administrator: Командная строка
Performing system checks...

Jerry
25
Bill
40
Bill
40
Donald
20
Duck
25
Pluto
25
Jerry
25
Bill
40
System check identified no issues (0 silenced).

```

Рисунок 2.9 – Результат роботи `in_bulk()`

Метод `in_bulk()` повертає словник, де ключі представляють *id* об'єктів, а значення за цими ключами - власне ці об'єкти, тобто в цьому випадку об'єкти *Person*.

Обмеження кількості

За допомогою синтаксису списків можна отримати певну порцію даних із *QuerySet*:

```

from .models import Person

people = Person.objects.all()[:5]

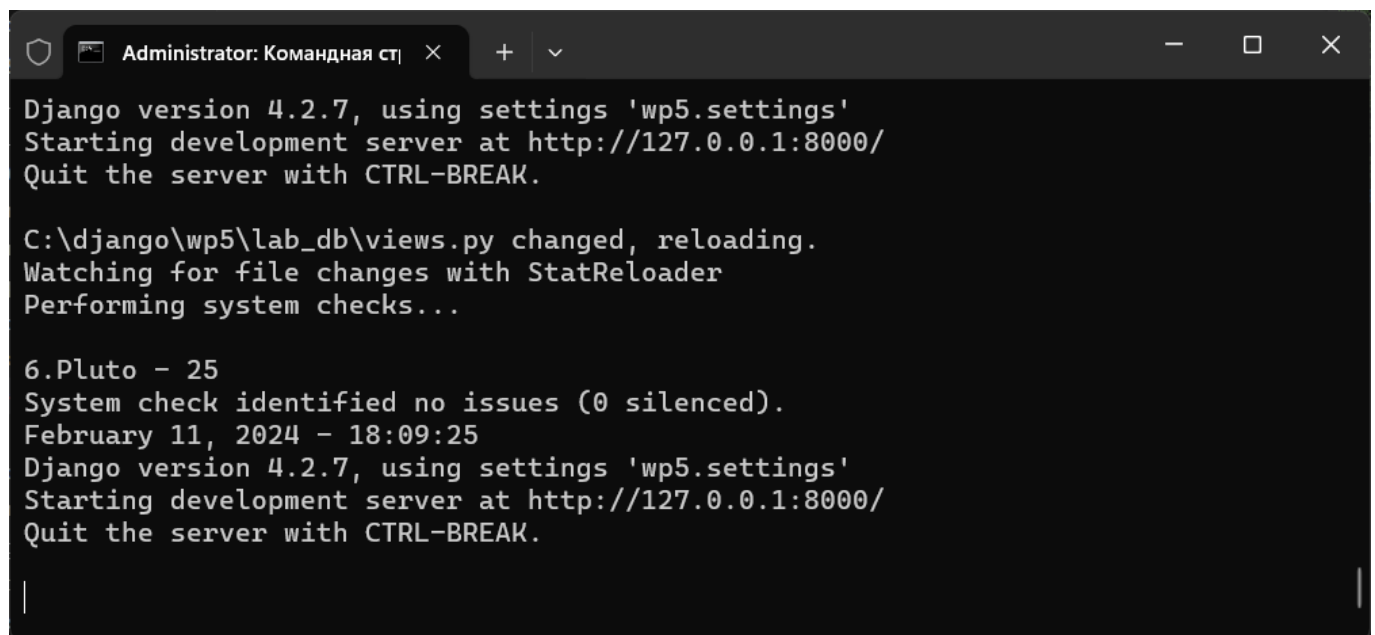
```

У цьому випадку вибираємо перші 5 об'єктів, що на рівні бази даних транслюється в SQL-вираз *LIMIT 5*

Перший параметр вказує, скільки об'єктів треба пропустити:

```
from .models import Person

people = Person.objects.all()[5:10]
for person in people:
    print(f"{person.id}. {person.name} - {person.age}")
```



```
Administrator: Командная строка
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp5\lab_db\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

6.Pluto - 25
System check identified no issues (0 silenced).
February 11, 2024 - 18:09:25
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.10 – Результат пагінації

У цьому випадку пропускаємо перші 5 об'єктів і вибираємо наступні 5 об'єктів до 10-го індексу, що на рівні бази даних транслюється у вираз *OFFSET 5 LIMIT 5*.

2.5 Редагування та видалення об'єктів моделі

Розглянемо приклад з редагуванням і видаленням об'єктів моделі на прикладі моделі *Person*:

```
from django.db import models

class Person(models.Model):
    name = models.CharField(max_length=20)
    age = models.IntegerField()
```


2.5.1 Оновлення

save()

Для оновлення об'єкта також застосовується метод *save()*:

```
willy = Person.objects.get(id=2)
willy.name = "Willy"
willy.save()
```

У цьому разі Django повністю оновлює об'єкт, усі його властивості, навіть якщо ми їх не змінювали. Щоб вказати, що нам треба оновити тільки певні поля, слід використовувати параметр *update_fields*:

```
from .models import Person

pooh = Person.objects.get(id=2)
pooh.name = "Pooh"
pooh.save(update_fields=["name"])
```

Це дасть змогу підвищити продуктивність.

update()

Інший спосіб оновлення об'єктів представляє метод *update()* (і його асинхронна версія *update()*) у поєднанні з методом *filter()*, які разом виконують один запит до бази даних:

```
from .models import Person

number = Person.objects.filter(id=1).update(name="Rabbit")
print(number) # кількість оновлених рядків
```

```
Administrator: Командная строка
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp5\lab_db\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

1
System check identified no issues (0 silenced).
February 11, 2024 - 18:25:22
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.11 – Результат дії методу *update()*

У цьому випадку в об'єктів з *id = 1* встановлюємо для поля *name* значення "Rabbit". Метод повертає кількість оновлених рядків.

Якщо нам не треба отримувати оновлюваний об'єкт, то цей спосіб дасть нам змогу збільшити продуктивність взаємодії з БД.

Також можна встановити і велику кількість полів:

```
Person.objects.filter(id=1).update(name="Mike", age = 35)
```

Іноді буває необхідно змінити значення стовпця в БД на підставі вже наявного значення. У цьому випадку ми можемо використовувати функцію *F()*:

```
from .models import Person
from django.db.models import F

Person.objects.all(id=2).update(age = F("age") + 1)
```

У цьому разі полю *age* присвоюється вже наявне значення, збільшене на одиницю.

При цьому важливо враховувати, що метод *update()* оновлює всі записи в таблиці, які відповідають умові.

Якщо треба оновити взагалі всі записи, незалежно від умови, то необхідно комбінувати метод `update()` з методом `all()`:

```
from .models import Person
from django.db.models import F

Person.objects.all().update(name="Mike")
Person.objects.all().update(age = F("age") + 1)
```

update_or_create()

Метод `update_or_create()` (і його асинхронна версія `auupdate_or_create()`) оновлює запис, а якщо його немає, то додає його в таблицю:

```
values_for_update={"name":"Robin", "age": 15}
bob, created = Person.objects.update_or_create(id=7, defaults = values_for_update)
```

Метод `update_or_create()` приймає два параметри. Перший параметр представляє критерій вибірки об'єктів, які будуть оновлюватися. Другий параметр представляє об'єкт зі значеннями, які отримають обрані об'єкти. Якщо критерію не відповідає жодних об'єктів, то в таблицю додається новий об'єкт, а змінна `created` дорівнюватиме `True`.

bulk_update()

Метод `bulk_update()` (і його асинхронна версія `abulk_update()`) дає змогу оновити за один раз набір об'єктів.

```
bulk_update(objs, fields, batch_size=None)
```

Перший параметр - `obj` вказує на оновлювані об'єкти, а другий параметр - `fields` представляє оновлювані поля з новими значеннями. Останній параметр - `batch_size` вказує, скільки об'єктів оновлюється в одному запиті (за замовчуванням оновлюються всі об'єкти)

```

from .models import Person

first_person = Person.objects.get(id=3)
first_person.name = "Tomas"

second_person = Person.objects.get(id=2)
second_person.age = 31

number = Person.objects.bulk_update([first_person, second_person], ["name", "age"])
print(number)

```

```

Administrator: Командная строка
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp5\lab_db\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

2
System check identified no issues (0 silenced).
February 11, 2024 - 18:38:23
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

|

```

Рисунок 2.12 – Результат пакетного оновлення даних

У цьому випадку у першого об'єкта оновлюється значення поля *"name"*, а у другого - значення поля *"age"*. Тому як другий параметр передається список із цими полями. Результатом методу є кількість оновлених об'єктів.

Цей метод має деякі обмеження. Зокрема, ми не можемо оновити значення первинного ключа. Також якщо в оновлюваному наборі є дублі, то тільки перше входження об'єкта буде використовуватися для оновлення.

2.5.2 Видалення

Для видалення ми можемо викликати метод *delete()* (або його асинхронну версію *adelete()*) в об'єкта, що видаляється:

```
person = Person.objects.get(id=2)
person.delete()
```

Якщо не потрібне отримання окремого об'єкта з бази даних, тоді можна видалити об'єкт за допомогою комбінації методів *filter()* і *delete()*:

```
Person.objects.filter(id=4).delete()
```

Видалення всіх даних із таблиці:

```
Person.objects.all().delete()
```

2.6 Фільтрація

Django надає багаті можливості щодо фільтрації даних. Для фільтрації даних у об'єкта *QuerySet* можуть застосовуватися такі методи:

- *get()*: отримує один об'єкт моделі
- *filter()*: отримує набір об'єктів моделі, які відповідають умові. Результат методу - об'єкт *QuerySet*
- *exclude()*: отримує набір об'єктів моделі, які НЕ відповідають умові. Результат методу - об'єкт *QuerySet*

Усі три методи як параметр отримують умову, за якою відбувається фільтрація. Розглянемо, які умови ми можемо визначити.

Припустимо, у файлі *models.py* визначено таку модель *Person*:

```
from django.db import models

class Person(models.Model):
    name = models.CharField(max_length=20)
    age = models.IntegerField()
```

Найпростіший простий тип умови представляє рівність властивості моделі деякому значенню:

```

from .models import Person

# отримуємо користувача на ім'я
robin = Person.objects.get(name="Robin")
print(f"{robin.name} - {robin.age}")

# отримуємо користувачів, у яких вік дорівнює 15
people_by_age1 = Person.objects.filter(age=15)
for person in people_by_age1:
    print(f"{person.name} - {person.age}")

# отримуємо користувачів, у яких вік НЕ дорівнює 15
people_by_age2 = Person.objects.exclude(age=15)
for person in people_by_age2:
    print(f"{person.name} - {person.age}")

```

```

Mike - 26
Tomas - 41
Mike - 21
Mike - 26
Mike - 26
Bill - 40
Jerry - 25
Bill - 35
Donald - 20
Duck - 25
System check identified no issues (0 silenced).
February 11, 2024 - 19:43:08
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

[11/Feb/2024 19:43:24] "POST / HTTP/1.1" 200 10664

```

Рисунок 2.13 – Результат фільтрації даних

У першому випадку шукаємо один об'єкт на ім'я:

```
robin = Person.objects.get(name="Robin")
```

або пошук за віком

```
people_by_age1 = Person.objects.filter(age=15)
```

Розглянемо специфікатори, які можна використовувати.

2.6.1 exact та iexact

exact вибирає всі об'єкти моделей, у яких властивість дорівнює певному значенню. *inexact* виконує те саме завдання, тільки вибірка значень не залежить від регістру:

```
# отримуємо користувача на ім'я Tom
tom = Person.objects.get(name__exact="Tom")

# отримуємо користувачів, у яких вік дорівнює 32
people_by_age = Person.objects.filter(age__exact=32)

# отримуємо користувача на ім'я Tom або tom або TOM
tom = Person.objects.get(name__iexact="Tom")
```

Варто враховувати, що в деяких СУБД, наприклад, у MySQL, пошук за рядками регістрозалежний, тобто "Tom" і "tom" - це два різні рядки.

Якщо необхідно знайти дані, де поле таблиці має значення *NULL*, то властивості моделі передається значення *None*:

```
people_by_name = Person.objects.filter(name__exact=None)
```

2.6.2 contains та icontains

contains знаходить рядки, які містять деякий підрядок, причому пошук регістрозалежний. *icontains* виконує аналогічне завдання, за тим винятком, що пошук не залежить від регістру:

```
# отримуємо користувачів, у яких ім'я містить букву o
people1 = Person.objects.filter(name__contains="o")

# отримуємо користувачів, у яких ім'я містить букву T або t
people2 = Person.objects.filter(name__icontains="T")
```

2.6.3 in

in знаходить моделі з властивістю, значення якої дорівнює одному зі значень зі списку:

```
# отримуємо користувачів, у яких вік дорівнює або 32, або 35, або 38,
people = Person.objects.filter(age__in=[32, 35, 38])
```

2.6.4 gt, gte, lt, lte

Низка специфікаторів дає змогу знайти об'єкти, властивості яких більші або менші за певне значення: *gt* (більше ніж), *gte* (більше ніж або дорівнює), *lt* (менше ніж), *lte* (менше ніж або дорівнює):

```
# отримуємо користувачів, у яких вік менше або дорівнює 32
people = Person.objects.filter(age__lte=32)

# отримуємо користувачів, у яких вік більше 40
people = Person.objects.filter(age__gt=40)
```

2.6.5 startswith та istartswith

startswith вибирає об'єкти моделей зі значеннями, які починаються на певний рядок. Вибірка залежить від регістру.

istartswith виконує те саме завдання, тільки вибірка не залежить від регістру.

```
# отримуємо користувачів, у яких ім'я починається з То
people = Person.objects.filter(name__startswith="To")

# отримуємо користувачів, у яких ім'я починається з То або to
people = Person.objects.filter(name__istartswith="To")
```

2.6.6 endswith та iendswith

endswith вибирає об'єкти моделей зі значеннями, які закінчуються на певний рядок. Вибірка залежить від регістру.

iendswith виконує те саме завдання, тільки вибірка не залежить від регістру.

```
# отримуємо користувачів, у яких ім'я закінчується на m
people = Person.objects.filter(name__endswith="m")

# отримуємо користувачів, у яких ім'я закінчується на m або M
people = Person.objects.filter(name__iendswith="m")
```

2.6.7 range

range визначає діапазон, у який має входити значення властивості моделі:

```
# отримуємо користувачів, у яких вік у діапазоні від 28 до 38 включно
people = Person.objects.filter(age__range=(28, 38))
```

2.6.8 isnull

isnull вибирає об'єкти моделей, у яких поле в таблиці має значення *NULL* (при значенні *True*) або, навпаки, не має значення *NULL* (при значенні *False*)

```
# отримуємо користувачів, у яких ім'я не встановлено
people = Person.objects.filter(name__isnull=True)

# отримуємо користувачів, у яких вік встановлено
people = Person.objects.filter(age__isnull=False)
```

2.6.8 regex та iregex

regex і *iregex* задають регулярний вираз, якому має відповідати значення властивості моделі. У випадку з *regex* вибірка залежить від регістру, а у *iregex* - не залежить від регістру.

```
# отримуємо користувачів, у яких ім'я закінчується на am або om
people = Person.objects.filter(name__regex=r"(am|om)$")
```

2.6.9 Специфікатори дат

Цілий ряд специфікаторів фільтрації призначений для роботи з датами:

– *date*: значення поля має відповідати певній даті. Представляє об'єкт *datetime.date*

- *year*: рік дати має відповідати певному значенню.
- *month*: визначає місяць дати
- *day*: визначає день дати
- *week*: визначає номер тижня дати (1-52 або 53)
- *week_day*: визначає день тижня дати (від 1 (неділя) до 7 (субота))
- *iso_week_day*: визначає день тижня дати (від 1 (понеділок) до 7 (неділя))
- *quarter*: визначає номер кварталу дати
- *time*: визначає час дати. Представляє об'єкт *datetime.time*
- *hour*: визначає годину дати (від 0 до 23)
- *minute*: визначає хвилину дати (від 0 до 59)
- *second*: визначає секунду дати (від 0 до 59)

Усі ці специфікатори, крім *date* і *time*, приймають цілі числа. Припустимо, у нас є така модель:

```
from django.db import models

class Order(models.Model):
    datetime = models.DateTimeField()
```

Отримаємо замовлення за певні місяці:

```
from .models import Order
from datetime import datetime

Order.objects.all().delete()

# додавання початкових даних
if Order.objects.count() == 0:
    Order.objects.create(datetime = datetime(2022, 12, 26, 11, 25, 34))
    Order.objects.create(datetime = datetime(2023, 5, 12, 12, 25, 34))
    Order.objects.create(datetime = datetime(2023, 5, 22, 13, 25, 34))
    Order.objects.create(datetime = datetime(2023, 3, 19, 14, 25, 34))

# отримуємо замовлення, зроблені в 5-му місяці
orders = Order.objects.filter(datetime__month=5)
print ('test datetime__month=5')
for order in orders:
    print(order.datetime)

# отримуємо замовлення, зроблені після 5-го місяця
orders = Order.objects.filter(datetime__month__gt=5)
print ('test datetime__month__gt=5')
for order in orders:
    print(order.datetime)
```

```
Administrator: Командная строка
Quit the server with CTRL-BREAK.

C:\django\wp5\lab_db\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

test datetime__month=5
2023-05-12 12:25:34
2023-05-22 13:25:34
test datetime__month__gt=5
2022-12-26 11:25:34
System check identified no issues (0 silenced).
February 11, 2024 - 20:21:02
Django version 4.2.7, using settings 'wp5.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.13 – Результат обробки дат

До специфікаторів дат можна додавати додаткові специфікатори, щоб конкретизувати умову, як у другому випадку: `datetime__month__gt=5`.

Задля коректної роботи з датами необхідно у налаштування відключити

```
USE_TZ = False
```

Пошук за датою та часом:

```
from .models import Order
from datetime import datetime, date, time

if Order.objects.count() == 0:
    Order.objects.create(datetime = datetime(2021, 12, 26, 11, 25, 34))
    Order.objects.create(datetime = datetime(2022, 5, 12, 12, 25, 34))
    Order.objects.create(datetime = datetime(2022, 5, 22, 13, 25, 34))
    Order.objects.create(datetime = datetime(2022, 8, 19, 14, 25, 34))

# отримуємо замовлення, зроблені 22 травня
orders = Order.objects.filter(datetime__date=date(2022, 5, 22))
for order in orders:
    print(order.datetime)

# отримуємо замовлення, зроблені після 12 години
orders = Order.objects.filter(datetime__time__gt=time(12, 20, 0))
for order in orders:
    print(order.datetime)
```

2.6.10 Логічні оператори

Логічні оператори дають змогу скомбінувати дві вибірки. Є такі логічні оператори: *AND* (&), *OR* (|) і *XOR* (^)

Оператор *AND* (&) вказує, що обидві умови мають бути істинними:

```
people = Person.objects.filter(name="Jerry") & Person.objects.filter(age=22)
```

У цьому випадку в базі даних буде йти пошук рядка, в якому одночасно поле *name* дорівнює "Jerry" і поле *age* дорівнює 22. А на рівні бази даних це буде вираз:

```
SELECT ... WHERE name='Jerry' AND age=22
```

Оператор *OR* (`|`) вказує, що достатньо, щоб одна з двох умов була істинною:

```
people = Person.objects.filter(name="Jerry") | Person.objects.filter(age=22)
```

У цьому випадку в базі даних буде йти пошук рядка, в якому або поле *name* дорівнює "Jerry", або поле *age* дорівнює 22. А на рівні бази даних це буде вираз:

```
SELECT ... WHERE name='Jerry' OR age=22
```

Оператор *XOR* (`^`) вказує, що необхідно, щоб тільки одна з двох умов була істинною:

```
people = Person.objects.filter(name="Jerry") ^ Person.objects.filter(age=22)
```

У цьому разі в базі даних буде йти пошук рядка, в якому істинна або умова *name="Jerry"*, або поле умова *age=22*, але не одночасно обидві умови. На рівні бази даних можуть формуватися різні вирази залежно від підтримки оператора *XOR*.

2.7 Сортування

2.7.1 Функція *order_by()*

Для сортування за зростанням застосовується функція *order_by()*. У неї передаються поля, за якими йде сортування:

```
from .models import Person

# упорядковуємо за ім'ям за зростанням
people = Person.objects.order_by("name")
for person in people:
    print(person.name)
```

Подібним чином у функцію можна передати і більшу кількість полів, за якими послідовно буде виконуватися сортування:

```
# упорядковуємо за ім'ям і віком
people = Person.objects.order_by("name", "age")
```

Для сортування за спаданням перед назвою поля вказується знак мінус:

```
# упорядковуємо за ім'ям за спаданням
people = Person.objects.order_by("-name")
```

2.7.2 Функції `values()` та `values_list()`

Методи `values()` і `values_list()` призначені для оптимізації: для вилучення піднабору даних без необхідності створення повного об'єкта моделі.

Функція `values()` повертає об'єкт `QuerySet`, де кожен об'єкт моделі представлений у вигляді словника:

```
from .models import Person

people = Person.objects.values()
print(people)
```

The screenshot shows a terminal window titled "Administrator: Командная строка". It displays the output of a Django development server. At the top, it says "C:\django\wp6\lab_db2\views.py changed, reloading." followed by "Watching for file changes with StatReloader" and "Performing system checks...". The main output is a `<QuerySet>` containing seven dictionaries, each representing a person with 'id', 'name', and 'age' attributes. Below the `<QuerySet>`, it says "System check identified no issues (0 silenced)." and "February 12, 2024 - 10:05:11". At the bottom, it says "Django version 4.2.7, using settings 'wp6.settings'", "Starting development server at http://127.0.0.1:8000/", and "Quit the server with CTRL-BREAK."

```
C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

<QuerySet [{ 'id': 1, 'name': 'Tim', 'age': 26}, { 'id': 2, 'name':
'Bill', 'age': 40}, { 'id': 3, 'name': 'Jerry', 'age': 25}, { 'id'
: 4, 'name': 'Bill', 'age': 35}, { 'id': 5, 'name': 'Donald', 'age
': 20}, { 'id': 6, 'name': 'Duck', 'age': 25}, { 'id': 7, 'name': '
Pluto', 'age': 25}]>
System check identified no issues (0 silenced).
February 12, 2024 - 10:05:11
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.14 – Результат виборки функцією `values()`

Також можна передати в метод `values()` назви полів, які мають бути в словнику (за замовчуванням обираються всі поля моделі):

```
from .models import Person

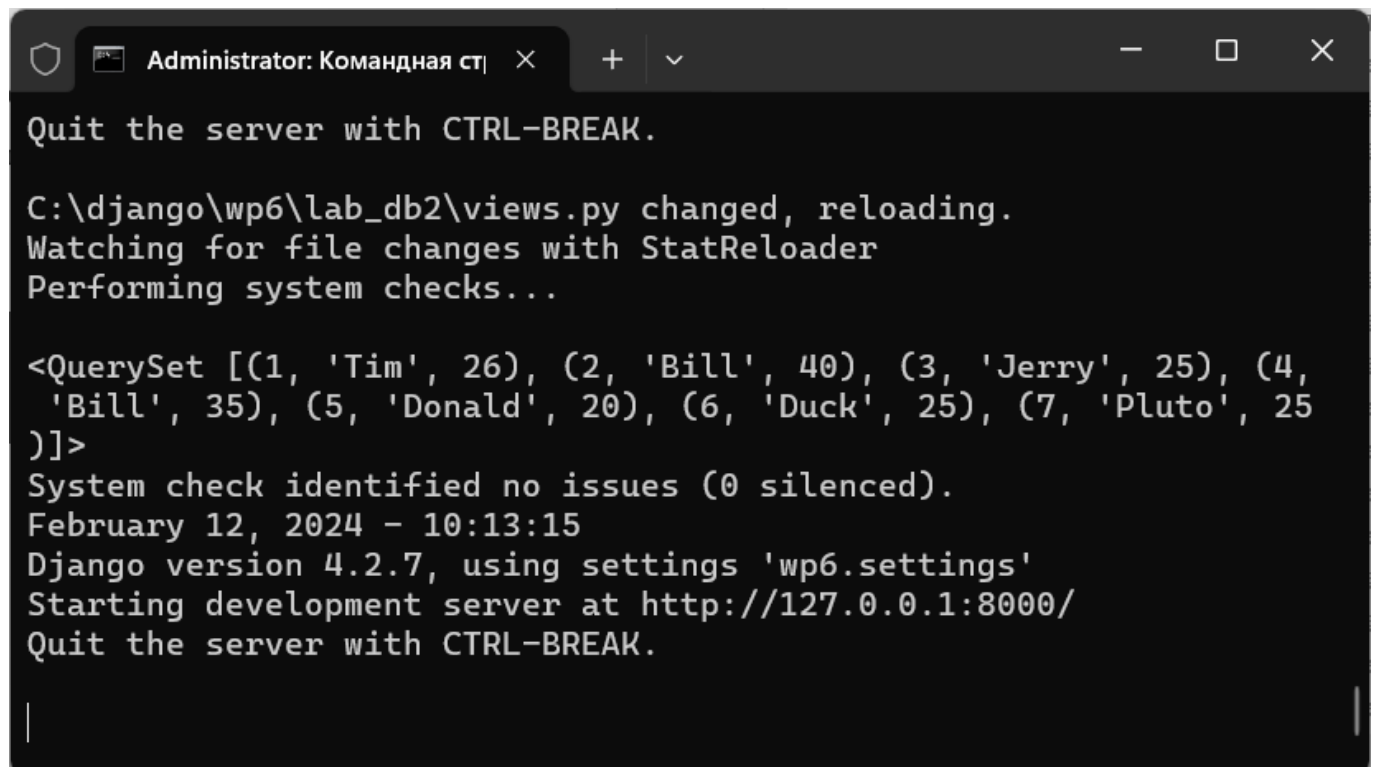
people = Person.objects.values("id", "name")
print(people)
```

У цьому випадку словник включатиме тільки поля *id* і *name*.

Метод `values_list()` багато в чому аналогічний `values()`: він повертає об'єкт `QuerySet`, який складається з кортежів. Кожен кортеж зберігає дані одного об'єкта моделі. Наприклад:

```
from .models import Person

people = Person.objects.values_list()
print(people)
```



The screenshot shows a terminal window titled "Administrator: Командная строка". The output of the Django development server is as follows:

```
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

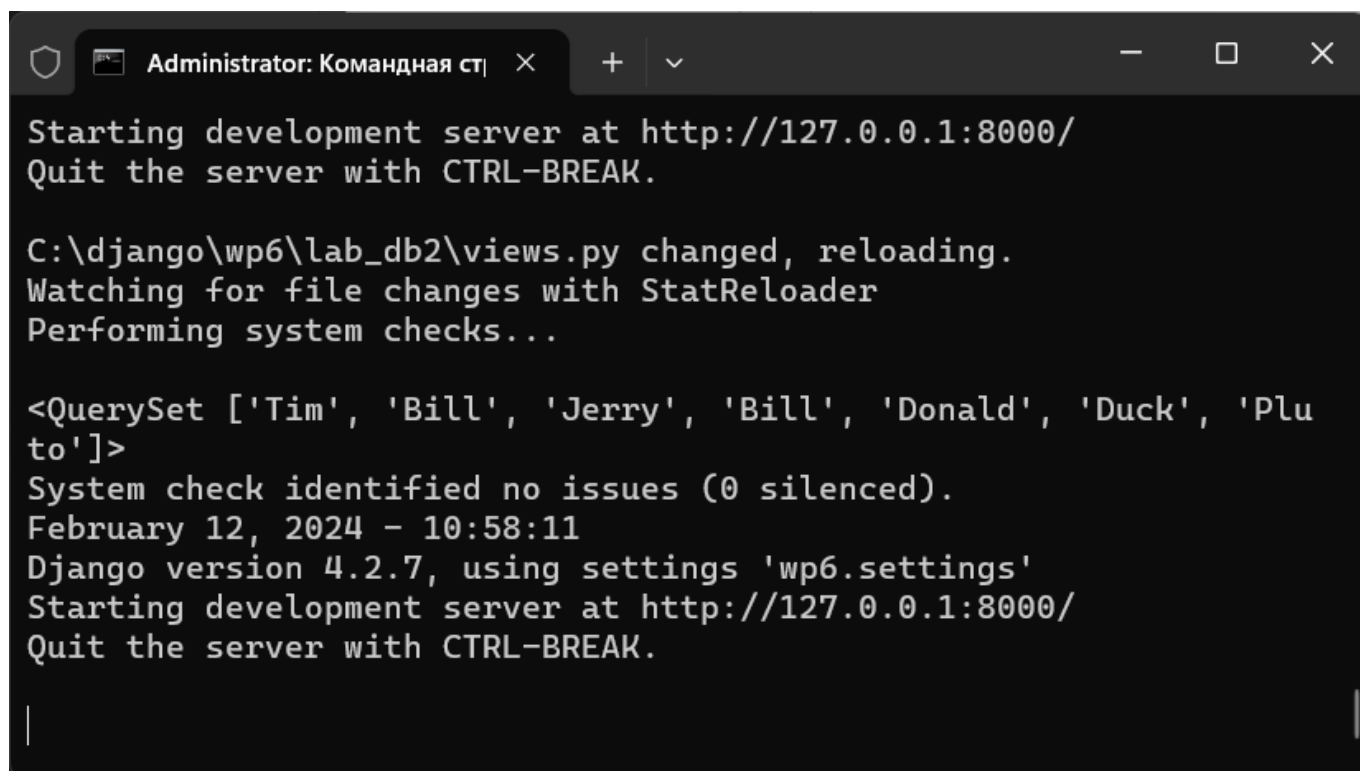
<QuerySet [(1, 'Tim', 26), (2, 'Bill', 40), (3, 'Jerry', 25), (4, 'Bill', 35), (5, 'Donald', 20), (6, 'Duck', 25), (7, 'Pluto', 25)]>
System check identified no issues (0 silenced).
February 12, 2024 - 10:13:15
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.15 – Результат виборки функцією `values_list()`

Також можна вибрати окремі поля, передавши їхні назви в метод:

```
people = Person.objects.values_list("id", "name")
print(people)
```

Якщо вибирається тільки одне поле, то в підсумку вийде набір кортежів, у кожному з яких буде по одному значенню. Але передавши параметру *flat* значення *True* можна спростити набір, винісши значення на рівень вище:



```
Administrator: Командная строка
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

<QuerySet ['Tim', 'Bill', 'Jerry', 'Bill', 'Donald', 'Duck', 'Pluto']>
System check identified no issues (0 silenced).
February 12, 2024 - 10:58:11
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.16 – Результат виборки

2.8 Операції з множинами

2.8.1 Функція `distinct()`

Функція *distinct()* вибирає унікальні значення:

```
from .models import Person

print('без distinct()')
people = Person.objects.values_list("name", flat=True)
print(people)

print('з distinct()')
people = Person.objects.values_list("name", flat=True).distinct()
print(people)
```



```

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

без distinct()
<QuerySet ['Tim', 'Bill', 'Jerry', 'Bill', 'Donald', 'Duck', 'Pluto']>
з distinct()
<QuerySet ['Tim', 'Bill', 'Jerry', 'Donald', 'Duck', 'Pluto']>
System check identified no issues (0 silenced).
February 12, 2024 - 11:26:21
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

Рисунок 2.17 – Результат застосування функції *distinct()*

У цьому випадку отримуємо набір з усіх імен користувачів. Але в другому випадку також додаємо метод *distinct()*, який дає змогу відсікти дублі.

2.8.2 Функція *union()*

Метод *union()* повертає *QuerySet*, який є об'єднанням двох *QuerySet*, що на рівні бази даних відповідає виконанню оператора *UNION*:

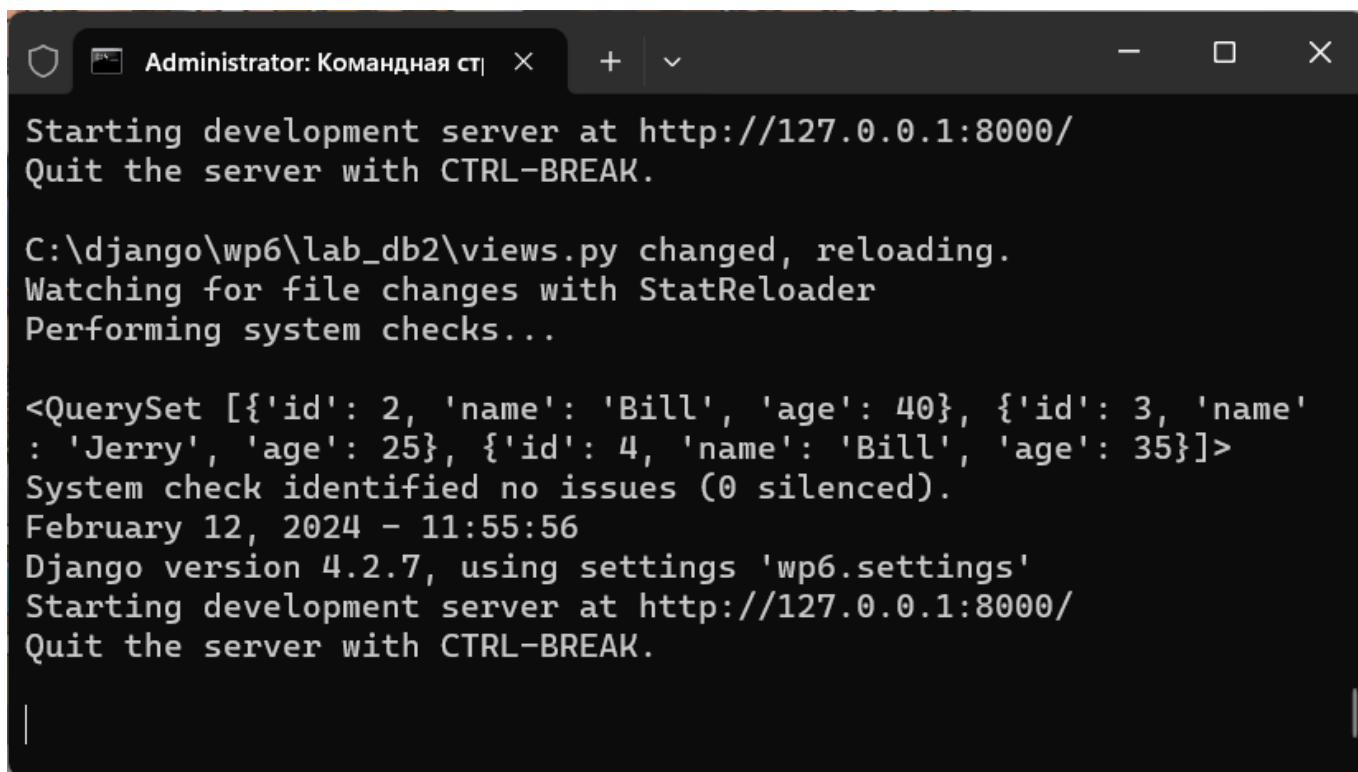
```

from .models import Person

bills = Person.objects.filter(name="Bill")
jerris = Person.objects.filter(name="Jerry")

# объединяем два QuerySet
people = bills.union(jerris)
print(people.values())

```



```

Administrator: Командная строка
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

<QuerySet [{'id': 2, 'name': 'Bill', 'age': 40}, {'id': 3, 'name': 'Jerry', 'age': 25}, {'id': 4, 'name': 'Bill', 'age': 35}]>
System check identified no issues (0 silenced).
February 12, 2024 - 11:55:56
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

Рисунок 2.18 -Результат виконання функції `union()`

За замовчуванням метод `union()` вибирає тільки унікальні значення з обох вибірок. Якщо така поведінка небажана, то в метод необхідно передати аргумент `all=True`:

```

from .models import Person
bills = Person.objects.filter(name="Bill")
jerris = Person.objects.filter(name="Jerry")

print('вибираємо тільки унікальні (за замовчуванням)')
people = bills.values("name").union(jerris)
print(people)

print('вибираємо всі')
people = bills.values("name").union(jerris, all=True)
print(people)

```

```

Administrator: Командная строка
C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

вибираємо тільки унікальні (за замовчуванням)
<QuerySet [{'name': 'Bill'}, {'name': 'Jerry'}]>
вибираємо всі
<QuerySet [{'name': 'Bill'}, {'name': 'Bill'}, {'name': 'Jerry'}]>
>
System check identified no issues (0 silenced).
February 12, 2024 - 16:44:29
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

Рисунок 2.18 – Відмінність виборки

Метод може приймати кілька вибірок, які об'єднуються з першою вибіркою.

2.8.3 Функція *intersection()*

Метод *intersection()* повертає *QuerySet* у вигляді переліку інших *QuerySet* (тобто знаходить об'єкти, які є у всіх вибірках). На рівні бази даних це буде відповідати виконанню SQL-оператора *INTERSECT*:

```

from .models import Person

bills = Person.objects.filter(name="Bill")
print(bills.values())

less37 = Person.objects.filter(age__lt=37)
print(less35.values())

print('знаходимо перетин двох QuerySet')
people = bills.intersection(less37)
print(people.values())

```

```
Administrator: Командная ст| X + v - □ X
<QuerySet [{ 'id': 2, 'name': 'Bill', 'age': 40}, { 'id': 4, 'name':
: 'Bill', 'age': 35}]>
<QuerySet [{ 'id': 1, 'name': 'Tim', 'age': 26}, { 'id': 3, 'name':
'Jerry', 'age': 25}, { 'id': 4, 'name': 'Bill', 'age': 35}, { 'id'
: 5, 'name': 'Donald', 'age': 20}, { 'id': 6, 'name': 'Duck', 'age
': 25}, { 'id': 7, 'name': 'Pluto', 'age': 25}]>
знаходимо перетин двох QuerySet
<QuerySet [{ 'id': 4, 'name': 'Bill', 'age': 35}]>
System check identified no issues (0 silenced).
February 12, 2024 - 16:51:24
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
|
```

Рисунок 2.19 – Результат перетину двох *QuerySet*

Метод може приймати кілька вибірок, які перетинаються з першою вибіркою.

2.8.4 Функція *difference()*

Метод *difference()* повертає *QuerySet* у вигляді різниці інших *QuerySet* (тобто знаходить об'єкти, які є в першій вибірці, але відсутні в інших). На рівні бази даних це буде відповідати виконанню SQL-оператора *EXCEPT*:

```
from .models import Person
bills = Person.objects.filter(name="Bill")
print(bills.values())

less37 = Person.objects.filter(age__lt=37)
print(less37.values())

print('знаходимо різницю двох QuerySet')
people = bills.difference(less37)
print(people.values())
```

```

Administrator: Командная строка
<QuerySet [{ 'id': 2, 'name': 'Bill', 'age': 40}, { 'id': 4, 'name': 'Bill', 'age': 35}]>
<QuerySet [{ 'id': 1, 'name': 'Tim', 'age': 26}, { 'id': 3, 'name': 'Jerry', 'age': 25}, { 'id': 4, 'name': 'Bill', 'age': 35}, { 'id': 5, 'name': 'Donald', 'age': 20}, { 'id': 6, 'name': 'Duck', 'age': 25}, { 'id': 7, 'name': 'Pluto', 'age': 25}]>
знаходимо різницю двох QuerySet
<QuerySet [{ 'id': 2, 'name': 'Bill', 'age': 40}]>
System check identified no issues (0 silenced).
February 12, 2024 - 16:58:26
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

Пмсуюнок 2.20 – Результат різниці двох *QuerySet*

Метод може приймати безліч вибірок, які треба відняти від першої.

2.9 Отримання окремих об'єктів і перевірка їх наявності

2.9.1 Функції *latest()* та *earliest()*

Метод *latest()* (і його асинхронна версія *alatest()*) повертає об'єкт, де були найостанніші зміни в певних полях.

Метод *earliest()* (і його асинхронна версія *aeartliest()*) повертає об'єкт, де були найраніші зміни у визначених полях.

Як параметр обидва методи приймають набір полів, на підставі яких повертається об'єкт:

```
from .models import Person

print("отримуємо об'єкт з найостаннішими змінами в полі id")
latest_person = Person.objects.latest("id")
print(f"{latest_person.name} - {latest_person.age}")

print("отримуємо об'єкт із найбільш ранніми змінами в полі name")
earliest_person = Person.objects.earliest("id")
print(f"{earliest_person.name} - {earliest_person.age}")
```

```
C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

отримуємо об'єкт з найостаннішими змінами в полі id
Pluto - 25
отримуємо об'єкт із найбільш ранніми змінами в полі name
Tim - 26
System check identified no issues (0 silenced).
February 12, 2024 - 17:08:57
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.21 – Результат виборки найстарішого та найновішого запису

Можна передати набір полів:

```
# отримуємо об'єкт з найостаннішими змінами в полі age
# і з найбільш ранніми змінами в полі name
latest_person = Person.objects.latest("age", "-name")
print(f"{latest_person.name} - {latest_person.age}")
```

Якщо передається поле з мінусом, то пошук відбувається у зворотному порядку: для методу *latest()* відбувається пошук поля з найбільш ранніми змінами, а для методу *earliest()* - з найпізнішими змінами.

Якщо жодного об'єкта не знайдено (наприклад, у випадку з порожнім набором), обидва методи генерують виняток *DoesNotExist*.

2.9.2 Функції *first()* і *last()*

Метод *first()* (і його асинхронна версія *afirst()*) повертає перший об'єкт із набору.

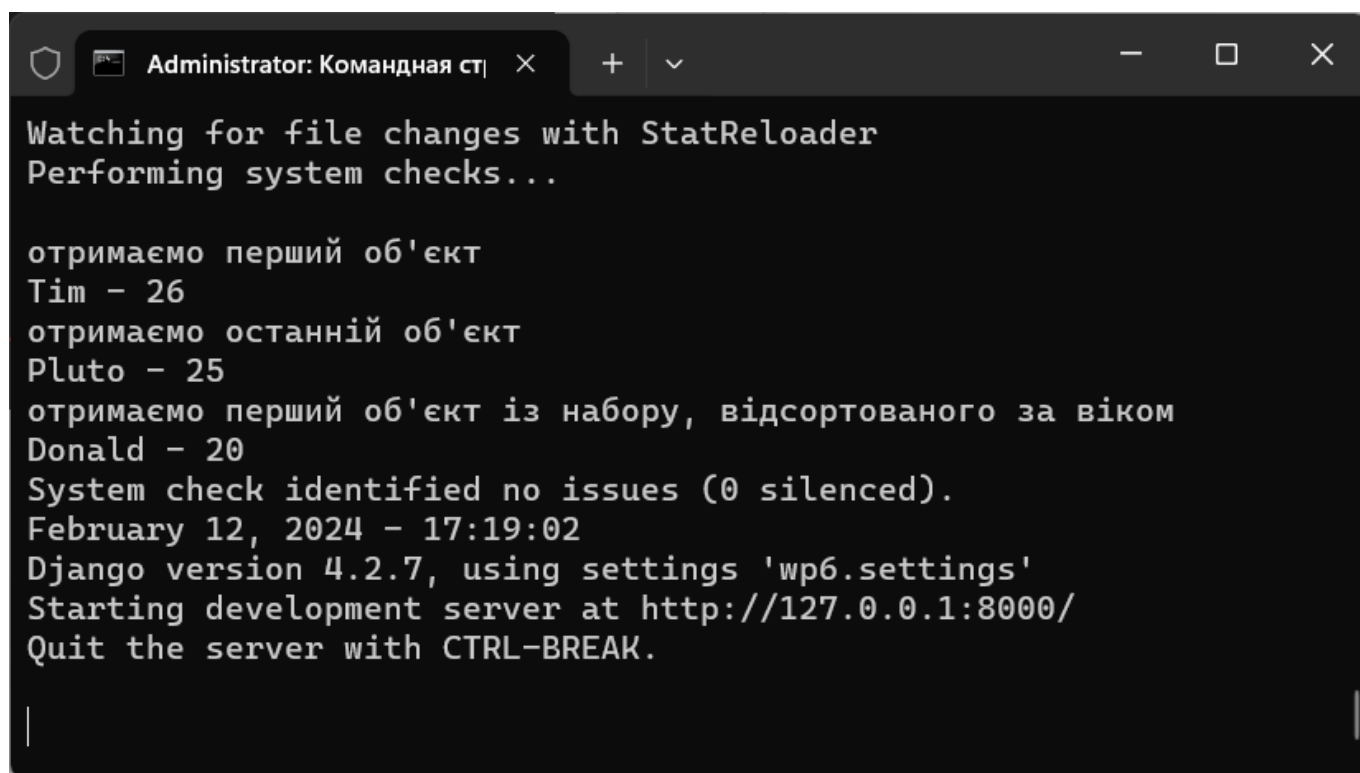
Метод *last()* (і його асинхронна версія *alast()*) повертає останній об'єкт із набору.

```
from .models import Person

print("отримаємо перший об'єкт")
first_person = Person.objects.first()
print(f"{first_person.name} - {first_person.age}")

print("отримаємо останній об'єкт")
last_person = Person.objects.last()
print(f"{last_person.name} - {last_person.age}")

print("отримаємо перший об'єкт із набору, відсортованого за віком")
first_person = Person.objects.order_by("age").first()
print(f"{first_person.name} - {first_person.age}")
```



```
Administrator: Командная стл  X + v
Watching for file changes with StatReloader
Performing system checks...

отримаємо перший об'єкт
Tim - 26
отримаємо останній об'єкт
Pluto - 25
отримаємо перший об'єкт із набору, відсортованого за віком
Donald - 20
System check identified no issues (0 silenced).
February 12, 2024 - 17:19:02
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.22 – Результат пошуку першого та останнього запису

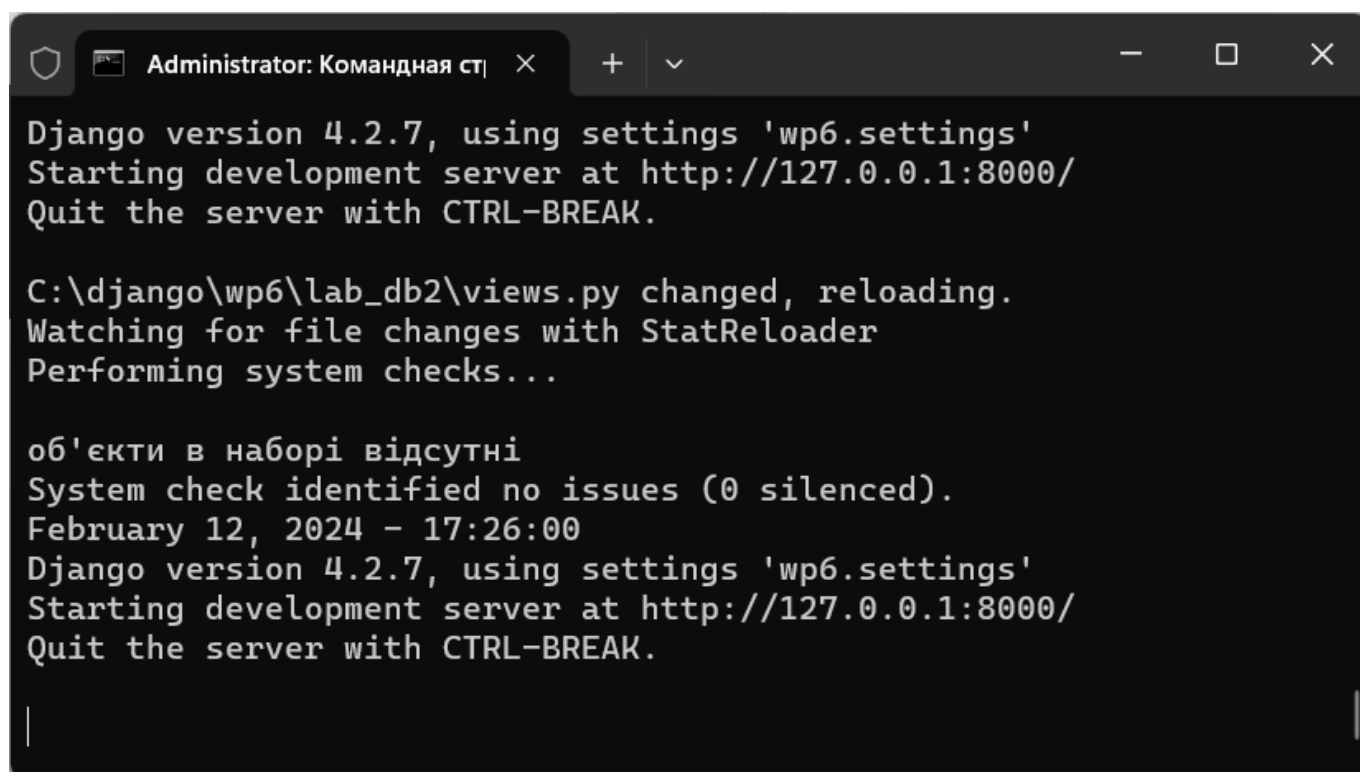
Якщо жодного об'єкта не знайдено (наприклад, у випадку з порожнім набором), обидва методи повертають значення `None`

2.9.3 Функція `exists()`

Метод `exists()` (і його асинхронна версія `aexists()`) повертає `True`, якщо набір `QuerySet` містить дані, і `False` - якщо не містить:

```
from .models import Person

is_present = Person.objects.filter(name = "Jack").exists()
if is_present:
    print("у наборі є об'єкти")
else:
    print("об'єкти в наборі відсутні")
```



```
Administrator: Командная строка
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

об'єкти в наборі відсутні
System check identified no issues (0 silenced).
February 12, 2024 - 17:26:00
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.23 – Результат перевірки наявності об'єкта

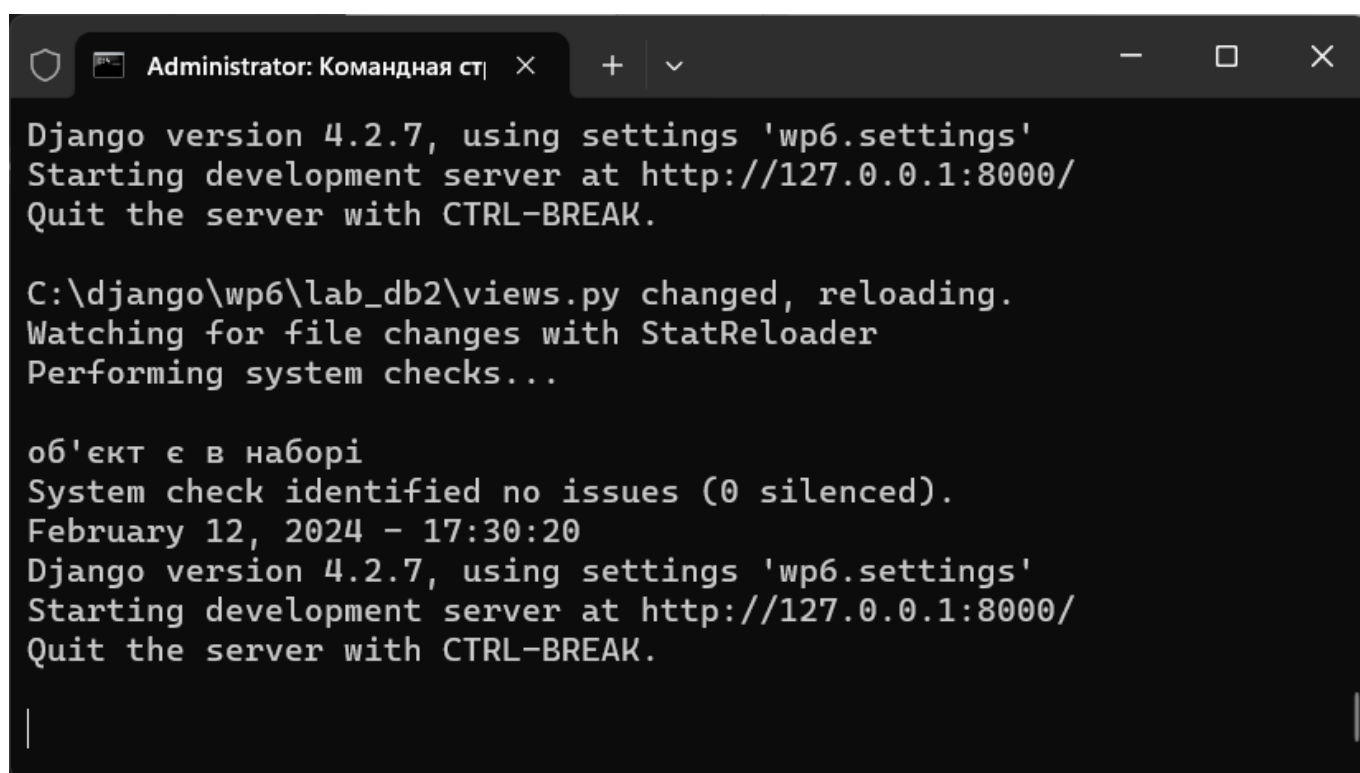
2.9.4 Функція contains()

Метод `contains()` (і його асинхронна версія `contains()`) повертає `True`, якщо набір `QuerySet` містить певний об'єкт, і `False` - якщо не містить:

```
from .models import Person

# отримаємо останній об'єкт
last_person = Person.objects.last()

# чи є об'єкт last_person серед тих, у яких age менше 35
is_present = Person.objects.filter(age__lt=35).contains(last_person)
if is_present:
    print("об'єкт є в наборі")
else:
    print("об'єкт відсутній")
```



```
Administrator: Командная строка
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

об'єкт є в наборі
System check identified no issues (0 silenced).
February 12, 2024 - 17:30:20
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.24 – Результат перевірки наявності об'єкта

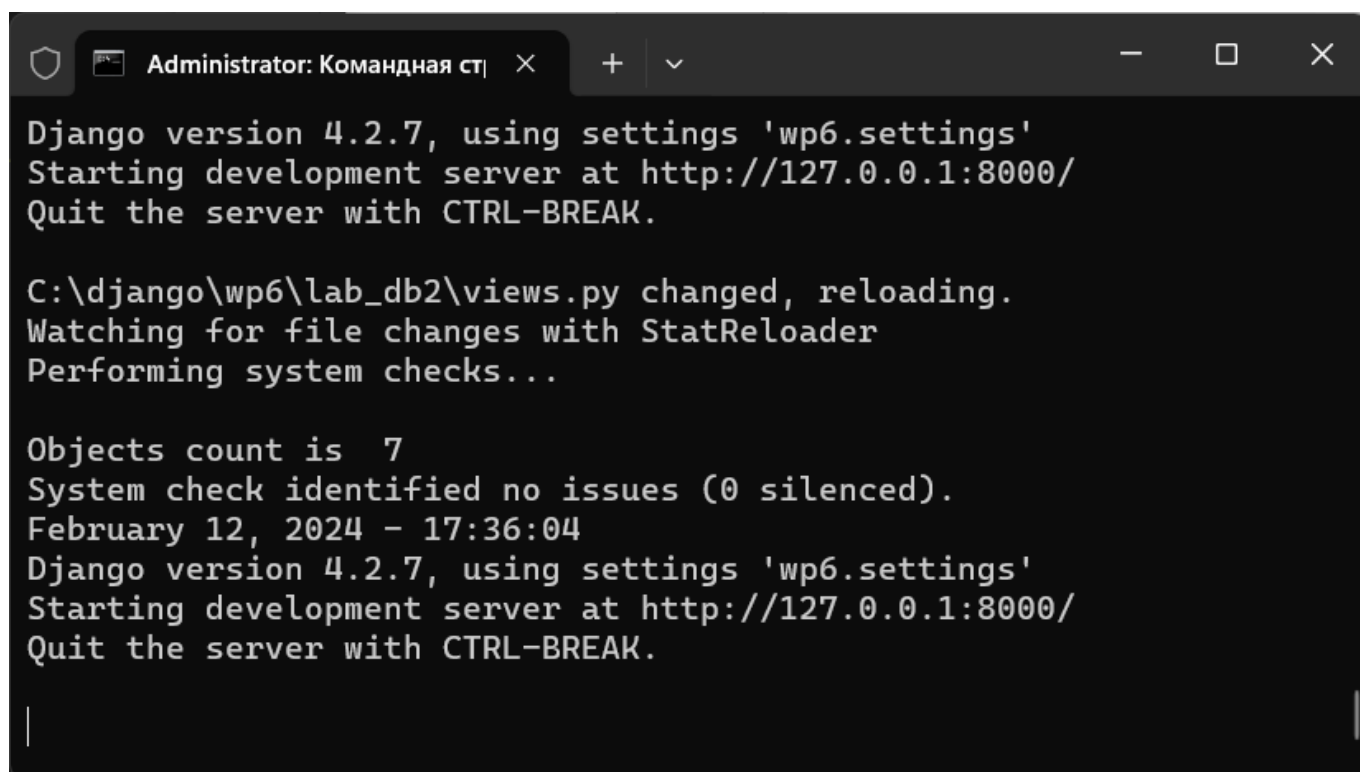
2.10 Агрегатні функції

2.10.1 Кількість об'єктів

Для обчислення кількості об'єктів у *QuerySet* застосовується метод *count()* і його асинхронна версія *acount()*:

```
from .models import Person

number = Person.objects.count()
print('Objects count is ', number)
```



```
Administrator: Командная строка
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

Objects count is 7
System check identified no issues (0 silenced).
February 12, 2024 - 17:36:04
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.24 – Результат визначення кількості записів

На рівні бази даних метод трансліюється в SQL-вираз *SELECT COUNT(*)*

У Django для підрахунку кількості об'єктів також може використовуватися функція *len()*. У неї передається *QuerySet*, для якого треба обчислити кількість об'єктів.

```
from .models import Person

people = Person.objects.all()
number = len(people)
print("кількість об'єктів у наборі people =", number)
```

```
Administrator: Командная строка
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

кількість об'єктів у наборі people = 7
System check identified no issues (0 silenced).
February 12, 2024 - 17:38:21
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.25 – Результат визначення кількості записів

Що використовувати: *count()* чи *len()*? Якщо об'єкти вже раніше були завантажені (наприклад, за допомогою методу *all()*), то оптимальніше використовувати функцію *len()*, яка не виконує до бази даних запит SQL, а працює з уже завантаженим набором об'єктів. Якщо ж об'єкти раніше НЕ були завантажені, то краще виконати метод *count()*, для якого не потрібно завантажувати всі об'єкти з БД.

2.10.2 Метод *aggregate()*

Агрегатні операції дають змогу отримати одне скалярне значення, підраховане на наборі даних за певним критерієм. Для виконання агрегатних операцій застосовується метод *aggregate*, який приймає один з об'єктів таких класів:

Avg: обчислює середнє значення

Min: обчислює мінімальне значення

Max: обчислює максимальне значення

Sum: обчислює суму

Як один із параметрів конструктори цих класів приймають вираз - зазвичай властивість, за якою обчислюється значення:

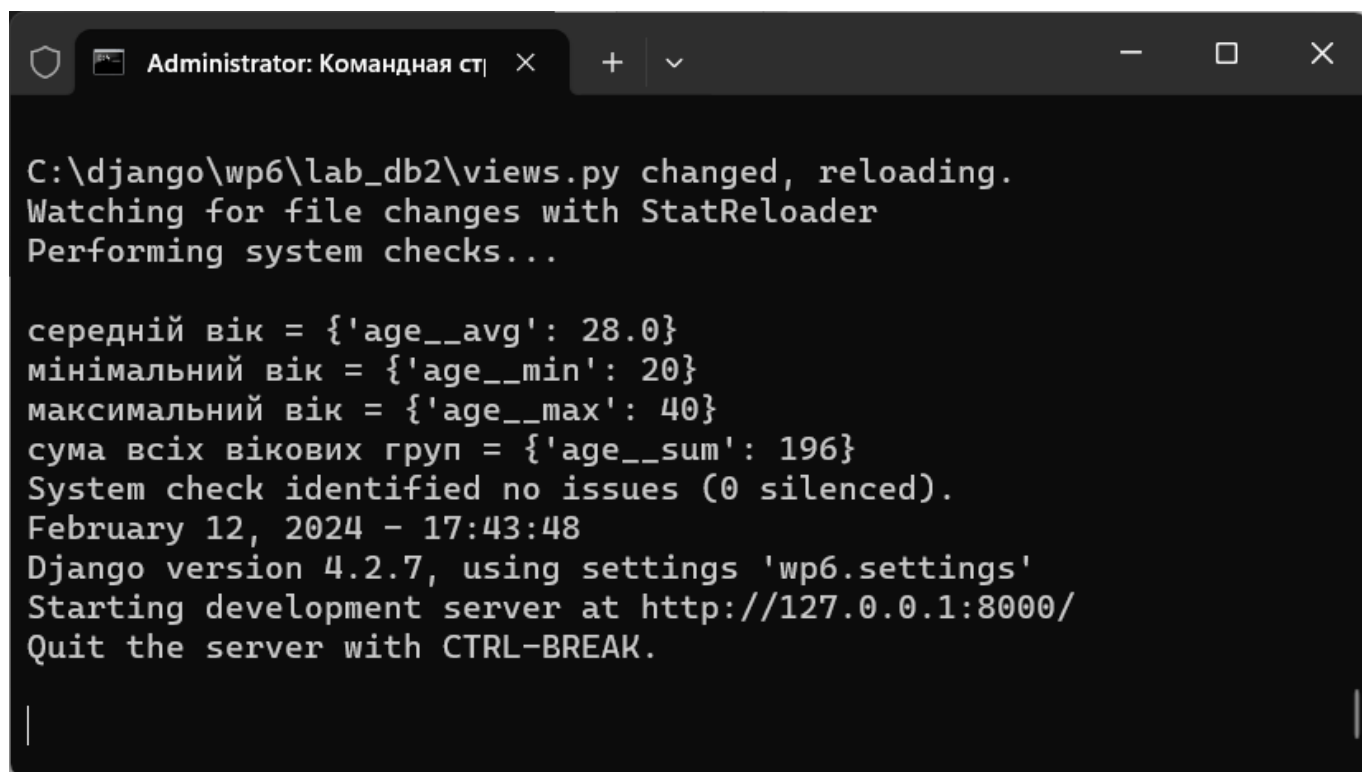
```
from .models import Person
from django.db.models import Avg, Min, Max, Sum

avg_age = Person.objects.aggregate(Avg("age"))
print("середній вік =", avg_age)

min_age = Person.objects.aggregate(Min("age"))
print("мінімальний вік =", min_age)

max_age = Person.objects.aggregate(Max("age"))
print("максимальний вік =", max_age)

sum = Person.objects.aggregate(Sum("age"))
print("сума всіх вікових груп =", sum)
```



The screenshot shows a terminal window titled "Administrator: Командная строка". The output text is as follows:

```
C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

середній вік = {'age__avg': 28.0}
мінімальний вік = {'age__min': 20}
максимальний вік = {'age__max': 40}
сума всіх вікових груп = {'age__sum': 196}
System check identified no issues (0 silenced).
February 12, 2024 - 17:43:48
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.26 – Результат виконання агрегатних функцій

2.11 Виконання SQL-виразів

Django автоматично трансліює методи *QuerySet* у відповідні SQL-вирази, які потім виконуються в базі даних. Однак фреймворк також дає змогу безпосередньо визначити SQL-запит і виконувати його.

2.11.1 Функція `row()`

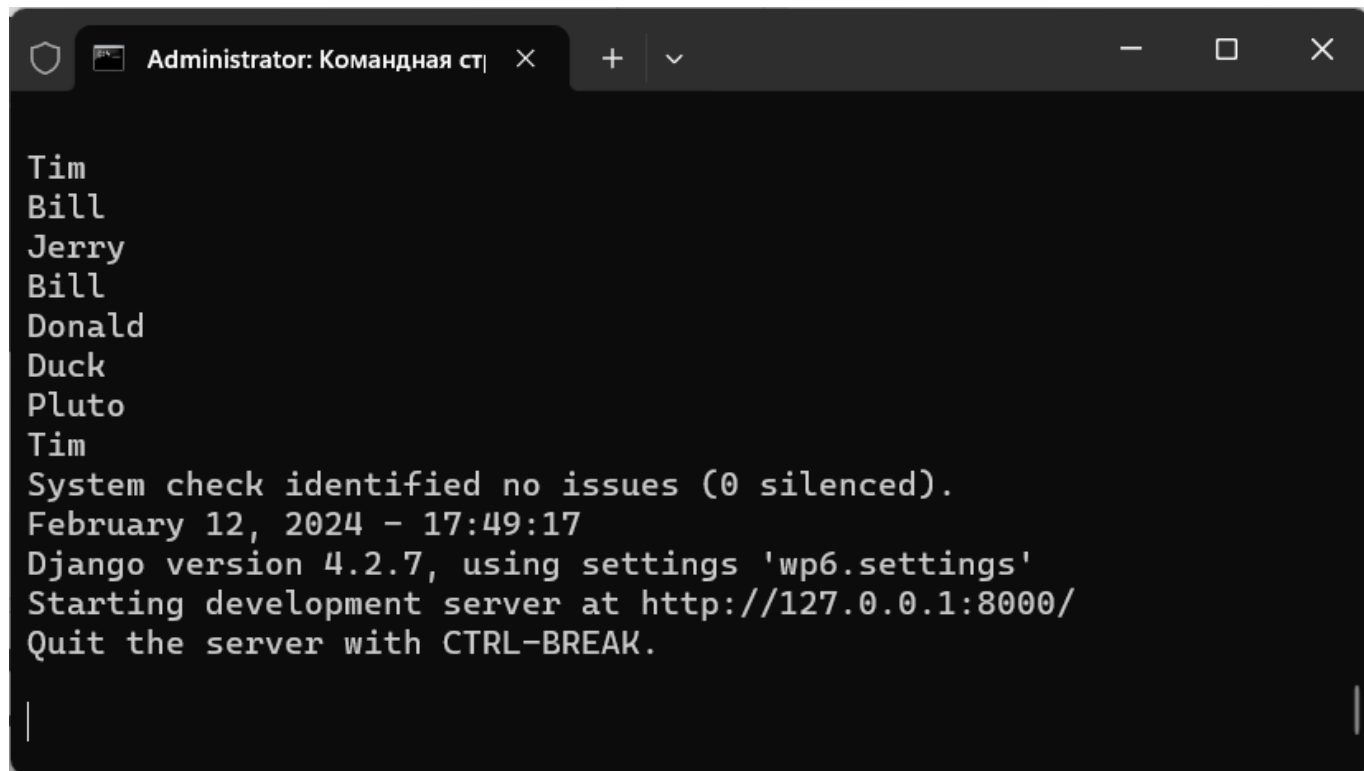
Для цього застосовується метод `raw()`, у який передається SQL-запит:

```
from .models import Person

people = Person.objects.raw("SELECT id, name FROM lab_db2_person")

# перебираємо отримані рядки
for person in people:
    print(person.name)

# отримуємо один об'єкт
print(people[0].name)
```



```
Administrator: Командная строка

Tim
Bill
Jerry
Bill
Donald
Duck
Pluto
Tim
System check identified no issues (0 silenced).
February 12, 2024 - 17:49:17
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.26 – Результат виконання `row()`

У цьому випадку в `raw()` передається запит, який отримує значення полів `id` і `name` з таблиці `lab_db2_person`. Цей метод повертає об'єкт `django.db.models.query.RawQuerySet`, який, подібно до `QuerySet`, також можна перебрати в циклі і витягти з нього дані.

Варто враховувати, що цей метод має повертати набір рядків. Також варто зазначити, що метод не враховує жодних виразів фільтрації, які йдуть до його виклику

```
# тут filter() НЕ матиме жодного впливу
people = Person.objects.filter(age__lt=35).raw("SELECT * FROM hello_person")
```

2.11.2 Передача параметрів

За допомогою параметра `params` у метод `raw()` можна передати значення для параметрів у SQL-запиті:

```
from .models import Person
# визначаємо значення для параметра name
name_for_filter = "Bill"
# визначаємо значення для параметра age
age_for_filter = 40

people = Person.objects.raw("SELECT * FROM lab_db2_person WHERE name = %s AND age = %s",
[name_for_filter, age_for_filter])
for person in people:
    print(person.name, '-', person.age)
```

У цьому випадку SQL-запит використовує два параметри, які визначаються через спецсимволи `%s`. Другий параметр методу `raw()` представляє список аргументів, які передаються в запит замість символів `%s`. Аргументи передаються за позицією, тобто значення `name_for_filter` передається на місце першого входження `%s`, друге значення - `age_for_filter` - на місце другого входження `%s` тощо.

```
Administrator: Командная строка
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

Bill - 40
System check identified no issues (0 silenced).
February 12, 2024 - 18:08:37
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Рисунок 2.27 – Результат виконання SQL-запиту

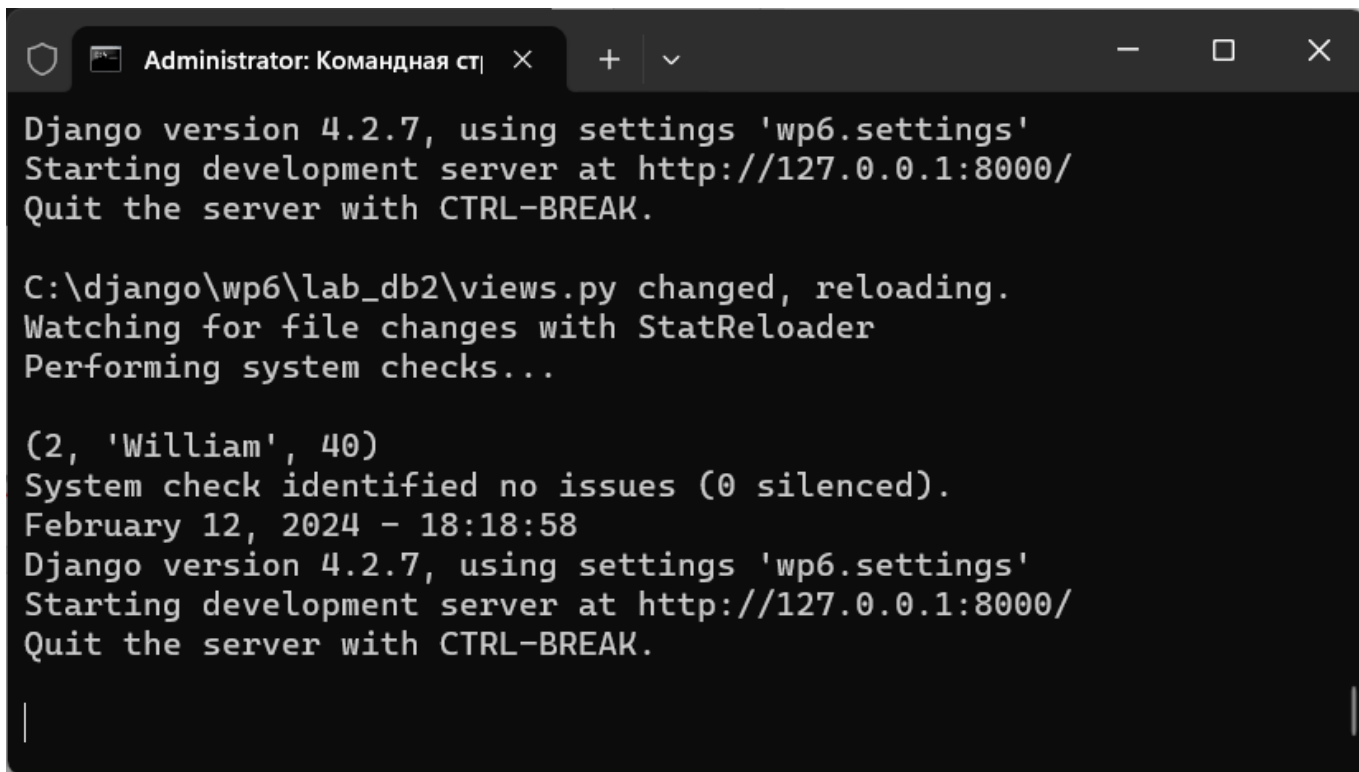
2.11.3 Виконання команд SQL

Слабким місцем методу *raw()* є те, що він дозволяє виконувати тільки команду *SELECT*, яка повертає набір рядків. Якщо ж ми хочемо виконати інші sql-команди: *UPDATE*, *INSERT* або *DELETE* чи інші типи *SELECT*-запитів, то в цьому випадку ми можемо скористатися функціоналом *django.db.connection*.

Об'єкт *django.db.connection* представляє підключення до бази даних:

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute("UPDATE lab_db2_person SET name = 'William' WHERE name='Bill' AND age=40")
    cursor.execute("SELECT * FROM lab_db2_person WHERE name = 'William'")
    row = cursor.fetchone()
    print(row)
```



```
Administrator: Командная ст | X + v
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

(2, 'William', 40)
System check identified no issues (0 silenced).
February 12, 2024 - 18:18:58
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

|
```

Рисунок 2.28 – Результат виконання SQL-запиту

Для використання підключення спочатку необхідно отримати курсор за допомогою методу `connection.cursor()`. А потім у курсора викликається метод `cursor.execute()`, в який передається виконуваний sql-запит. Для повернення результату застосовуються методи `cursor.fetchone()` і `cursor.fetchall()`.

Також можна використовувати параметри:

```
from django.db import connection

old_name = "Jerry"
new_name = "Tomas"
with connection.cursor() as cursor:
    cursor.execute("UPDATE lab_db2_person SET name =%s WHERE name=%s", [new_name,
old_name])
    cursor.execute("SELECT * FROM lab_db2_person WHERE name = 'Tomas'")
    rows = cursor.fetchall()

    for row in rows:
        print(row)
```



```

Administrator: Командная строка
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

C:\django\wp6\lab_db2\views.py changed, reloading.
Watching for file changes with StatReloader
Performing system checks...

(3, 'Tomas', 25)
System check identified no issues (0 silenced).
February 12, 2024 - 18:24:52
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

```

Рисунок 2.29 – Результат запиту на оновлення даних

2.12 CRUD. Усі базові операції з моделями у веб-додатку

Розглянемо базові операції з моделями на найпростішому прикладі. створення і виведення об'єктів моделі на прикладі. Нехай, у файлі *models.py* визначено модель *Person*:

```

from django.db import models

class Person(models.Model):
    name = models.CharField(max_length=20)
    age = models.IntegerField()

```

2.12.1 Визначення подань

У файлі *views.py* пропишемо чотири подання для отримання, збереження, редагування та видалення даних:

```

from django.shortcuts import render
from django.http import HttpResponseRedirect, HttpResponseNotFound
from .models import Person

# отримання даних із БД
def index(request):
    people = Person.objects.all()
    return render(request, "index.html", {"people": people})

# збереження даних у БД
def create(request):
    if request.method == "POST":
        person = Person()
        person.name = request.POST.get("name")
        person.age = request.POST.get("age")
        person.save()
    return HttpResponseRedirect("/")

# зміна даних у БД
def edit(request, id):
    try:
        person = Person.objects.get(id=id)

        if request.method == "POST":
            person.name = request.POST.get("name")
            person.age = request.POST.get("age")
            person.save()
            return HttpResponseRedirect("/")
        else:
            return render(request, "edit.html", {"person": person})
    except Person.DoesNotExist:
        return HttpResponseNotFound("<h2>Person not found</h2>")

# видалення даних із БД
def delete(request, id):
    try:
        person = Person.objects.get(id=id)
        person.delete()
        return HttpResponseRedirect("/")
    except Person.DoesNotExist:
        return HttpResponseNotFound("<h2>Person not found</h2>")

```

У функції *index()* отримуємо всі дані за допомогою методу *Person.objects.all()* і передаємо їх у шаблон *index.html*.

У функції *create()* отримуємо дані із запиту типу *POST*, зберігаємо дані за допомогою методу *save()* і виконуємо переадресацію на корінь веб-сайту (тобто на функцію *index()*).

Функція *edit()* виконує редагування об'єкта. Функція як параметр приймає ідентифікатор об'єкта в базі даних. І спочатку за цим ідентифікатором ми намагаємося знайти об'єкт за допомогою методу *Person.objects.get(id=id)*. Оскільки в разі відсутності об'єкта ми можемо зіткнутися з виключенням *Person.DoesNotExist*, то відповідно нам треба обробити подібне виключення, якщо раптом буде передано неіснуючий ідентифікатор. І якщо об'єкт не буде знайдений, то користувачеві буде надана помилка 404 через виклик *return HttpResponseNotFound()*.

Якщо об'єкт знайдено, то обробка ділиться на дві гілки. Якщо запит *POST*, тобто якщо користувач відправив нові змінені дані для об'єкта, то зберігаємо ці дані в БД і виконуємо переадресацію на корінь веб-сайту. Якщо запит *GET*, то відображаємо користувачеві сторінку *edit.html* з формою для редагування об'єкта.

Функція *delete()* аналогічним чином знаходить об'єкт і виконує його видалення.

2.12.2 Визначення шаблонів

У папці *templates* визначимо шаблон *index.html*, який виводитиме дані на веб-сторінку:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>CRUD operations example on Django</title>
</head>
<body>
    <form method="POST" action="create/">
        {% csrf_token %}
        <p>
            <label>Enter the Name</label><br>
            <input type="text" name="name" />
        </p>
        <p>
            <label>Enter the Age</label><br>
            <input type="number" name="age" />
        </p>
        <input type="submit" value="Сохранить" >
    </form>
    {% if people.count > 0 %}
    <h2>Users list</h2>
    <table>
```

```

<thead><th>Id</th><th>Name</th><th>Age</th><th></th></thead>
{% for person in people %}
<tr>
    <td>{{ person.id }}</td>
    <td>{{ person.name }}</td>
    <td>{{ person.age }}</td>
    <td><a href="edit/{{person.id}}">Edit</a> | <a
href="delete/{{person.id}}">Delete</a></td>
</tr>
{% endfor %}
</table>
{% endif %}
</body>
</html>

```

На початку шаблону визначено форму для додавання даних, які потім отримуватиме функція *create()* у *POST*-запиті. А нижче визначено таблицю, в яку виводяться дані з переданого з подання набору *people*. У кожному рядку визначено посилання на редагування та видалення об'єкта.

Тепер додамо в папку *templates* файл *edit.html* з таким вмістом:

```

<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>CRUD operations example on Django</title>
</head>
<body>
    <form method="POST">
        {% csrf_token %}
        <p>
            <label>Enter the Name</label><br>
            <input type="text" name="name" value="{{person.name}}" />
        </p>
        <p>
            <label>Enter the Age</label><br>
            <input type="number" name="age" value="{{person.age}}" />
        </p>
        <input type="submit" value="Save" />
    </form>
</body>
</html>

```

Тут визначено форму для редагування об'єкта. Після натискання на кнопку введені на форму дані будуть йти за тією ж адресою в запиті *POST*.

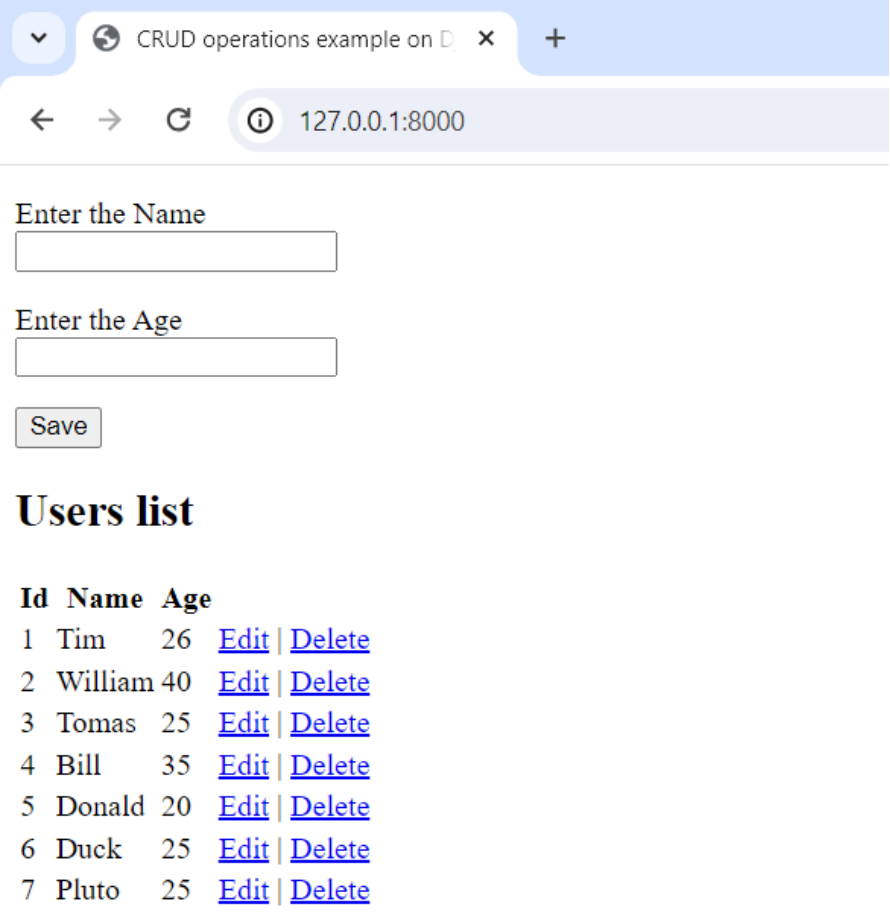
2.12.3 Встановлення маршрутів

І також у файлі *urls.py* проєкту зв'яжемо маршрути з поданнями:

```
from django.urls import path
from lab_db2 import views

urlpatterns = [
    path("", views.index),
    path("create/", views.create),
    path("edit/<int:id>/", views.edit),
    path("delete/<int:id>/", views.delete),
]
```

Запустимо проєкт і звернемося до додатка в браузері. Спочатку додамо кілька об'єктів через форму на веб-сторінці. І після кожного додавання ми побачимо, як на веб-сторінці в таблиці з'являються нові дані:



Enter the Name

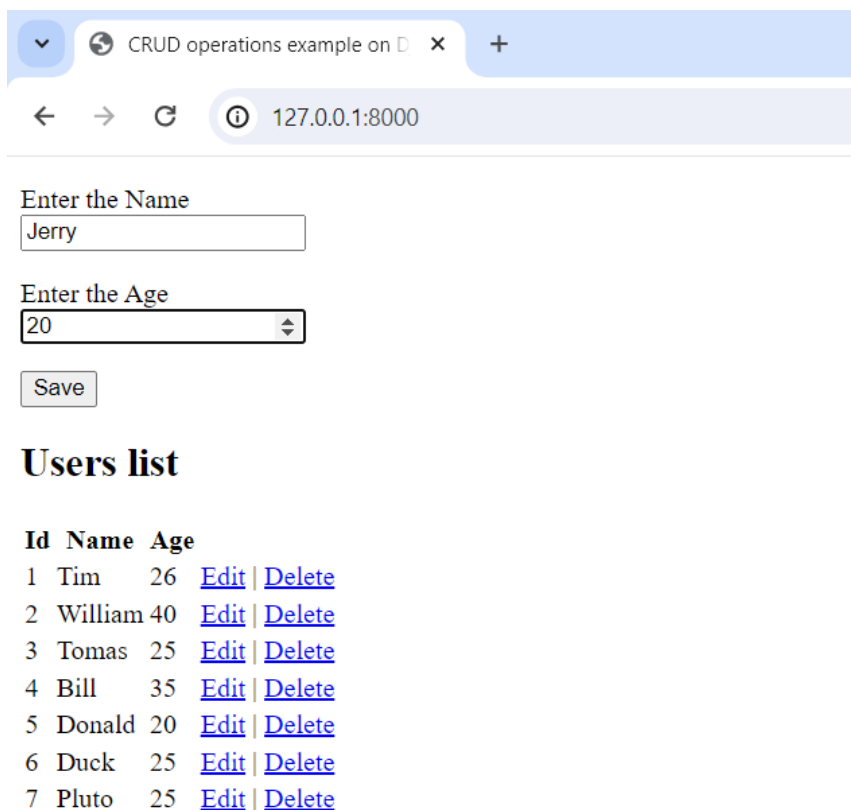
Enter the Age

Save

Users list

Id	Name	Age	
1	Tim	26	Edit Delete
2	William	40	Edit Delete
3	Tomas	25	Edit Delete
4	Bill	35	Edit Delete
5	Donald	20	Edit Delete
6	Duck	25	Edit Delete
7	Pluto	25	Edit Delete

Рисунок 2.30 – Головна сторінка вебдодатка



Enter the Name
Jerry

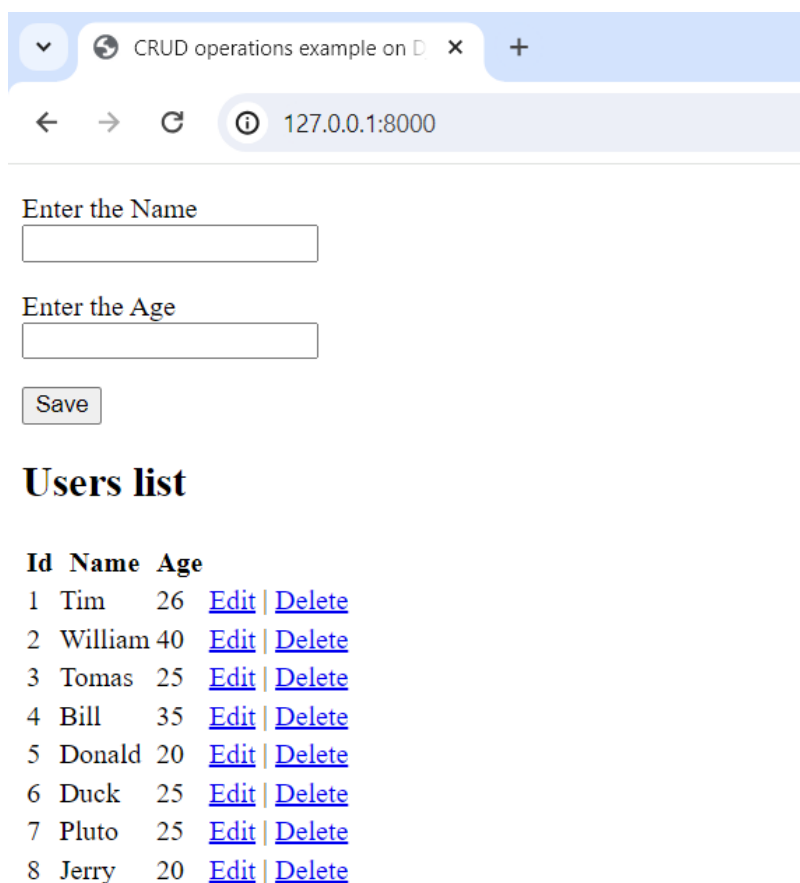
Enter the Age
20

Save

Users list

Id	Name	Age	
1	Tim	26	Edit Delete
2	William	40	Edit Delete
3	Tomas	25	Edit Delete
4	Bill	35	Edit Delete
5	Donald	20	Edit Delete
6	Duck	25	Edit Delete
7	Pluto	25	Edit Delete

Рисунок 2.31 – Додавання нового запису



Enter the Name

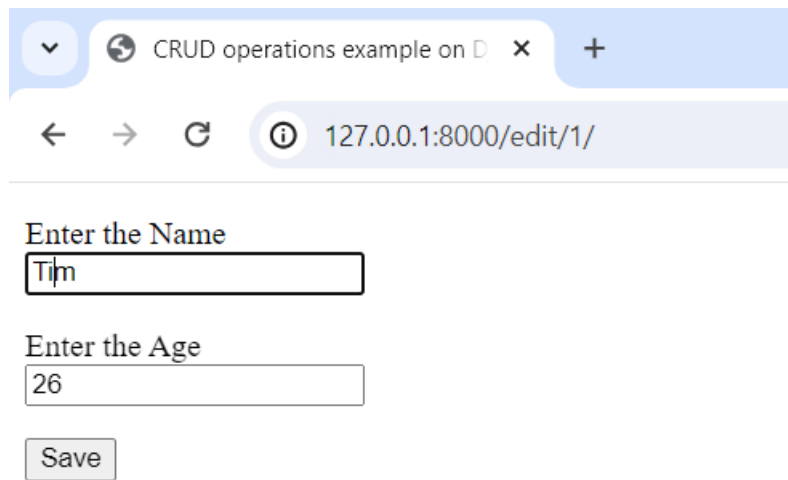
Enter the Age

Save

Users list

Id	Name	Age	
1	Tim	26	Edit Delete
2	William	40	Edit Delete
3	Tomas	25	Edit Delete
4	Bill	35	Edit Delete
5	Donald	20	Edit Delete
6	Duck	25	Edit Delete
7	Pluto	25	Edit Delete
8	Jerry	20	Edit Delete

Рисунок 2.32 – Результат доавання нового запису



CRUD operations example on D

127.0.0.1:8000/edit/1/

Enter the Name

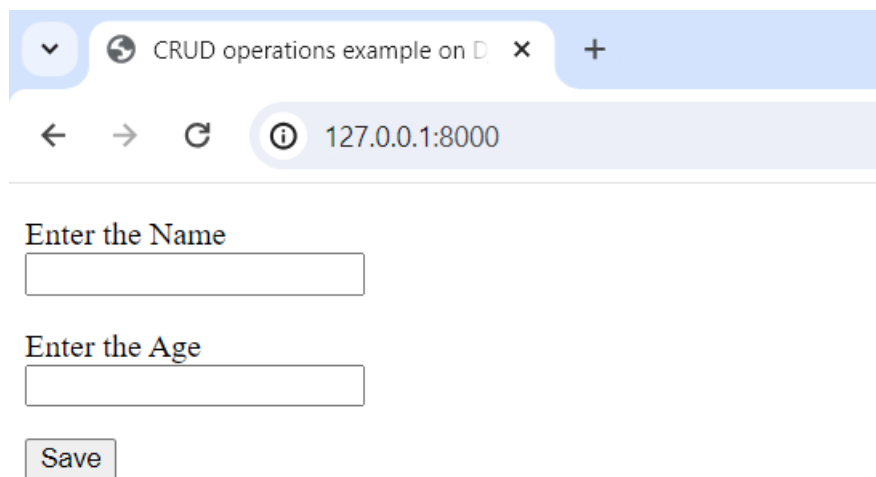
Tim

Enter the Age

26

Save

Рисунок 2.33 – Режим редагування запису (id = 1)



CRUD operations example on D

127.0.0.1:8000

Enter the Name

Enter the Age

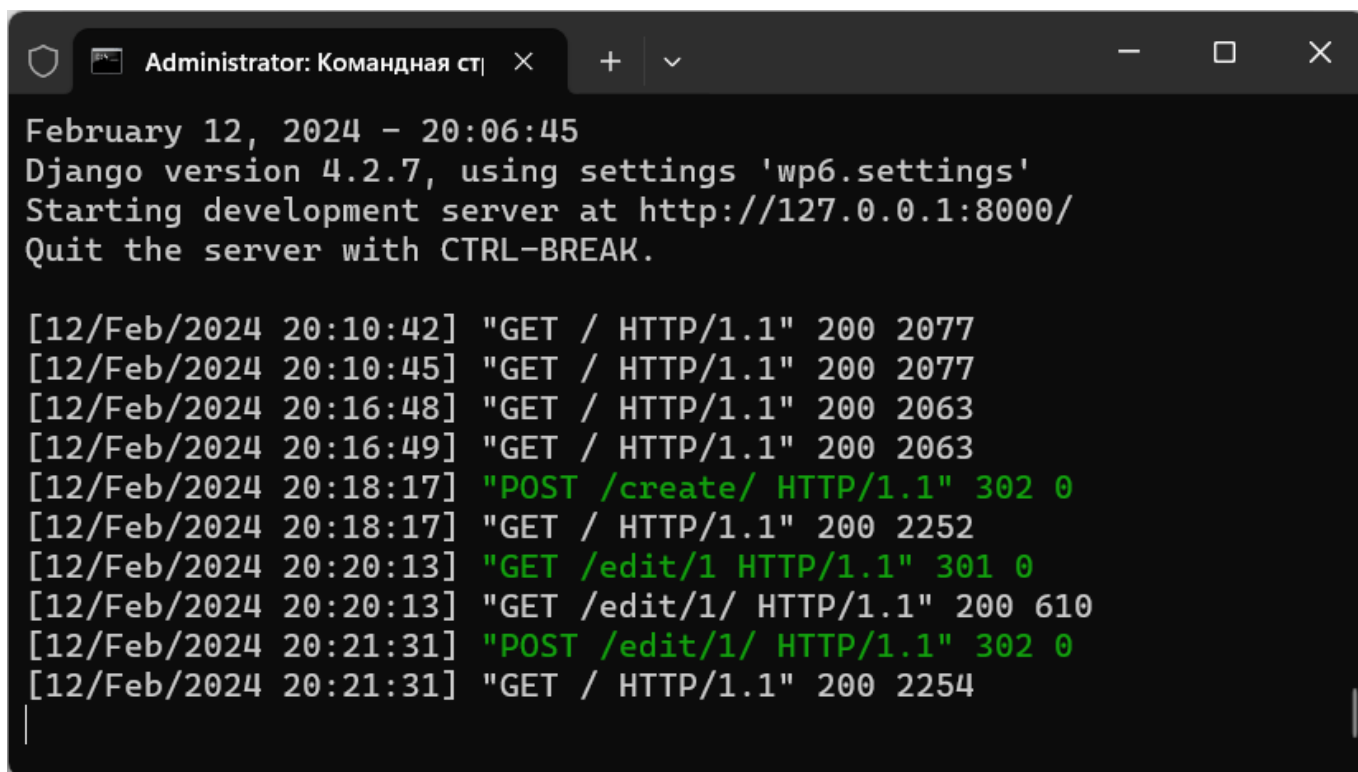
Save

Users list

Id	Name	Age	
1	Tomas	25	Edit Delete
2	William	40	Edit Delete
3	Tomas	25	Edit Delete
4	Bill	35	Edit Delete
5	Donald	20	Edit Delete
6	Duck	25	Edit Delete
7	Pluto	25	Edit Delete
8	Jerry	20	Edit Delete

Рисунок 2.34 – Результат оновлення запису (id = 1)

Відповідно натиснувши на кнопку видалення в таблиці об'єктів, ми видалимо обраний об'єкт.



```

Administrator: Командная строка
February 12, 2024 - 20:06:45
Django version 4.2.7, using settings 'wp6.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

[12/Feb/2024 20:10:42] "GET / HTTP/1.1" 200 2077
[12/Feb/2024 20:10:45] "GET / HTTP/1.1" 200 2077
[12/Feb/2024 20:16:48] "GET / HTTP/1.1" 200 2063
[12/Feb/2024 20:16:49] "GET / HTTP/1.1" 200 2063
[12/Feb/2024 20:18:17] "POST /create/ HTTP/1.1" 302 0
[12/Feb/2024 20:18:17] "GET / HTTP/1.1" 200 2252
[12/Feb/2024 20:20:13] "GET /edit/1 HTTP/1.1" 301 0
[12/Feb/2024 20:20:13] "GET /edit/1/ HTTP/1.1" 200 610
[12/Feb/2024 20:21:31] "POST /edit/1/ HTTP/1.1" 302 0
[12/Feb/2024 20:21:31] "GET / HTTP/1.1" 200 2254

```

Рисунок 2.35 – Лог операцій на сервері

2.13 Відношення один до багатьох (One to Many)

Розглянемо організацію зв'язку один до багатьох, за якої одна головна сутність може бути пов'язана з кількома залежними сутностями.

2.13.1 Розробка моделі

Наприклад, одна компанія може випускати кілька товарів:

```

from django.db import models

class Company(models.Model):
    name = models.CharField(max_length=30)

class Product(models.Model):
    company = models.ForeignKey(Company, on_delete = models.CASCADE)
    name = models.CharField(max_length=30)
    price = models.IntegerField()

```


У цьому випадку модель *Company* представляє виробника і є головною моделлю, а модель *Product* представляє товар компанії і є залежною моделлю.

Конструктор типу *models.ForeignKey* налаштовує зв'язок із головною сутністю. Перший параметр вказує, з якою моделлю буде створюватися зв'язок - у цьому випадку це модель *Company*. Другий параметр - *on_delete* задає опцію видалення об'єкта поточної моделі при видаленні пов'язаного об'єкта головної моделі. Загалом для параметра *on_delete* ми можемо використовувати такі значення:

- *models.CASCADE*: автоматично видаляє рядок із залежної таблиці, якщо видаляється пов'язаний рядок із головної таблиці
- *models.PROTECT*: блокує видалення рядка з головної таблиці, якщо з ним пов'язані будь-які рядки із залежної таблиці
- *models.SET_NULL*: встановлює *NULL* при видаленні пов'язаного рядка з головної таблиці
- *models.SET_DEFAULT*: встановлює значення за замовчуванням для зовнішнього ключа в залежній таблиці. У цьому випадку для цього стовпця має бути задано значення за замовчуванням
- *models.DO_NOTHING*: при видаленні пов'язаного рядка з головної таблиці не виконується жодних дій у залежній таблиці

І в результаті міграції в базі даних *SQLite* створюватимуться такі таблиці:

```
CREATE TABLE "lab_o2m_company" ("id" integer NOT NULL PRIMARY KEY
AUTOINCREMENT, "name" varchar(30) NOT NULL)
CREATE TABLE " lab_o2m_product" (
    "id" integer NOT NULL PRIMARY KEY AUTOINCREMENT,
    "name" varchar(30) NOT NULL,
    "price" integer NOT NULL,
    "company_id" bigint NOT NULL REFERENCES " lab_o2m_company" ("id")
DEFERRABLE INITIALLY DEFERRED
)
```

2.13.2 Операції з моделями

З визначення таблиць ми бачимо, що модель *Product* пов'язана з таблицею *Company* через стовпець "*company_id*". Однак у самому визначенні моделі *Product* є поле *company*, через яке можна отримати пов'язану сутність:

```
# отримання id пов'язаної з товаром компанії
Product.objects.get(id=1).company.id

# отримання назви пов'язаної з товаром компанії
Product.objects.get(id=1).company.name

# отримання товарів, які належать до компанії "Apple"
Product.objects.filter(company__name="Apple")
```

За допомогою виразу *модель__властивість* (два підкреслення) можна використовувати властивість головної моделі для фільтрації за об'єктами залежної моделі.

Хоча з точки зору моделі *Company* вона не має жодних властивостей, які пов'язували б її з моделлю *Product*. Але за допомогою синтаксису

```
"головна_модель"."залежна_модель"__set
```

можна змінити напрямок зв'язку. Наприклад:

```
from .models import Company, Product
apple = Company.objects.get(name="Apple")

# отримання всіх товарів
apple.product_set.all()

# отримання кількості товарів
apple.product_set.count()

# отримання товарів, назва яких починається на "iPhone"
apple.product_set.filter(name__startswith="iPhone")
```

Причому за допомогою виразу `_set` можна виконувати операції додавання, зміни, видалення об'єктів залежної моделі з головної моделі.

```
# створюємо об'єкт Company
apple = Company.objects.create(name="Apple")

# створення товару певної компанії
apple.product_set.create(name="iPhone 14", price=1999)

# окреме створення об'єкта з подальшим додаванням
ipad = Product(name="iPad", price=1500)
# при додаванні необхідно вказати параметр bulk =False
apple.product_set.add(ipad, bulk =False)

# виключає з компанії всі товари,
# при цьому товари залишаються в бд, просто їм не призначена компанія
# працює, якщо в залежній моделі ForeignKey(Company, null = True)
# apple.product_set.clear()

# те ж саме, тільки щодо одного об'єкта
# ipad = Product.objects.get(name="iPad")
# apple.product_set.remove(ipad)
```

Варто відзначити три методи:

add(): додає зв'язок між об'єктом залежної моделі та об'єктом головної моделі. За своєю суттю цей метод фактично викликає для моделі метод *update()* для додавання зв'язку. Однак це вимагає, щоб обидві моделі вже були в базі даних. І щоб обійти це обмеження, застосовується параметр *bulk=False*, для того, щоб об'єкт залежної моделі одразу було додано і для нього було встановлено зв'язок.

clear(): видаляє зв'язок між усіма об'єктами залежної моделі та об'єктом головної моделі. При цьому самі об'єкти залежної моделі залишаються в базі даних, і для їхнього зовнішнього ключа встановлюється значення *NULL*. Тому цей метод працюватиме, якщо в самій залежній моделі під час встановлення зв'язку використовували параметр *null=True: ForeignKey(Company, null = True)*.

remove(): також, як і *clear()*, видаляє зв'язок, тільки між одним об'єктом залежної моделі та об'єктом головної моделі. При цьому також всі об'єкти залишаються в БД. І також у самій залежній моделі під час встановлення зв'язку має використовуватися параметр *null=True*.

2.14 Практичний приклад зв'язку один до багатьох

Розглянемо, як зв'язок "один-до багатьох" може бути реалізовано в Django на найпростішому прикладі. Нехай у файлі *models.py* додатка визначено такі моделі:

```
from django.db import models

class Company(models.Model):
    name = models.CharField(max_length=30)

class Product(models.Model):
    company = models.ForeignKey(Company, on_delete = models.CASCADE)
    name = models.CharField(max_length=30)
    price = models.IntegerField()
```

Тут визначено моделі *Company* і *Product*, які пов'язані зв'язком "один-до-багатьох": одна компанія може мати безліч товарів.

У файлі *views.py* визначимо всі необхідні подання для роботи з об'єктами *Product*:

```
from django.shortcuts import render
from .models import Company, Product
from django.http import HttpResponseRedirect, HttpResponseNotFound

# отримання даних із БД
def index(request):
    products = Product.objects.all()
    return render(request, "index.html", {"products": products})

# додавання даних із БД
def create(request):
    create_companies() # додаємо початкові дані для компаній

    # якщо запит POST, зберігаємо дані
    if request.method == "POST":
        product = Product()
        product.name = request.POST.get("name")
        product.price = request.POST.get("price")
        product.company_id = request.POST.get("company")
        product.save()
        return HttpResponseRedirect("/")

    # передаємо дані в шаблон
    companies = Company.objects.all()
    return render(request, "create.html", {"companies": companies})

# зміна даних у БД
```

```

def edit(request, id):
    try:
        product = Product.objects.get(id=id)

        if request.method == "POST":
            product.name = request.POST.get("name")
            product.price = request.POST.get("price")
            product.company_id = request.POST.get("company")
            product.save()
            return HttpResponseRedirect("/")
        else:
            companies = Company.objects.all()
            return render(request, "edit.html", {"product": product, "companies":
companies})
    except Product.DoesNotExist:
        return HttpResponseRedirect("<h2>Product not found</h2>")

# видалення даних із БД
def delete(request, id):
    try:
        product = Product.objects.get(id=id)
        product.delete()
        return HttpResponseRedirect("/")
    except Product.DoesNotExist:
        return HttpResponseRedirect("<h2>Product not found</h2>")

# додавання початкових даних у таблицю компаній
def create_companies():

    if Company.objects.all().count() == 0:
        Company.objects.create(name = "Apple")
        Company.objects.create(name = "Asus")
        Company.objects.create(name = "MSI")
        Company.objects.create(name = "nvidia")
        Company.objects.create(name = "Intel")

```

Для спрощення додавання даних тут також визначено допоміжну функцію *create_companies()*, яка додає дані компаній до бази даних, щоб у нас були деякі початкові дані.

Функція *index()* отримує з бази даних усі об'єкти *Product* і передає їх у шаблон.

Функція *create()*, якщо запит типу *POST*, то отримує дані із запиту і зберігає їх у БД. Інакше отримує набір компаній і передає їх у шаблон для додавання.

Функція *edit()*, якщо запит типу *POST*, отримує дані запиту і змінює значення потрібного товару. Якщо запит типу *GET*, то передає в шаблон дані редагованого

товару і список компаній (щоб можна було змінити компанію товару - уявімо, що ми можемо змінити виробника товару)

Функція *delete()* видаляє товар.

У каталозі додатка визначимо папку *templates* і додамо в неї три шаблони: *index.html*, *create.html* і *edit.html*.

У шаблоні *index.html* визначимо логіку виведення списку товарів:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>One to Many example on Django</title>
</head>
<body>
  <h2>Products list</h2>
  <a href="create/">Adding product</a>
  <table>
    <thead><th>Name</th><th>Price</th><th>Company</th><th></th></thead>
    {% for product in products %}
    <tr>
      <td>{{ product.name }}</td>
      <td>{{ product.price }}</td>
      <td>{{ product.company.name }}</td>
      <td>
        <a href="edit/{{product.id}}">Edit</a> | <a
href="delete/{{product.id}}">Delete</a>
      </td>
    </tr>
    {% endfor %}
  </table>
</body>
</html>
```

У шаблоні *create.html* визначимо поля для додавання товару:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>One to Many example on Django</title>
</head>
<body>
  <h2>Adding product</h2>
  <form method="POST">
    {% csrf_token %}
    <p>
      <label>Enter product name</label><br>
      <input type="text" name="name" />
    </p>
    <p>
      <label>Enter price</label><br>
      <input type="number" name="price" />
    </p>
    <p>
      <label>Enter company</label><br>
      <select name="company">
        {% for company in companies %}
          <option value="{{company.id}}">{{company.name}}</option>
        {% endfor %}
      </select>
    </p>
    <input type="submit" value="Save" >
  </form>
</body>
</html>
```

У шаблоні *edit.html* визначимо поля для редагування товару:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>One to Many example on Django</title>
</head>
<body>
    <h2>Product editing</h2>
    <form method="POST">
        {% csrf_token %}
        <p>
            <label>Enter product name</label><br>
            <input type="text" name="name" value="{{product.name}}" />
        </p>
        <p>
            <label>Enter price</label><br>
            <input type="number" name="price" value="{{product.price}}"/>
        </p>
        <p>
            <label>Enter company</label><br>
            <select name="company">
                {% for company in companies %}
                    <option value="{{company.id}}"
                        {%if company.id == product.company_id%}
                            selected
                        {%endif%}
                    >{{company.name}}</option>
                {% endfor %}
            </select>
        </p>
        <input type="submit" value="Save" >
    </form>
</body>
</html>
```

У файлі *urls.py* пропишемо маршрути для подань:

```
from django.urls import path
from lab_o2m import views

urlpatterns = [
    path("", views.index),
    path("create/", views.create),
    path("edit/<int:id>/", views.edit),
    path("delete/<int:id>/", views.delete),
]
```


Під час запуску проєкту і звернення до головної сторінки за замовчуванням немає ніяких товарів. І для цього перейдемо до додавання і додамо який-небудь товар:

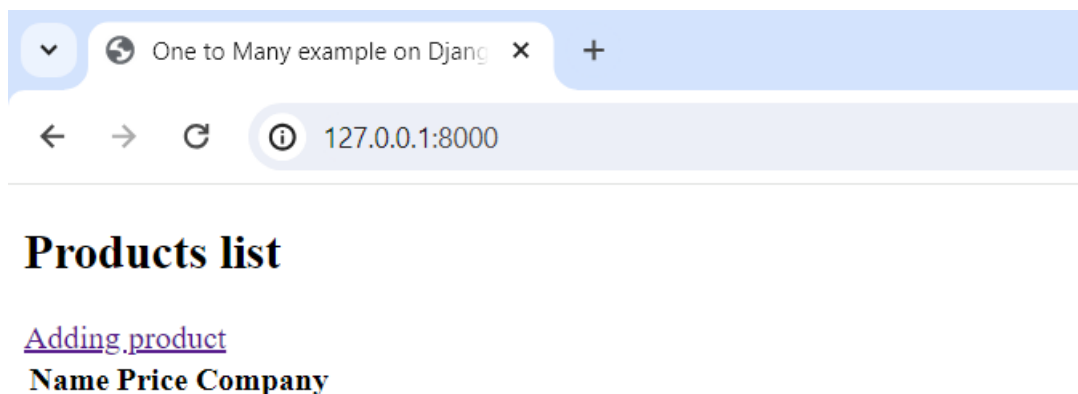


Рисунок 2.36 – Стартова сторінка вебдодатка

Adding product

Enter product name
MacBook

Enter price
15000

Enter company
Apple

Save

Рисунок 2.37 – Додавання нового товару

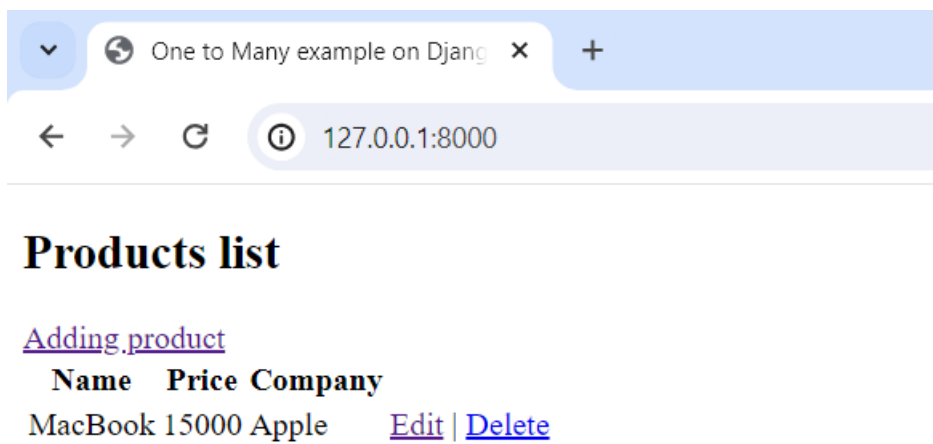


Рисунок 2.38 – Список товарів

2.15 Відношення багато до багатьох (Many to Many)

Зв'язок багато до багатьох описує ситуацію, коли об'єкт першої моделі може одночасно асоціюватися з кількома об'єктами другої моделі. І навпаки, один об'єкт другої моделі може також одночасно бути асоційований з кількома об'єктами першої моделі. Наприклад, один студент може відвідувати кілька курсів, а один курс можуть відвідувати кілька студентів.

Для створення відношення багато до багатьох застосовується тип *ManyToManyField*.

```
from django.db import models

class Course(models.Model):
    name = models.CharField(max_length=30)

class Student(models.Model):
    name = models.CharField(max_length=30)
    courses = models.ManyToManyField(Course)
```

У конструктор *models.ManyToManyField* передається сутність, з якою встановлюється відношення багато до багатьох. У результаті буде створюватися проміжна таблиця, через яку власне і буде здійснюватися зв'язок.

У результаті міграції в базі даних SQLite створюватимуться такі таблиці:

```
CREATE TABLE "lab_m2m_course" ("id" integer NOT NULL PRIMARY KEY
AUTOINCREMENT, "name" varchar(30) NOT NULL)
CREATE TABLE " lab_m2m _student" ("id" integer NOT NULL PRIMARY KEY
AUTOINCREMENT, "name" varchar(30) NOT NULL)
CREATE TABLE " lab_m2m _student_courses" (
    "id" integer NOT NULL PRIMARY KEY AUTOINCREMENT,
    "student_id" bigint NOT NULL REFERENCES "lab_m2m_student" ("id")
DEFERRABLE INITIALLY DEFERRED,
    "course_id" bigint NOT NULL REFERENCES "lab_m2m_course" ("id") DEFERRABLE
INITIALLY DEFERRED
)
```

У цьому випадку *"lab_m2m_student_courses"* виступає як сполучна таблиця. Вона називається за шаблоном ім'я_таблиці + ім'я_зв'язуючого_поля_з_таблиці.

2.15.1 Операції з моделями

Через властивість *courses* у моделі *Student* ми можемо отримувати пов'язані зі студентом курси та керувати ними.

```
# створимо студента
tom = Student.objects.create(name="Alex")

# створимо один курс і додамо його до списку курсів Алекса
tom.courses.create(name="Programing")

# отримаємо всі курси студента
courses = Student.objects.get(name="Alex").courses.all()

# отримуємо всіх студентів, які відвідують курс Алгебра
students = Student.objects.filter(courses__name="Programing")
```

Варто відзначити останній випадок, де проводиться фільтрація студентів за відвідуваним курсом. Для передачі в метод *filter()* назви курсу використовується параметр, назва якого починається з назви властивості, через яку йде зв'язок із другою моделлю. І далі через два знаки підкреслення вказується ім'я властивості другої моделі, наприклад, *courses__name* або *courses__id*.

У цьому випадку ми можемо отримати інформацію про курси студента через властивість *courses*, яка визначена в моделі *Student*. Однак що якщо ми хочемо

отримати інформацію про студентів за певним курсом? У цьому випадку нам треба використовувати синтаксис `_set`.

```
# створимо курс
python = Course.objects.create(name="Python")

# створюємо студента і додаємо його на курс
python.student_set.create(name="Bob")

# окремо створюємо студента і додаємо його на курс
sam = Student(name="Sam")
sam.save()
python.student_set.add(sam)

# отримуємо всіх студентів курсу
students = python.student_set.all()

# отримуємо кількість студентів за курсом
number = python.student_set.count()

# знімаємо з курсу одного студента
python.student_set.remove(sam)

# видаляємо всіх студентів з курсу
python.student_set.clear()
```

2.15.2 Визначення проміжної сутності

Варто враховувати, що вищевказана організація моделей не завжди може підійти. Наприклад, у цьому випадку створюється проміжна таблиця, яка зберігає *id* студента та *id* курсу. Якщо нам треба в проміжній таблиці зберігати ще якісь дані, наприклад, дату зарахування студента на курс, його оцінку тощо, то така конфігурація не підійде. І в цьому випадку є кілька можливих варіантів. Найпростіший із них - створити проміжну модель вручну, яка зберігатиме всі додаткові атрибути (наприклад, оцінку студента за курсом, дату зарахування) і яка буде пов'язана відношенням один до багатьох з обома моделями.

Але Django також надає інший спосіб, за якого обидві значущі моделі залишаються пов'язаними відношенням багато до багатьох, і водночас також визначається допоміжна проміжна модель. Так, змінимо вище визначені моделі *Course* і *Student* таким чином:

```

from django.db import models

class Course(models.Model):
    name = models.CharField(max_length=30)

class Student(models.Model):
    name = models.CharField(max_length=30)
    courses = models.ManyToManyField(Course, through="Зарахування")

class Enrollment(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    date = models.DateField() # дата вступу
    mark = models.IntegerField(max_value=5) # отриманий бал

```

Моделі *Student* і *Course* залишаються пов'язаними відношенням багато-ко-множини, однак тепер ми явно визначаємо проміжну модель - *Enrollment*, яка пов'язана відношенням один-ко-множини з обома моделями та при цьому також визначає два додаткових поля. Для зв'язку з проміжною моделлю в конструктор *ForeignKey* передається параметр *through*, який вказує на назву проміжної таблиці, що створюється для проміжної моделі

У випадку з SQLite ми отримаємо такі таблиці:

```

CREATE TABLE "lab_m2m_course" ("id" integer NOT NULL PRIMARY KEY
AUTOINCREMENT, "name" varchar(30) NOT NULL)
CREATE TABLE "lab_m2m_student" ("id" integer NOT NULL PRIMARY KEY
AUTOINCREMENT, "name" varchar(30) NOT NULL)
CREATE TABLE "lab_m2m_enrollment" (
    "id" integer NOT NULL PRIMARY KEY AUTOINCREMENT,
    "date" date NOT NULL,
    "mark" integer NOT NULL,
    "course_id" bigint NOT NULL REFERENCES "lab_m2m_course" ("id") DEFERRABLE
INITIALLY DEFERRED,
    "student_id" bigint NOT NULL REFERENCES "lab_m2m_student" ("id")
DEFERRABLE INITIALLY DEFERRED
)

```

2.15.3 Використання проміжної моделі

```
# створимо курс
python = Course.objects.create(name="Python")

# створимо двох студентів
tom = Student.objects.create(name="Tom")
sam = Student.objects.create(name="Sam")

# створюємо надходження студентів на курс
tom_python = Enrollment(student = tom, course= python,
                        date=date.today(), mark=5)
sam_python = Enrollment(student = sam, course= python,
                        date=date.today(), mark = 4)
# зберігаємо надходження в бд
tom_python.save()
sam_python.save()

# отримуємо всі курси у Тома
tom_courses = tom.courses.all()
print(tom_courses[0].name) # python

# отримуємо всіх студентів курсу за python
python_students = python.student_set.all()
print(python_students[0].name) # Tom
```

Подібним чином ми можемо викликати в об'єктів моделей методи *add()*, *create()* і *set()* для встановлення відношення з об'єктів іншої моделі. Але при цьому необхідно передати через параметр *through_defaults* значення для полів проміжної моделі, які вимагають наявності значення.

```
# створимо курси
django = Course.objects.create(name="Django")
python = Course.objects.create(name="Python")
java = Course.objects.create(name="Java")

# створимо студента
bob = Student.objects.create(name="Bob")

# додаємо курс для студента bob
bob.courses.add(django, through_defaults={"date": date.today(), "mark": 5})
# створюємо курс для студента bob
bob.courses.create(name="C++", through_defaults={"date": date.today(),
"mark": 4})

# отримуємо всі курси Boba
print(bob.courses.all().values_list()) # <QuerySet [(11, 'Django'), (14,
'C++')]>

# перевстановлюємо курси для студента bob
bob.courses.set([python, java], through_defaults={"date": date.today(),
"mark": 4})

# знову отримуємо всі курси Boba
print(bob.courses.all().values_list()) # <QuerySet [(12, 'Python'), (13,
'Java')]>
```

Також за допомогою методів *remove()* і *clear()* можна видалити зв'язок однієї моделі з іншою:

```
# створимо курси
django = Course.objects.create(name="Django")
python = Course.objects.create(name="Python")

# створимо студента
tim = Student.objects.create(name="Tim")

# встановлюємо курси для студента Tim
tim.courses.set([python, django], through_defaults={"date": date.today(),
"mark": 4})

print(tim.courses.all().values_list()) # <QuerySet [(16, 'Python'), (15,
'Django')]>

# видаляємо один курс
tim.courses.remove(django)
print(tim.courses.all().values_list()) # <QuerySet [(16, 'Python')]>

# видаляємо всі курси
tim.courses.clear()
print(tim.courses.all().values_list()) # <QuerySet []>
```

Також ми можемо застосовувати фільтрацію за полями всіх трьох моделей. Наприклад, знайдемо всіх студентів, у яких курс - *Python*, а оцінка - 4 або нижче:

```
# фільтрація
students = Student.objects.filter(
    courses__name="Python",
    enrollment__mark__lte=4)

print(students.values_list())
```

2.16 Практичний приклад зв'язку багато до багатьох

Розглянемо найпростіший приклад додатка з моделями зі зв'язком багато до багатьох. Нехай у файлі *models.py* будуть визначені такі моделі:

```
from django.db import models

class Course(models.Model):
    name = models.CharField(max_length=30)

class Student(models.Model):
    name = models.CharField(max_length=30)
    courses = models.ManyToManyField(Course, through="Enrollment")

class Enrollment(models.Model):
    student = models.ForeignKey(Student, on_delete=models.CASCADE)
    course = models.ForeignKey(Course, on_delete=models.CASCADE)
    date = models.DateField() # дата вступу
    mark = models.IntegerField() # отриманий бал
```

Тут моделі *Student* і *Course* пов'язані зв'язком багато до багатьох через проміжну модель *Enrollment*.

У файлі `views.py` визначимо пару функцій-представлень для виведення і створення об'єктів *Student*:

```
from datetime import date
from django.shortcuts import render
from .models import Student, Course
from django.http import HttpResponseRedirect

# отримання даних із бд
def index(request):
    # фільтрація
    students = Student.objects.all()
    return render(request, "index.html", {"students":students})

# додавання даних у бд
def create(request):
    initialize()
    # якщо запит POST, зберігаємо дані
    if request.method == "POST":
        student = Student()
        student.name = request.POST.get("name")
        course_ids = request.POST.getlist("courses")
        student.save()
        # отримуємо всі обрані курси за їхніми id
        courses = Course.objects.filter(id__in=course_ids)
        student.courses.set(courses, through_defaults={"date": date.today(), "mark":0})
        return HttpResponseRedirect("/")
    # передаємо дані в шаблон
    courses = Course.objects.all()
    return render(request, "create.html", {"courses": courses})

def initialize():
    # Student.objects.all().delete()
    # Course.objects.all().delete()
    if Course.objects.all().count() == 0:
        Course.objects.create(name = "Python")
        Course.objects.create(name = "Django")
        Course.objects.create(name = "FastAPI")
```

Тут визначено допоміжну функцію `initialize()`, яка запускається під час старту додатка і додає в таблицю курсів, якщо таблиця порожня, три курси.

Функція `index()` отримує список студентів і передає їх у подання `index.html`.

Функція `create()`, якщо запит типу *POST*, отримує дані із запиту. Передбачається, що на сервер будуть приходити *id* курсів. Відповідно після додавання студента для

встановлення для нього курсів спочатку знаходимо за *id* ці курси в базі даних. Потім знайдені курси передаємо студенту через метод *student.courses.set()*. Якщо запит має тип GET, то повертаємо шаблон *create.html* і передаємо в нього список курсів.

Далі визначимо в каталозі додатка папку *templates*, а в неї додамо файл *index.html* для виведення всіх студентів:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Many to Many example on Django</title>
</head>
<body>
    <h2>Students list</h2>
    <a href="create/">Add student</a>
    <table>
        <thead><th>Name</th><th>Courses</th></thead>
        {% for student in students %}
        <tr>
            <td>{{ student.name }}</td>
            <td>
                {% for course in student.courses.all %}
                {{course.name}}
                {% endfor %}
            </td>
        </tr>
        {% endfor %}
    </table>
</body>
</html>
```

І також у папці *templates* визначимо шаблон *create.html* для додавання студента:

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8" />
    <title>Many to Many example on Django</title>
</head>
<body>
    <h2>Add student</h2>
    <form method="POST">
        {% csrf_token %}
        <p>
            <label>Enter Name</label><br>
            <input type="text" name="name" />
        </p>
        <p>
            <label>Select courses</label><br>
            <div>
                {% for course in courses %}
                    <input type="checkbox" name="courses" value="{{course.id}}" />{{
course.name}} <br />
                {% endfor %}
            </div>
        </p>
        <input type="submit" value="Save" >
    </form>
</body>
</html>
```

Під час додавання студента буде доступний вибір курсів. Для кожного курсу буде створено елемент `<input type="checkbox">`. Його властивість *value* зберігатиме *id* відповідного курсу. І таким чином при виборі певних курсів на сервер будуть відправлятися їх *id*.

Наприкінці у файлі *urls.py* налаштуємо маршрути:

```
from django.urls import path
from lab_m2m import views

urlpatterns = [
    path("", views.index),
    path("create/", views.create),
]
```

Додамо одного студента:

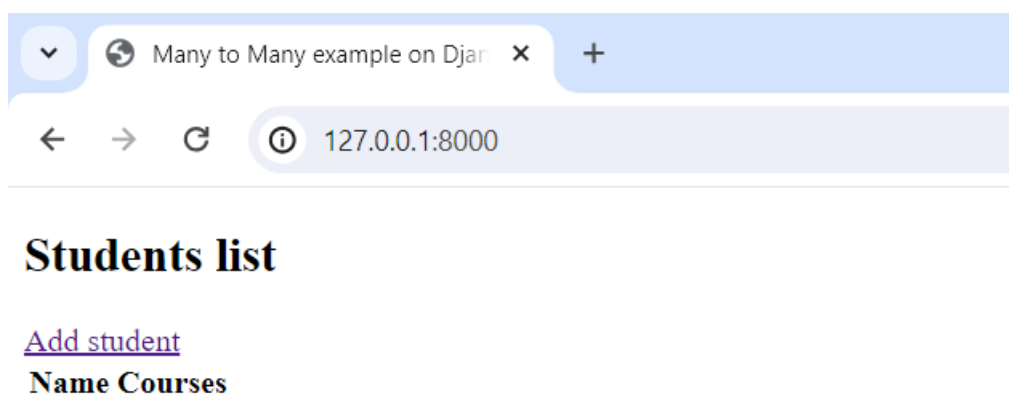


Рисунок 2.39 – Головна сторінка вебзастосунку

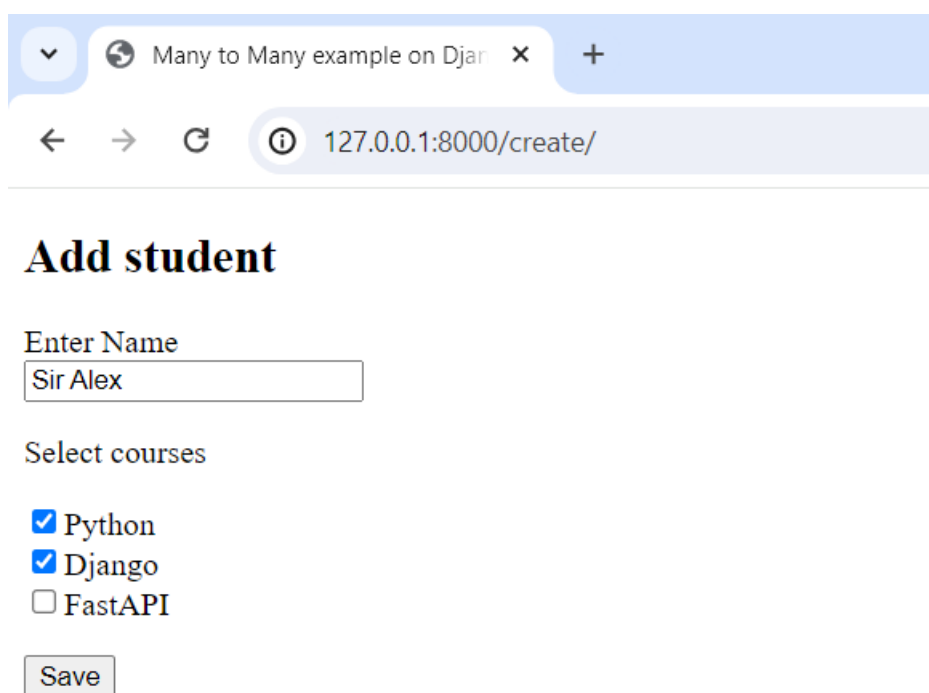


Рисунок 2.40 – Додавання нового студента

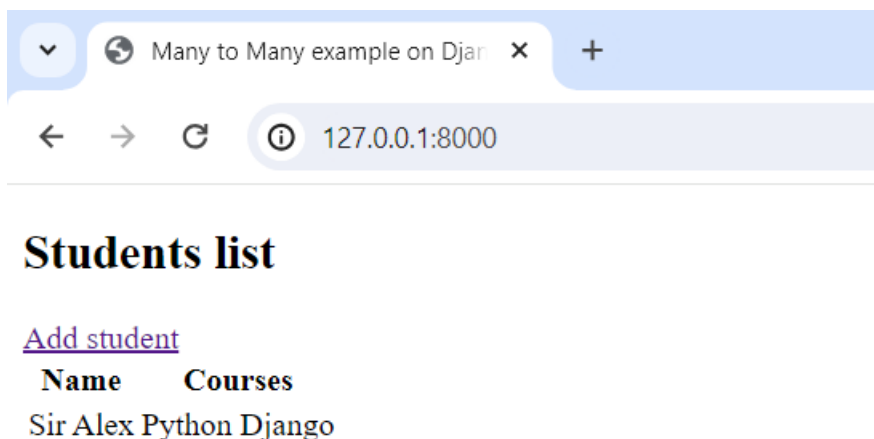


Рисунок 2.41 – Оновлений перелік студентів

Подібним чином можна зробити редагування та інші операції з подібними моделями.

2.17 Відношення один до одного (One to one)

Відношення один до одного передбачає, що один рядок з однієї таблиці може бути пов'язаний тільки з однією сутністю з іншої таблиці.

2.17.1 Створення моделі

Наприклад, користувач може мати будь-які дані, які описують його облікові дані. Усю базову інформацію про користувача, типу імені, віку, можна виділити в одну модель, а облікові дані - логін, пароль, час останнього входу в систему, кількість невдалих входів тощо. - в іншу модель:

```

from django.db import models

class User(models.Model):
    name = models.CharField(max_length=20)

class Account(models.Model):
    login = models.CharField(max_length=20)
    password = models.CharField(max_length=20)
    user = models.OneToOneField(User, on_delete = models.CASCADE, primary_key
= True)

```

Для створення цього відношення один до одного застосовується конструктор типу *models.OneToOneField()*. Його перший параметр вказує, з якою моделлю буде асоційована дана сутність (у цьому випадку асоціація з моделлю *User*). Другий параметр *on_delete = models.CASCADE* говорить, що дані поточної моделі (*Account*) будуть видалятися в разі видалення пов'язаного об'єкта головної моделі (*User*). Третій параметр *primary_key = True* вказує, що зовнішній ключ (через який йде зв'язок із головною моделлю) водночас буде виступати і як первинний ключ. І відповідно створювати окреме поле для первинного ключа не треба.

У результаті міграції в базі даних SQLite створюватимуться такі таблиці:

```

CREATE TABLE "lab_o2o_user" (
    "id" integer NOT NULL PRIMARY KEY AUTOINCREMENT,
    "name" varchar(20) NOT NULL
)
CREATE TABLE "lab_o2o_account" (
    "login" varchar(20) NOT NULL,
    "password" varchar(20) NOT NULL,
    "user_id" bigint NOT NULL PRIMARY KEY REFERENCES "lab_o2o_user" ("id")
DEFERRABLE INITIALLY DEFERRED
)

```

2.17.2 Операції з моделями

За допомогою властивості *user* у моделі *Account* ми можемо маніпулювати пов'язаним об'єктом моделі *User*:

```
# створимо користувача
sam = User.objects.create(name="Sam")

# створимо акаунт користувача Sam
acc = Account.objects.create(login = "1234", password="6565", user=sam)

# змінюємо ім'я користувача
acc.user.name = "Bob"
# зберігаємо зміни в бд
acc.user.save()

print(f"{acc.user.name}, login: {acc.login}, password: {acc.password}")
# Bob, login: 1234, password: 6565
```

Однак через модель *User* ми також можемо впливати на пов'язаний об'єкт *Account*. Незважаючи на те, що явним чином у моделі *User* визначено тільки одну властивість - *name*, під час зв'язку один до одного неявно створюється ще одна властивість, що називається на ім'я залежної моделі та вказує на пов'язаний об'єкт цієї моделі. Тобто в даному випадку ця властивість буде називатися "*account*":

```
# створимо користувача
tom = User.objects.create(name="Tom")

# створимо акаунт користувача
acc = Account(login = "1234", password="6565")
tom.account = acc
tom.account.save()

# оновлюємо дані
tom.account.login = "qwerty"
tom.account.password = "123456"
tom.account.save()
print(f"{tom.name}, login: {tom.account.login}, password: {tom.account.password}")
# Tom, login: qwerty, password: 123456
```

Подібним чином можна виконувати фільтрацію за обома моделями та їхніми властивостями:

```
# отримаємо користувача
tom = User.objects.get(name="Tom")

# отримаємо акаунт користувача
tom_acc = Account.objects.get(user=tom)
print(f "login: {tom_acc.login}, password: {tom_acc.password}")
# login: qwerty, password: 123456

# отримаємо акаунт за ім'ям користувача
bob_acc = Account.objects.get(user__name="Bob")
print(f "login: {bob_acc.login}, password: {bob_acc.password}")

# отримаємо користувача за логіном
user = User.objects.get(account__login="qwerty")
print(user.name)
```